# ARM PrimeCell

## Smart Card Interface (PL131)

## **Design Manual**

**ARM PrimeCell Smart Card Interface (PL131) Design Manual**



## Proprietary notice



## Document confidentiality status

This document is ARM Confidential (Distributed to ARM staff, product licensees & NDA signatories only).

## Product status

The information in this document is Final (information on a developed product).

## Web address

http://www.arm.com

## Feedback

ARM Limited welcomes feedback on both the product, and the documentation.

## Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

## Feedback on the ARM PrimeCell Smart Card Interface

If you have any comments or suggestions about this product, please contact your supplier giving:

- The Product name
- A concise explanation of your comments.

# Contents

**1 ABOUT THIS DOCUMENT 1**

**1.1 Change history 1**

**1.2 References 1**

**1.3 Terms and abbreviations 2**

**2 SCOPE 3**

**3 INTRODUCTION 3**

**4 SCI DESIGN OVERVIEW 4**

**4.1 Functional Description 4**

**4.2 Signal Description 5**

**4.3 Programmer's Model 7**
4.3.1 Functional Mode Registers 7
4.3.2 Test Mode Registers 10
4.3.3 Register Memory Map 11

**4.4 Block Partitioning 12**
4.4.1 APB Interface 12
4.4.2 Register Block 15
4.4.3 Transmit and Receive logic 16
4.4.4 Interrupt control logic 19
4.4.5 SCI Control logic 20
4.4.6 Transmit FIFO 22
4.4.7 Receive FIFO 23
4.4.8 DMA Interface 26
4.4.9 Test logic 27
4.4.10 Synchronisers 28
4.4.11 Revision Designator block 31

**5 SCI IMPLEMENTATION DETAILS 33**

**5.1 Design Hierarchy 33**

**5.2 SCI Top Level Block (Sci) 33**

**5.3 SCI APB Interface (SciApbif) 34**
5.3.1 Write Interface 34
5.3.2 Read Interface 34

**5.4 SCI Register Block (SciRegBlock) 34**
5.4.1 Generation of Interrupt Clear signals 34

5.4.2 Synchronisation of SCI registers to the SCICLK domain 35

**5.5 SCI Register Update Block (SciRegBlkUpdate) 35**

**5.6 SCI Transmit Receive logic (SciTxRx) 35**
5.6.1 Card Activation Sequence 36
5.6.2 Receive Logic 37
5.6.3 Transmit Logic 39
5.6.4 Clock Stop Mode 40
5.6.5 Card Activation / Deactivation control signal generation 41
5.6.6 Card Deactivation Sequence 41

**5.7 SCI Interrupt Generation Block (SciIntGen) 42**

**5.8 SCI Control State Machine (SciCntl) 42**

**5.9 Transmit FIFO (SciTxFIFO) 51**

**5.10 Transmit FIFO control logic (SciTXFCntl) 51**

**5.11 Transmit FIFO Register File (SciTXRegFile) 53**

**5.12 Receive FIFO (SciRxFIFO) 53**

**5.13 Receive FIFO Controller (SciRXFCntl) 53**

**5.14 Receive FIFO Register File (SciRxRegFile) 54**

**5.15 Synchronisers to the SCICLK domain (SciSynctoSCICLK) 54**

**5.16 Synchronisers to the PCLK domain (SciSynctoPCLK) 55**

**5.17 DMA Interface (SciDMA) 55**

**5.18 Test logic (SciTest) 56**
5.18.1 Integration testing of block inputs 57
5.18.2 Integration testing of block outputs 58

**6 DESIGN ASSUMPTIONS 61**

**6.1 Software Assumptions 61**
6.1.1 Response to the reception of the ATR initial (TS) character 61

**6.2 Hardware Assumptions 61**
6.2.1 Selection of Card Operating voltages is through external hardware 61

**7 SCI FUNCTIONAL VERIFICATION DETAILS 61**

**7.1 SCI VHDL Verification Testbench 61**
7.1.1 Unit Under Test 64
7.1.2 APB Bridge Model 64
7.1.3 Trickbox 64
7.1.4 Protocol Checker 64

7.1.5 APB Multiplexer 65
7.1.6 Test Vectors 65
7.1.7 Generics 65
7.1.8 Running Simulations 66

**7.2 SCI Verilog Verification Testbench 67**
7.2.1 Unit Under Test 69
7.2.2 APB Bridge Model 69
7.2.3 Trickbox 69
7.2.4 Protocol Checker 70
7.2.5 APB Multiplexer 70
7.2.6 Vector Sets 70
7.2.7 Parameters 70
7.2.8 Running Simulations 71

**7.3 SCI Trickbox 72**
7.3.1 Trickbox Signal Description 73
7.3.2 Trickbox functional description 74
7.3.3 Trickbox Register Descriptions 76

**7.4 Functional Verification Tests 82**
7.4.1 Register Tests 82
7.4.2 Debounce Test 82
7.4.3 Activation-Deactivation sequence Test 82
7.4.4 ATR Test 83
7.4.5 Receive non back-to-back Test 83
7.4.6 Receive back to back Test 83
7.4.7 Receive interrupt flag Test 83
7.4.8 Receive parity Test 83
7.4.9 Character timeout interrupt Test 83
7.4.10 Block time out interrupt Test 84
7.4.11 Synchronous mode transmit Test 84
7.4.12 Synchronous mode receive Test 84
7.4.13 Non EMV Test 84
7.4.14 Scan mode Test 84
7.4.15 CLKZ1 Test 84
7.4.16 Transmit non back to back Test 85
7.4.17 Transmit back to back Test 85
7.4.18 Transmit Interrupt Flag Test 85
7.4.19 Transmit parity Test 85
7.4.20 Transmit character guard Test 85
7.4.21 TXERRINTR Test 85
7.4.22 DMA Interface Transmit tests 85
7.4.23 DMA Interface Receive tests 86
7.4.24 Clock Stop Mode tests 86
7.4.25 Receive Over Run tests 87
7.4.26 Additional code coverage state machine tests 87

**8 SCI INTEGRATION TEST DETAILS 88**

**8.1 SCI VHDL Integration testbench 89**
8.1.1 Unit Under Test 91
8.1.2 Test Vectors 91
8.1.3 Running Simulations 91

**8.2 SCI Verilog Integration testbench 92**
8.2.1 Unit Under Test 94
8.2.2 Test Vectors 94
8.2.3 Running Simulations 94

**8.3 SCI Integration Trickbox 94**

**8.4 Integration Tests 94**
8.4.1 Integration Testing of Intra-Chip and Primary Inputs 94
8.4.2 Integration Testing of Intra-Chip and Primary Outputs 94

# 1 ABOUT THIS DOCUMENT

## 1.1 Change history

| Issue | Date | Change |
|---|---|---|
| A | 25th January, 2002 | First release. |

## 1.2 References

This document refers to the following documents.

| Ref | Doc No | Title |
|---|---|---|
| 1 | ARM-DDI-0228A | ARM PrimeCell Smart Card Interface (PL131) Technical Reference Manual |
| 2 | PL131 INTM 0000 | ARM PrimeCell Smart Card Interface (PL131) Integration Manual |
| 3 | ISO 7816-3 | Information technology - Identification cards - Integrated circuit(s) card(s) with contacts (Second edition 1997-12-15) |
| | | Part3: Electronic signals and transmission protocols |
| 4 | ISO 7816-10 | Identification cards – Integrated circuit(s) cards with contacts (First edition 1999-11-01) |
| | | Part 10: Electronic Signals and transmission cards. |
| 5 | EMV ’96 | Integrated Circuit Card, Specifications for Payment Systems (Version 3.1.1 May 31, 1998) hereby referred as ‘EMV Standard’. Part I – electromechanical characteristics, logical interface and transmission protocols. |
| | | Published by Europay International S.A, Mastercard International Inc, and Visa International Association. |
| 6 | ARM DDI 0138 | Example AMBA System Technical Reference Manual |
| 7 | ARM DUI 0092 | Example AMBA System User Guide |
| 8 | ARM PL000 USGD 0000 | ARM PrimeCell Generic Peripheral User Guide |
| 9 | SCB00-QPRO-0008-A03 | PrimeCell Integration Vector Strategy |

## 1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

| Term | Meaning |
|---|---|
| AMBA | Advanced Micro-controller Bus Architecture |
| APB | Advanced Peripheral Bus |
| ASB | Advanced System Bus |
| ATPG | Automatic Test Pattern Generation |
| etu | Elementary Time Unit |
| IP | Intellectual Property |
| PMU | Power Management Unit |
| RTL | Register Transfer Level |
| SCI | Smart Card Interface |

# 2 SCOPE

This document describes the design and implementation of the ARM PrimeCell Smart Card Interface (PL131). The design description is split between two sections – Section 4 and Section 5.

Section 4 presents an overview of the SCI design, describing the functional partitioning of the SCI and the signal interconnections. Section 5 presents detailed descriptions on the SCI design.

The test bench and the test programs used to generate test vectors for functional verification of the SCI are contained within Section 6.

# 3 INTRODUCTION

The SCI is a part of a library of reusable IP blocks, which can be more easily integrated into a typical ASIC design flow.

For further information on this block refer to the ARM PrimeCell Smart Card Interface (PL131) Technical Reference Manual [Ref.1].

For guidance on integrating this block within a System-On-Chip design using a typical industry standard design flow, refer to the ARM PrimeCell Smart Card Interface (PL131) Integration Manual [Ref.2].

For details about signals and transmission protocols of Asynchronous Integrated Circuit Cards with contacts, refer to the ISO/IEC 7816-3 Specification [Ref.3].

For details about signals and transmission protocols of Synchronous Integrated Circuit Cards with contacts, refer to the ISO/IEC 7816-10 Specification [Ref.4].

For details about implementing payment systems using Integrated Circuit Cards, refer to the EMV ’96 standard [Ref.5].

The Example AMBA System Technical Reference Manual [Ref.6] and User Guide [Ref.7] describe an example micro-controller based on an ARM processor and the low-power, generic design methodology of AMBA. Further information can also be found on use of TICTalk test vectors for integration test.

# 4 SCI DESIGN OVERVIEW

## 4.1 Functional Description

The ARM PrimeCell SCI provides an interface to an external Smart Card reader. The SCI can autonomously control data transfer to and from the Smart card. Transmit and receive paths are buffered with 8 byte internal FIFOs and there is support for Direct Memory Access (DMA). The ARM PrimeCell SCI supports asynchronous T0 and T1 protocols, with limited support for synchronous cards via registered I/0. It also supports programmable clock rate conversion and bit rate adjustment factors. A block diagram of the device is shown in ***Figure 4.1-1***.

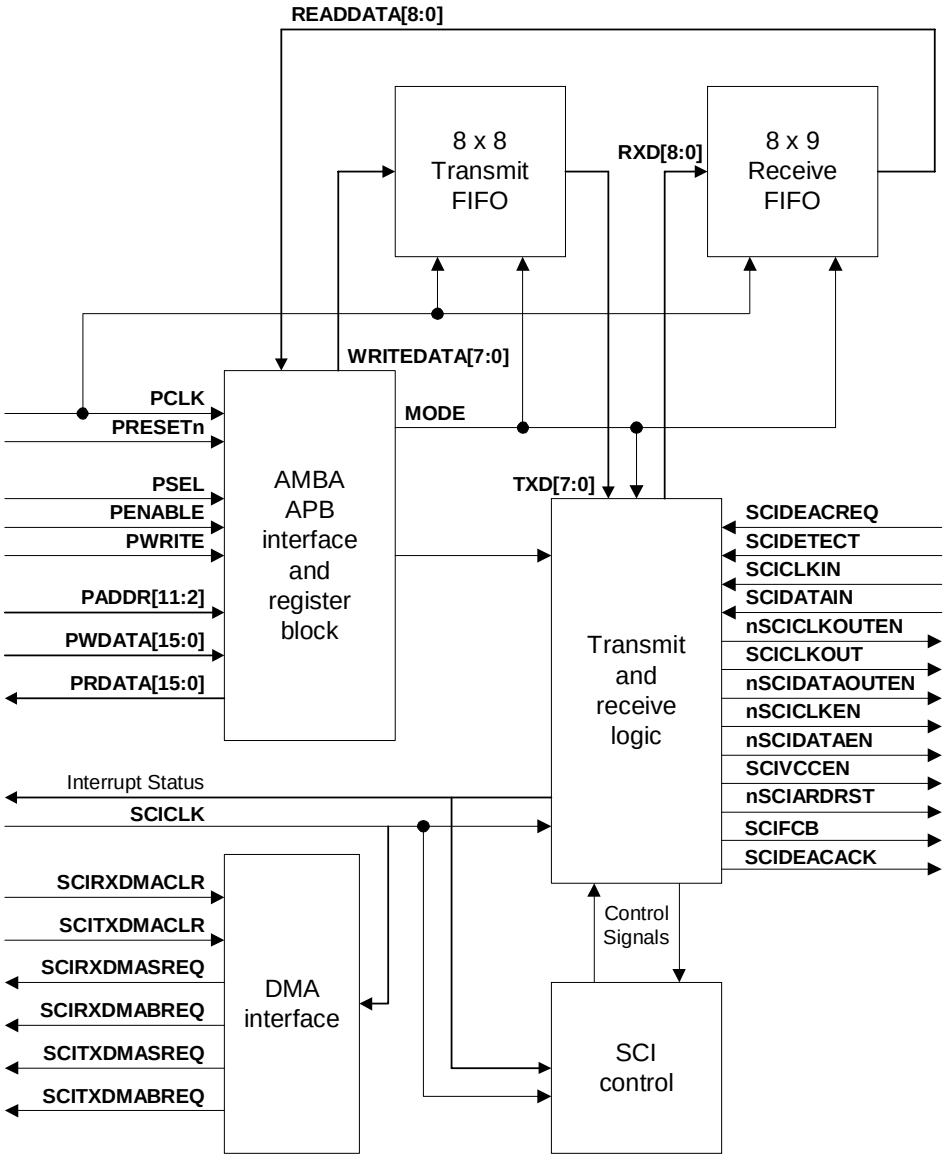


***Figure 4.1-1 ARM PrimeCell SCI Block Diagram***

## 4.2 Signal Description

The following tables summarise the SCI signal list. ***Table 4.2-1*** lists the APB interface signals, ***Table 4.2-2*** lists the block-specific non-APB signals and ***Table 4.2-3*** lists the I/O pad signals.

| Signal Name | Type | Source / Destination | Description |
|---|---|---|---|
| PRESETn | IN | Reset Controller | Bus reset (active LOW). |
| PADDR [11:2] | IN | APB Bridge | Subset of the APB address bus. |
| PCLK | IN | Clock Generator | APB Bus Clock |
| PENABLE | IN | APB Bridge | APB enable signal. PENABLE is asserted HIGH for one cycle of PCLK to enable a bus transfer. |
| PRDATA[15:0] | OUT | APB Bridge | Subset of the unidirectional APB read data bus. |
| PSEL | IN | APB Bridge | SCI select signal from the decoder. When HIGH, this signal indicates the slave device is selected by the APB bridge and that a data transfer is required. |
| PWDATA[15:0] | IN | APB Bridge | Subset of the unidirectional APB write data bus. |
| PWRITE | IN | APB Bridge | APB transfer direction signal. Indicates a write access when HIGH, read access when LOW. |

***Table 4.2-1 APB signal descriptions***

| Signal Name | Type | Source / Destination | Description |
|---|---|---|---|
| SCICLK | IN | Clock Generator | SCI Reference clock |
| nSCIRST | IN | Reset Controller | SCI reset signal to SCICLK clock domain, active LOW. The reset controller must use PRESETn to assert nSCIRST asynchronously but de-assert it synchronously with respect to SCICLK. |
| SCICARDININTR | OUT | Interrupt Controller | SCI Card In interrupt. |
| SCICARDOUTINTR | OUT | Interrupt Controller | SCI Card Out interrupt. |
| SCICARDUPINTR | OUT | Interrupt Controller | SCI Card Powered Up interrupt. |
| SCICARDDNINTR | OUT | Interrupt Controller | SCI Card Powered Down interrupt. |
| SCITXERRINTR | OUT | Interrupt Controller | SCI Character transmission error interrupt. |
| SCIATRSTOUTINTR | OUT | Interrupt Controller | SCI ATR start timeout interrupt. |
| SCIATRDTOUTINTR | OUT | Interrupt Controller | SCI ATR duration timeout interrupt. |
| SCIBLKTOUTINTR | OUT | Interrupt Controller | SCI Block timeout interrupt between blocks. |
| SCICHTOUTINTR | OUT | Interrupt Controller | SCI Character timeout interrupt between characters. |
| SCIRTOUTINTR | OUT | Interrupt Controller | SCI Receive FIFO Read timeout interrupt. |
| SCIRXTIDEINTR | OUT | Interrupt Controller | SCI Receive FIFO tide mark reached interrupt. |
| SCITXTIDEINTR | OUT | Interrupt Controller | SCI Transmit FIFO tide mark reached interrupt. |
| SCIRORINTR | OUT | Interrupt Controller | SCI Receive Over Run interrupt. |
| SCICLKSTPINTR | OUT | Interrupt Controller | SCI Clock Stopped interrupt |
| SCICLKACTINTR | OUT | Interrupt Controller | SCI Clock Active interrupt |

| | | | |
|---|---|---|---|
| SCIINTR | OUT | Interrupt Controller | SCI interrupt. A single combined interrupt generated as an OR function of the twelve individually maskable interrupts above. |
| SCITXDMASREQ | OUT | DMA Controller | SCI Transmit DMA single request |
| SCITXDMABREQ | OUT | DMA Controller | SCI Transmit DMA burst request |
| SCIRXDMASREQ | OUT | DMA Controller | SCI Receive DMA single request |
| SCIRXDMABREQ | OUT | DMA Controller | SCI Receive DMA burst request |
| SCITXDMACLR | IN | DMA Controller | SCI Transmit DMA Clear, asserted by the DMA controller to clear the transmit request signals. If DMA burst transfer is requested, the clear signal is asserted during the transfer of the last data in the burst. |
| SCIRXDMACLR | IN | DMA Controller | SCI Receive DMA Clear, asserted by the DMA controller to clear the receive request signals. If DMA burst transfer is requested, the clear signal is asserted during the transfer of the last data in the burst. |
| SCANENABLE | IN | Test Controller | Test Scan enable signal for both clock domains. |
| SCANINPCLK | IN | Test Controller | Test Scan data input signal for the PCLK domain |
| SCANINSCICLK | IN | Test Controller | Test Scan data input signal for the SCICLK domain |
| SCANOUTPCLK | OUT | Test Controller | Test Scan data output signal for the PCLK domain |
| SCANOUTSCICLK | OUT | Test Controller | Test Scan data output signal for the SCICLK domain |

***Table 4.2-2 Block signal descriptions***

| Signal Name | Type | Source / Destination | Description |
|---|---|---|---|
| SCICLKIN | IN | PAD | SCI serial clock input. |
| SCIDATAIN | IN | PAD | SCI serial data input. |
| nSCICLKOUTEN | OUT | PAD | Tristate output buffer control for Clock Pad (active LOW). |
| SCICLKOUT | OUT | PAD | Clock output. |
| nSCIDATAOUTEN | OUT | PAD | Data output enable (typically drives an open-drain configuration, active LOW). |
| nSCICLKEN | OUT | PAD | Tristate control for external off-chip Clock buffer (active LOW). |
| nSCIDATAEN | OUT | PAD | Tristate control for external off-chip Data buffer (active LOW). |
| SCIVCCEN | OUT | PAD | Supply voltage controls. |
| nSCICARDRST | OUT | PAD | Reset to Card (active LOW). |
| SCIFCB | OUT | PAD | Function code bit, used in conjunction with nSCICARDRST. |
| SCIDETECT | IN | PAD | Card detect signal from card interface device. |
| SCIDEACREQ | IN | PMU | Card deactivation request signal from the PMU. |
| SCIDEACACK | OUT | PMU | Card deactivation acknowledgement to the PMU. |

***Table 4.2-3 I/O Pad signals***

## 4.3 Programmer's Model

### 4.3.1 Functional Mode Registers

| Address | Type | Width | Reset Value | Name | Description |
|---|---|---|---|---|---|
| SCIBase + 0x00 | R/W | 9/8 | 0x-- | **SCIDATA** | Data read or written from the interface. |
| SCIBase + 0x04 | R/W | 8/8 | 0x00 | **SCICR0** | Control register 0. |
| SCIBase + 0x08 | R /W | 7/7 | 0x00 | **SCICR1** | Control register 1 |
| SCIBase + 0x0C | W | 3 | 0X0 | **SCICR2** | Control register 2 |
| SCIBase + 0x10 | R/W | 8/8 | 0x00 | **SCICLKICC** | External smart card frequency divisor |
| SCIBase + 0x14 | R/W | 8/8 | 0x00 | **SCIVALUE** | Number of SCIBAUD cycles per etu. |
| SCIBase + 0x18 | R/W | 16/16 | 0x0000 | **SCIBAUD** | Baud rate clock divisor. |
| SCIBase + 0x1C | R/W | 8/8 | 0x00 | **SCITIDE** | Transmit and receive FIFO water level marks. |
| SCIBase + 0x20 | R/W | 2/2 | 0x0 | **SCIDMACR** | Direct Memory Access control register. |
| SCIBase + 0x24 | R/W | 8/8 | 0x00 | **SCISTABLE** | Card stable after insertion debounce duration |
| SCIBase + 0x28 | R/W | 16/16 | 0x0000 | **SCIATIME** | Card activation event time. |
| SCIBase + 0x2C | R/W | 16/16 | 0x0000 | **SCIDTIME** | Card deactivation event time. |
| SCIBase + 0x30 | R/W | 16/16 | 0x0000 | **SCIATRSTIME** | Time to the start of the ATR reception. |
| SCIBase + 0x34 | R/W | 16/16 | 0x0000 | **SCIATRDTIME** | Maximum allowed duration of the ATR character stream. |
| SCIBase + 0x38 | R/W | 12/12 | 0x000 | **SCISTOPTIME** | Duration before the card clock can be stopped. |
| SCIBase + 0x3C | R/W | 12/12 | 0x000 | **SCISTARTTIME** | Duration before transactions can commence after clock is restarted. |
| SCIBase + 0x40 | R/W | 6/6 | 0x00 | **SCIRETRY** | Retry limit register. |

| | | | | | |
|---|---|---|---|---|---|
| SCIBase + 0x44 | R/W | 16/16 | 0x0000 | **SCICHTIMELS** | Character to character timeout (least significant 16 bits). |
| SCIBase + 0x48 | R/W | 16/16 | 0x0000 | **SCICHTIMEMS** | Character to character timeout (most significant 16 bits). |
| SCIBase + 0x4C | R/W | 16/16 | 0x0000 | **SCIBLKTIMELS** | Receive timeout between blocks (least significant 16 bits). |
| SCIBase + 0x50 | R/W | 16/16 | 0x0000 | **SCIBLKTIMEMS** | Character to character timeout (most significant 16 bits) |
| SCIBase + 0x54 | R/W | 8/8 | 0x00 | **SCICHGUARD** | Character to character extra guard time. |
| SCIBase + 0x58 | R/W | 8/8 | 0x00 | **SCIBLKGUARD** | Block guard time. |
| SCIBase + 0x5C | R/W | 16/16 | 0x0000 | **SCIRXTIME** | Receive read time-out. |
| SCIBase + 0x60 | R | 4 | 0xA | **SCIFIFOSTATUS** | Transmit and receive FIFO status |
| SCIBase + 0x64 | R/(W) | 4 | 0x0 | **SCITXCOUNT** | Transmit FIFO fill level. A write of any value to this location clears the TX FIFO. |
| SCIBase + 0x68 | R/(W) | 4 | 0x0 | **SCIRXCOUNT** | Receive FIFO fill level. A write of any value to this location clears the RX FIFO. |
| SCIBase + 0x6C | R/W | 15/15 | 0x0000 | **SCIIMSC** | Interrupt mask set clear register |
| SCIBase + 0x70 | R/W | 15/15 | 0x400A | **SCIRIS** | Raw interrupt status register |
| SCIBase + 0x74 | R/W | 15/15 | 0x0000 | **SCIMIS** | Masked interrupt status register |
| SCIBase + 0x78 | W | 13 | 0x0000 | **SCIICR** | Interrupt clear register |
| SCIBase + 0x7C | R/W | 11/4 | 0x000 | **SCISYNCACT** | Synchronous mode activation register |
| SCIBase + 0x80 | R/W | 6/6 | 0x00 | **SCISYNCTX** | Synchronous mode transmit clock and data stream register. |
| SCIBase + 0x84 | R/W | 2/2 | 0x0 | **SCISYNCRX** | Synchronous mode receive clock and data stream register. |
| SCIBase + 0x88 to 0xEFC | - | - | - | **-** | Reserved |
| SCIBase + 0xF00 to | - | - | - | **-** | Reserved for test purposes |

| | | | | | |
|---|---|---|---|---|---|
| 0xF10 | | | | | |
| SCIBase + 0xF14 to 0cFCC | - | - | - | **-** | Reserved |
| SCIBase + 0xFD0 to 0xFDC | - | - | - | **-** | Reserved for future expansion. |
| SCIBase + 0xFE0 | R | 8 | 0x31 | **SCIPeriphID0** | Peripheral identification register bits 7:0. |
| SCIBase + 0xFE4 | R | 8 | 0x11 | **SCIPeriphID1** | Peripheral identification register bits 15:8. |
| SCIBase + 0xFE8 | R | 8 | 0x*4[a] | **SCIPeriphID2** | Peripheral identification register bits 23:16. |
| SCIBase + 0xFEC | R | 8 | 0x00 | **SCIPeriphID3** | Peripheral identification register bits 31:24. |
| SCIBase + 0xFF0 | R | 8 | 0x0D | **SCIPCellID0** | PrimeCell identification register bits 7:0. |
| SCIBase + 0xFF4 | R | 8 | 0xF0 | **SCIPCellID1** | PrimeCell identification register bits 15:8. |
| SCIBase + 0xFF8 | R | 8 | 0x05 | **SCIPCellID2** | PrimeCell identification register bits 23:16. |
| SCIBase + 0xFFC | R | 8 | 0xB1 | **SCIPCellID3** | PrimeCell identification register bits 31:24. |

***Table 4.3-1 SCI Register summary***

Please see the ARM PrimeCell Smart Card Interface (PL131) Technical Reference Manual [Ref. 1] for a detailed description of each register.

### 4.3.2 Test Mode Registers

| Address | Type | Width | Reset Value | Name | Description |
|---|---|---|---|---|---|
| SCIBase + 0xF00 | R/W | 2/2 | 0x00 | **SCITCR** | Test control register. |
| SCIBase + 0xF04 | R/W | 6/6 | 0x00 | **SCIITIP** | Integration test input register. |
| SCIBase + 0xF08 | R/W | 16/16 | 0x0000 | **SCIITOP1** | Integration test output register 1. |
| SCIBase + 0xF0C | R/W | 13/13 | 0x0000 | **SCIITOP2** | Integration test output register 2. |
| SCIBase + 0xF10 | R/W | 16/16 | 0x0000 | **SCITDR** | Test data register. |

***Table 4.3-2 Test Mode Registers***

Please see ARM PrimeCell Smart Card Interface (PL131) Technical Reference manual [Ref.1] for a detailed description of each register.

### 4.3.3 Register Memory Map

| Address | Read Location | Write Location |
|---|---|---|
| SCI Base + 0x000 | SCIDATA | SCIDATA |
| SCI Base + 0x004 | SCICR0 | SCICR0 |
| SCI Base + 0x008 | SCICR1 | SCICR1 |
| SCI Base + 0x00C | ---------- | SCICR2 |
| SCI Base + 0x010 | SCICLKICC | SCICLKICC |
| SCI Base + 0x014 | SCIVALUE | SCIVALUE |
| SCI Base + 0x018 | SCIBAUD | SCIBAUD |
| SCI Base + 0x01C | SCITIDE | SCITIDE |
| SCI Base + 0x020 | SCIDMACR | SCIDMACR |
| SCI Base + 0x024 | SCISTABLE | SCISTABLE |
| SCI Base + 0x028 | SCIATIME | SCIATIME |
| SCI Base + 0x02C | SCIDTIME | SCIDTIME |
| SCI Base + 0x030 | SCIATRSTIME | SCIATRSTIME |
| SCI Base + 0x034 | SCIATRDTIME | SCIATRDTIME |
| SCI Base + 0x038 | SCISTOPTIME | SCISTOPTIME |
| SCI Base + 0x03C | SCISTARTTIME | SCISTARTTIME |
| SCI Base + 0x040 | SCIRETRY | SCIRETRY |
| SCI Base + 0x044 | SCICHTIMELS | SCICHTIMELS |
| SCI Base + 0x048 | SCICHTIMEMS | SCICHTIMEMS |
| SCI Base + 0x04C | SCIBLKTIMELS | SCIBLKTIMELS |
| SCI Base + 0x050 | SCIBLKTIMEMS | SCIBLKTIMEMS |
| SCI Base + 0x054 | SCICHGUARD | SCICHGUARD |
| SCI Base + 0x058 | SCIBLKGUARD | SCIBLKGUARD |
| SCI Base + 0x05C | SCIRXTIME | SCIRXTIME |
| SCI Base + 0x060 | SCIFIFOSTATUS | ---------- |
| SCI Base + 0x064 | SCITXCOUNT | SCITXCOUNT |
| SCI Base + 0x068 | SCIRXCOUNT | SCIRXCOUNT |
| SCI Base + 0x06C | SCIIMSC | SCIIMSC |
| SCI Base + 0x070 | SCIRIS | ---------- |
| SCI Base + 0x074 | SCIMIS | ---------- |
| SCI Base + 0x078 | ---------- | SCIICR |
| SCI Base + 0x07C | SCISYNCACT | SCISYNCACT |
| SCI Base + 0x080 | SCISYNCTX | SCISYNCTX |
| SCI Base + 0x084 | SCISYNCRX | SCISYNCRX |
| SCI Base + 0x088 to Ox0EFC | Reserved | Reserved |
| SCI Base + 0xF00 | SCITCR | SCITCR |

| | | |
|---|---|---|
| SCI Base + 0xF04 | SCIITIP | SCIITIP |
| SCI Base + 0xF08 | SCIITOP1 | SCIITOP1 |
| SCI Base + 0xF0C | SCIITOP2 | SCIITOP2 |
| SCI Base + 0xF10 | SCITDR | SCITDR |
| SCI Base + 0xF14 to 0xFDC | Reserved for future expansion | Reserved for future expansion |
| SCI Base + 0xFE0 | SCIPeriphID0 | ---------- |
| SCI Base + 0xFE4 | SCIPeriphID1 | ---------- |
| SCI Base + 0xFE8 | SCIPeriphID2 | ---------- |
| SCI Base + 0xFEC | SCIPeriphID3 | ---------- |
| SCI Base + 0xFF0 | SCIPCellID0 | ---------- |
| SCI Base + 0xFF4 | SCIPCellID1 | ---------- |
| SCI Base + 0xFF8 | SCIPCellID2 | ---------- |
| SCI Base + 0xFFC | SCIPeriphID3 | ---------- |

**Table 4.3-3 Register Memory Map**

## 4.4 Block Partitioning

The sub-blocks in the design are listed below.

- APB Interface (Input module and Output module)
- Register block
- Transmit and Receive logic
- Interrupt control logic
- Smart Card Interface Control logic
- Transmit FIFO
- Receive FIFO
- Synchronisers
- Direct Memory Access (DMA)

The design of the above-mentioned sub-blocks is detailed below.

### 4.4.1 APB Interface

The APB interface comprises an input and output module. This interface deals with reads and writes to registers from the APB. A simplified block diagram of the APB interface block is shown in ***Figure 4.4-1.***

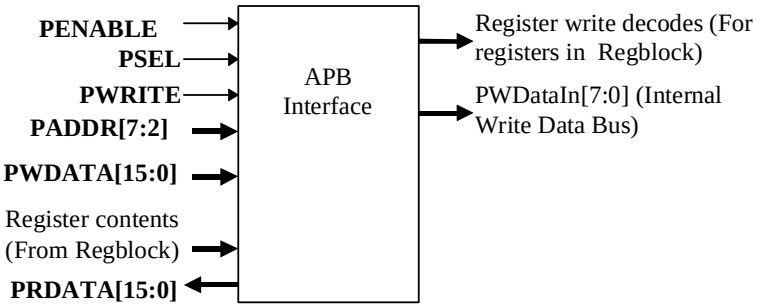


***Figure 4.4-1 APB Interface Block diagram***

**Input Module**

This module provides the APB write interface. It generates decodes for writes to all registers in the device. Writes to the SCIDATA register actually result in writes to the transmit FIFO provided the MODE bit in the Control Register1 is set for transmission. The SCICR2 register, implemented as a virtual register, is used to initiate the card activation, deactivation and warm reset sequences. The respective control signal is asserted when the register is written to with the corresponding bit in the write data set to one. The interrupt clear mechanism is also implemented in the similar manner.

To reduce the load on the bus signals such as PADDR and PENABLE, internal clocked versions of these signals (LatchedPADDR and IntPENABLE) are generated in the APB interface. Each register addressable from the APB is clocked by PCLK and is written to when the appropriate write enable is HIGH. The write enable terms are generated from a combinatorial decode of PSEL, PWRITE, LatchedPADDR[11:2] and IntPENABLE.

Non-AMBA inputs SCICLKIN, SCIDATAIN, SCIDETECT, SCIDEACREQ, SCITXDMACLR and SCIRXDMACLR are multiplexed with their corresponding integration test register bits. The block diagram in ***Figure 4.4-2*** illustrates the structure of the input module. In this block diagram, the write decodes are generated in the APB interface block, while the registers are actually located in the register block, RegBlock.

Since SCICLK is the main clock for the SCI core logic, the control register bits are synchronised to SCICLK before their use in the SCI core.

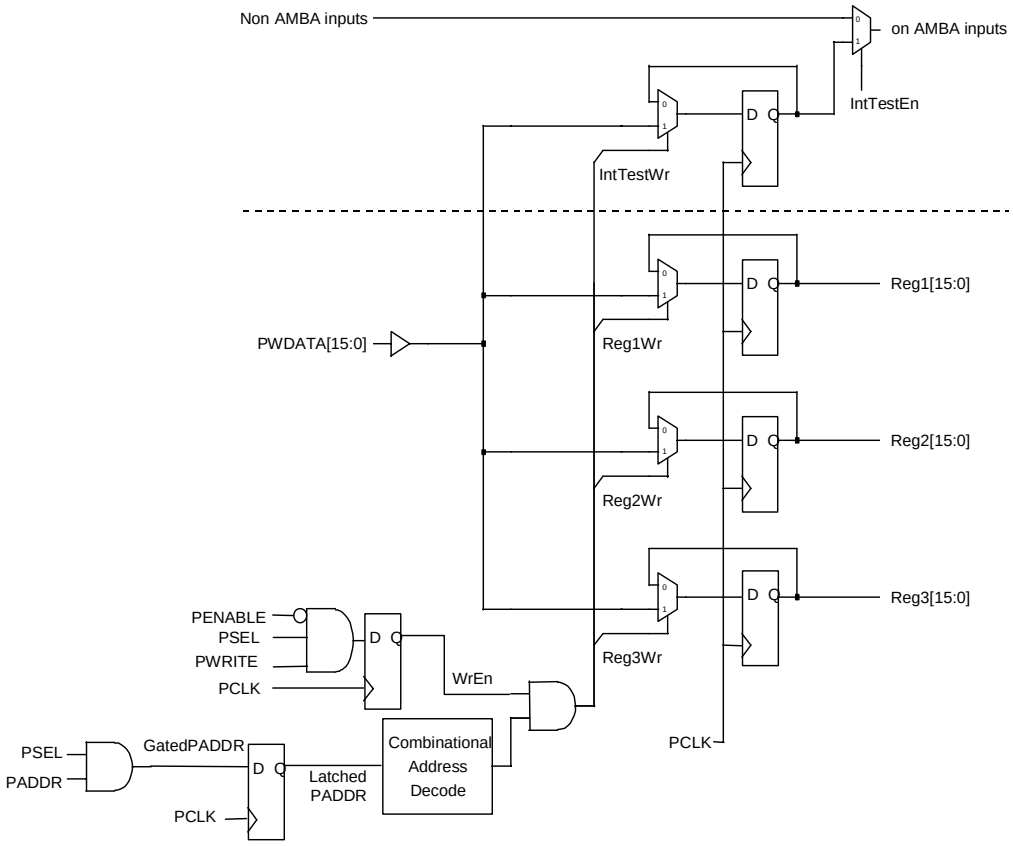


***Figure 4.4-2 Input module***

### Output module

The output module in the APB interface consists of a multiplexer for selecting which of the various sources for read data is driven onto the read data bus. The output of the multiplexer is fed to a bank of flip-flops clocked by the system PCLK. A combinatorial address decode is used to select which of the output sources are driven onto the bus. Non-AMBA outputs such as the interrupts and DMA request signals are also made observable through APB read accesses. The output multiplexer is left such that, by default, zeros are driven on the data bus. This ensures that there are no restrictions placed on the type of external bus connection method used for the data bus on the APB. A block diagram of the output module is shown **in *Figure 4.4-3***. The registers shown in this block diagram are actually located in the register block, while the output multiplexer and the final data register on the output of the multiplexer are located in the APB interface block.

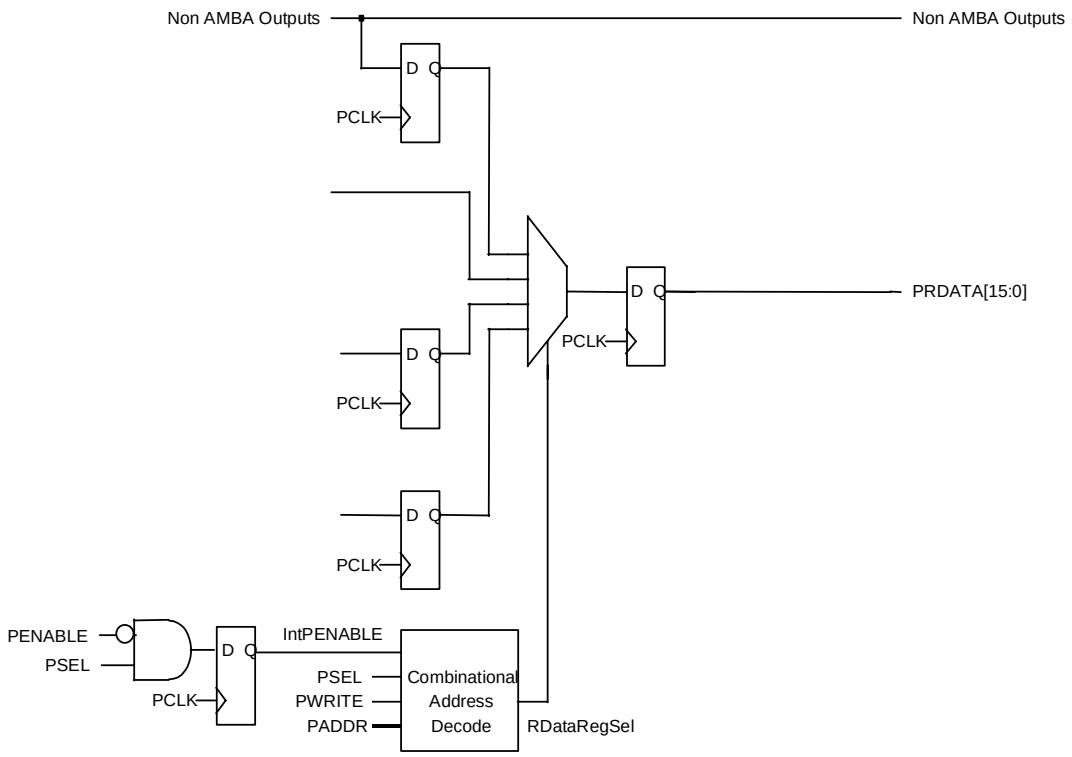


***Figure 4.4-3 Output Module***

## 4.4.2 Register Block

Besides the registers themselves, this block also contains the control logic required to synchronise the contents of the SCIRETRY, SCIRXTIME, SCISYNCACT, SCISTABLE, SCIATIME, SCIDTIME, SCIATRST, SCIATRDTIME, SCISTOPTIME, SCISTARTTIME, SCIBLKTIME, SCICHTIME, SCICLKICC, SCIBAUD, SCIVALUE, SCICHGUARD and SCIBLKGUARD registers from the PCLK to the SCICLK domain.

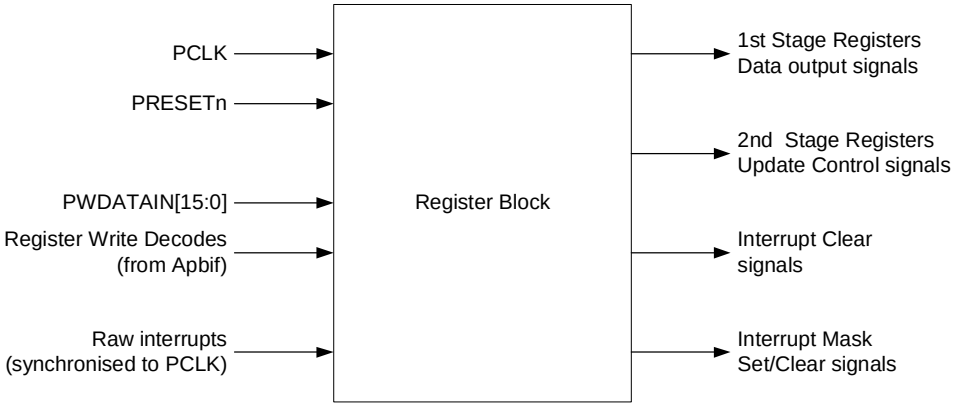


***Figure 4.4-4 Register Block***

A Two-buffer technique is used to synchronise the contents of these registers to the SCICLK domain.

In this 2-buffer technique, writes from the APB to these registers, go to the respective first stage buffers. When there is a write to these registers, a control signal toggles. This signal is double synchronised to the SCICLK domain. An edge on the synchronised signal is detected by generating a delayed version and XOR-ing the synchronised signal with its delayed version. Thus, a one-SCICLK-wide load pulse is generated with every write to these registers. When this load signal is asserted, the contents of the first stage buffer are copied into the second stage buffer in the SCICLK domain. ***Figure 4.4-5*** illustrates the 2-buffer mechanism for a register.

When synchronisation is in progress, it is expected that there are no further writes immediately to all these registers. This is a reasonable assumption since these registers are control registers and infrequently written.

In addition, the interrupt status bits are AND-ed with the respective interrupt enables in this block. The masked versions of the interrupts are driven as the interrupt outputs from the SCI and these are given to the interrupt controller.

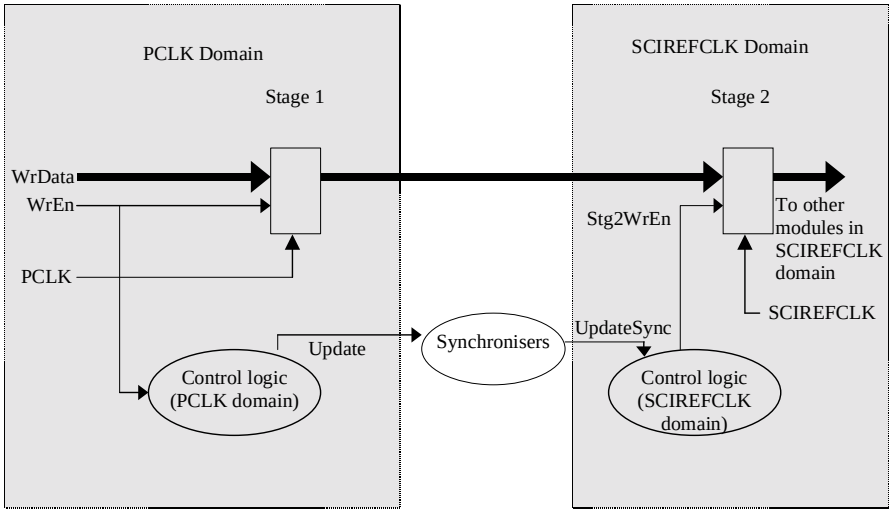


***Figure 4.4-5 Two-Buffer synchronisation mechanism for a register***

The SciRegBlk contains the logic for the assertion of the interrupt clear mechanism by writing a 1 to the respective bit within the SCIICR register and then subsequent removal by the respective synchronised clear signal from the SCICLK domain.

### 4.4.3 Transmit and Receive logic

This block implements the main functionality of the SCI with the help of control information from the SCI control block. This block drives all the card control signals besides the data and contains all the counters used for timing various transaction phases.

The SCIBAUDCNT and SCIVALUECNT timers perform the bit rate conversion and bit rate adjustment. Together, they define one ETU (Elementary Time Unit) period. The SCICLKICCCNT counter generates the smart card clock. The SCIACTTIME counter is used for eight different operations. This is used to generate timeout signals during debounce time, activation event time, Warm Reset event time, ATR (Answer To Reset) start time, ATR duration time, deactivation event time, clock stop and clock re-start times. The SCIDATATIME counter is used to measure block guard time, character guard time and to indicate timeout events between blocks and characters.

During activation, deactivation and warm reset sequence, this block activates and deactivates the power line, data line, clock line and the card reset line in the specified sequence. Software initiates the card activation, warm reset and deactivation by writing a one into the corresponding bit positions in Control Register2. Card deactivation can

be initiated by software, the physical removal of the card, deactivation request from the PMU and ATR start timeout after warm reset.

The Smart card clock from the SCICLKICCNT timer will drive the nSCICLKOUTEN line or the SCICLKOUT line depending upon the clock output configuration programmed in the Control Register1 through the CLKZ1 bit.

After the activation sequence, the ATR start time will be loaded into the SCIACTTIME timer and the interface will be ready for reception. The software has to set the Mode bit to zero prior to the activation of the card. This is to allow the interface to start ATR reception after the activation sequence. During reception, on detecting a valid start bit, the SCI control block enables the receive portion of this logic. After detecting a valid start bit after the activation sequence, the ATR start timer will be disabled and the ATR duration will be loaded at the same time into the SCIACTTIME timer. If the ATR start time expires before the reception of a valid start bit, the ATR start timeout indication signal will be set to generate ATR start timeout interrupt. The ATR duration timer will be counting continuously and if software received all the data in the ATR, software should disable the counter. If the ATR duration time expires before software disables the timer, the ATR duration timeout will be set to generate the ATR duration timeout interrupt.

After the receive logic is enabled, the receive logic samples the data line on every etu and stores it in an internal shift register. The stored bits are xor-ed with the SENSE bit to correctly interpret the incoming data stream. If a parity error is detected, the receive logic pulls the data line low for 2 etus to request the card to retransmit the same data, if the current reception mode is T0 mode and a non-zero value is programmed into the retry register. The interface repeatedly initiates a retry to get the correct data from the card until the retry attempts reach the RXRETRY value. After the RXRETRY, if the receiver still receives the wrong parity, it registers a parity error. If the reception is set to T1 mode or zero retry is programmed for T0 mode, the interface will not request the card to retransmit on detecting a parity error. After the completion of reception, the receive logic creates write enable signals to transfer the current content of the shift register into the RX FIFO. These data will be swapped (MSB to LSB) while storing into the RX FIFO if the ORDER bit is set. After finishing the reception at 10.5 etu in the case of error-free reception or 12.5 etu in the case of request retry, the character time will be loaded into the SCIDATATIME timer. This timer will be disabled when the card sends the next valid start bit or the interface is set in transmission mode. Before these events, if the timer expires, the character timeout interrupt will be set.

The SCIRTSTBPRECNT timer is used as a pre-scaler for the debounce timer and to generate the read timeout interrupt as well. After the debounce time, the SCIRTSTBPRECNT is loaded with the read timeout value and the counter will start counting down only if there is a byte in the RX FIFO. Whenever there is a read from the Rx FIFO, the counter will be reloaded with the read timeout value and the load event will be sent to the other domain through synchroniser to inform the RX FIFO control logic. The RX FIFO controller uses this signal to generate the control signal required to reload the read timeout value for the next RX FIFO read.

If the Mode bit is set for transmission, the transmitter logic waits for a minimum of 0.5 or 1.5 etu in addition to the block guard time before starting the transmission. This is because the interface finishes its reception at 10.5 etu. The transmitter logic should wait for 0.5 etu or 1.5 etu in addition to the block guard time to meet the minimum time of 11 etu in the case of T1 or 12 etu in the case of T0 mode before transmission.

After meeting the minimum etu time between leading edges of the start bits and block guard time, the SCI control block enables the transmission logic. When this block transmits the start bit, it transfers the data from the TX FIFO into the internal shift register and the transferred data bits will be swapped if the ORDER bit is set. After every etu, the data bit will be shifted out serially and the serial bits are xored with the SENSE bit, which will finally drive the dataout line. After the transmission of the parity bit, the data line will be placed in high impedance state. On the next etu, the data input line will be sampled for low if the transmission mode is set to T0. If the TXRETRY is programmed for non-zero value and the data input line is low, then the interface will retransmit the data. This process will be repeated TXRETRY times if the data line is low again and again after the parity bit is transmitted. If the card pulls down the data line to request retransmission even after the interface has retransmitted TXRETRY number of times, the interface will flag a TXERR interrupt and the software has to clear the TX FIFO contents to allow further activity to proceed. The interface will flag a transmission error if the retry value is 0 the first time itself. There is no TXERR facility in the case of T1 mode.

During error free transmission, the interface will wait character guard time before the next transmission after meeting the minimum period specified by the protocols. During retransmission, the interface will only wait 4 etu after the minimum period irrespective of the character guard time value. This block uses the SCIDATATIME to wait for the character guard period in the case of error free transmission and 4 etu in the case of retransmission. The read pointer will be incremented only after the error free transmission and the SCI control block will expect a read pointer increment done indication from the TX FIFO control logic to load the next data into internal shift register. This transmission will proceed until the Mode bit is set to receive and TX FIFO becomes empty. If the current transmission is the last byte transmission and the Mode bit is set to receive, the transmitter will not wait for character guard time and it loads the block time into the SCIDATATIME timer immediately after meeting the minimum time requirements and expects data from the card. Before the reception of a valid start bit, if the SCIDATATIME expires, the block timeout indication signal will be set to generate a block timeout interrupt.

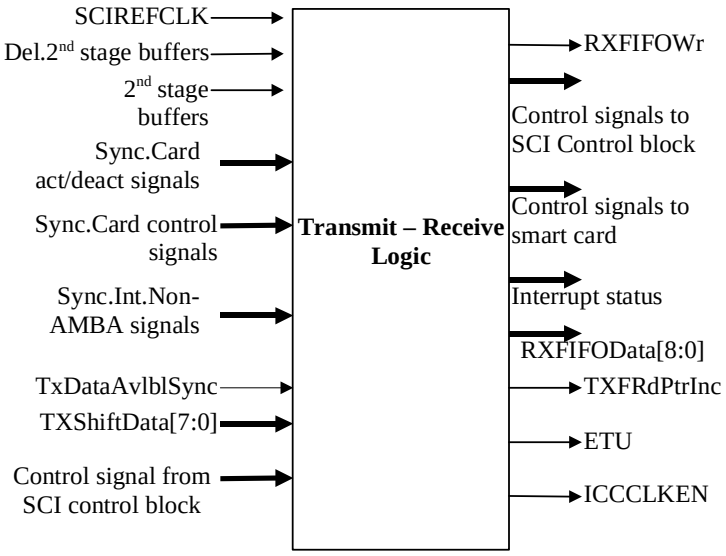


***Figure 4.4-6 SCI Transmit-Receive Logic***

This block generates all the interrupts except the FIFO related interrupts. After reset, the Card Out and Card Powered Down interrupts will be set. The Card Out interrupt will be reset and the Card In interrupt will be set as soon as a valid card is present in the interface which is indicated by the Card Present signal. The Card Up interrupt will be set and the Card Down interrupt will be reset after the activation sequence. The Card Up interrupt will be reset as soon as the deactivation sequence starts. Card Down will be generated again after the deactivation sequence. The transmission error – TXERR interrupt – will be set when the transmit logic sends a transmission error indication signal. The ATR start timeout, the ATR duration timeout, the Character timeout and the Block timeout interrupts will be set when their corresponding timeout indicator signals are asserted. The software can clear all the above interrupts by writing a value of one to its read location. The clear signals are created by xor-ing the synchronised clear triggers with their delayed versions. The RX FIFO read timeout will be set when receive timer expires after receiving a byte. Programming this register with a zero value is illegal. This interrupt will be cleared when the software reads the received byte from the RX FIFO or when the software clears the RX FIFO contents.

The card deactivation request from the PMU will be acknowledged after the deactivation sequence is completed. This acknowledgement signal will be high until the PMU deactivates the deactivation request.

The SCI PL131 supports the stop clock mode of operation and uses the SCISTOPTIME and SCISTARTTIME registers to program the stopping and re-starting of the card clock respectively. The SCIACTTIME counter is used

for this mode of operation. The SCICLKSTPINTR is asserted when the clock has stopped and the SCICLKACTINTR is asserted when the clock has been re-started for a duration defined by SCISTARTTIME register.

For non-EMV compliant cards, the synchronous control register should be set to drive data and clock through the smart card data register.

### 4.4.4 Interrupt control logic

This is a pure combinatorial logic block that take the raw interrupts and combines each with its respective interrupt set-clear mask and interrupt clear bits to form the resultant masked interrupt source outputs. The masked outputs are also logically OR'ed together to produce a single output INTR. ***Figure 4.4-7*** shows the associated signals relating to the Interrupt control logic block.

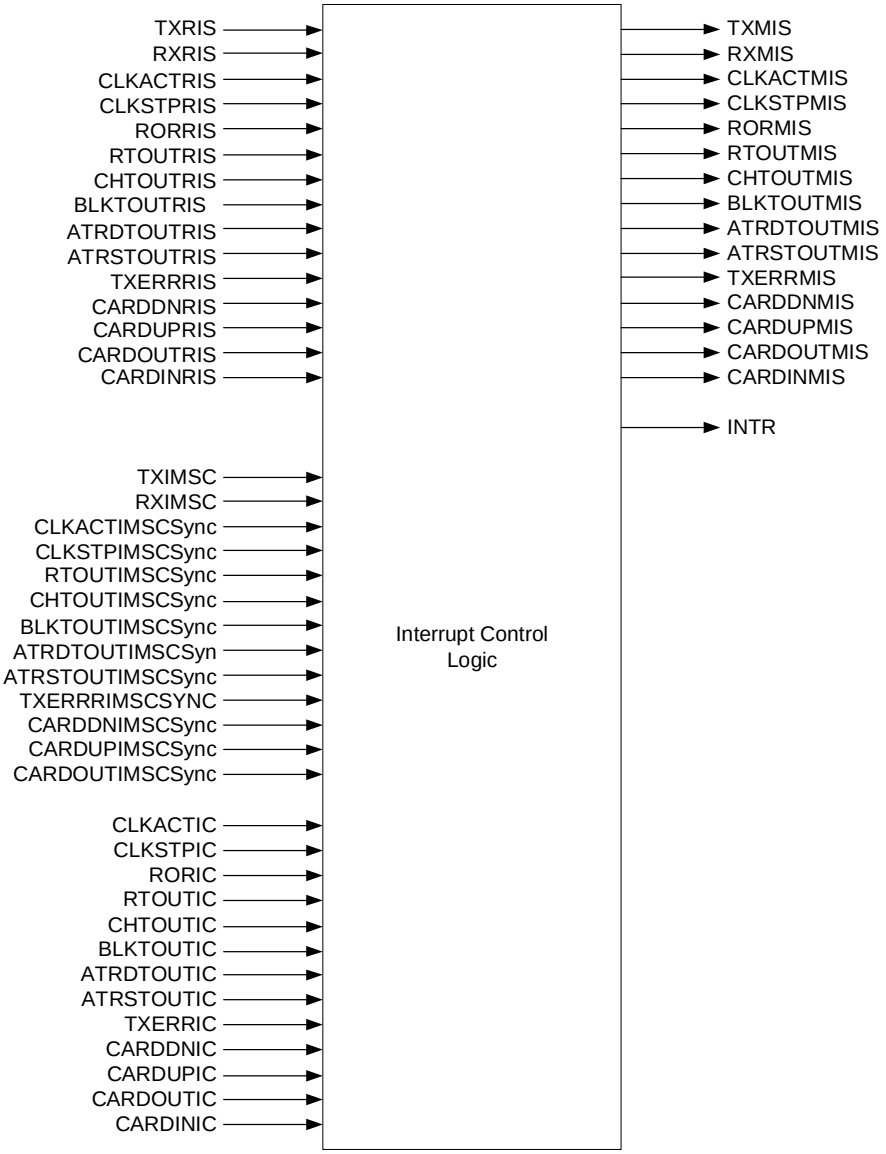


***Figure 4.4-7 Interrupt Control Logic block***

## 4.4.5 SCI Control logic

This block controls the SCI using the timeout and other control signals received from the SciTxRx block.

After reset, the Control logic will be ready to detect the presence of the card in the interface. The control logic waits for the debounce period if the card is present in the interface. It receives the debounce over signal from the SciTxRx block and issues the Card Present signal which indicates the presence of a valid card in the interface. After a valid card is detected, the software has to ensure that the Mode bit is cleared to zero (receive mode) and then set the Startup bit to initiate the activation sequence. When the software sets the Startup bit, the block initiates the activation sequence. After the activation sequence is over, the SciTxRx block sends the activation over signal to start reception. Using that signal, this block enables the ATR start time measuring timer. After the

activation sequence is complete, this block samples the data input line for a low level. If the sampled low is a valid start bit, this block enables the receive section of the SciTxRx. The ATR start timer will be disabled and the ATR duration timer will be enabled at the same time after the activation sequence. The ATR duration timer will be disabled by disabling the ATRDEN bit in the Control Register1 or when a warm reset command is issued by the software or the Mode bit is set for transmission. After the warm reset, the ATR start timer will be enabled and this logic expects a valid start bit before this timer expires. If the timer expires before receiving the start bit, this block initiates the deactivation sequence. After completion of reception, the SciTxRx block sends a receive completion indication to start reception or transmission depending upon the Mode bit and the character timer will be enabled. This timer will be disabled when this logic detects a valid start bit in the case of reception or when the Mode bit is set for transmission.

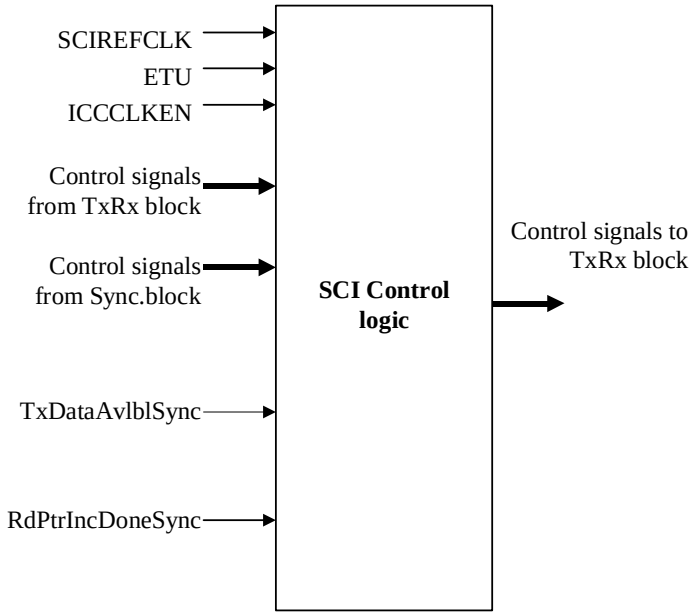


***Figure 4.4-8 SCI Control Logic***

Once the Mode bit is set for transmission, the logic waits for 0.5 etu in the case of T1 mode and 1.5 etu in the case of T0 mode in addition to the block guard time. This is to meet the minimum time between the leading edge of the receive start bit and the leading edge of the transmit start bit as this block completes the reception at 10.5 etu. After this, if a data available indication from the TX FIFO control logic is observed, transmission commences; else it awaits the arrival of this signal. After the completion of transmission, the SciTxRx block sends a transmission over signal. Using this signal, the block moves to the receive side and simultaneously enables the block timer. The timer is disabled only if there is a valid start bit received; else it again enters the transmit mode.

If the software issues a Warm reset, the block initiates a warm reset sequence irrespective of whether it is currently in the middle of transmission or reception. Prior to this the software must set the Mode bit for reception.

After the detection of a valid card, if a CARDDOWN indication is received, the deactivation sequence is initiated. The SciTxRx block then issues the deactivation over signal. The signal indicates the start of another card session.

Any write to the ICC status indicates that the software is dealing with a non-EMV compliant card. The software will activate the non-EMV compliant card through the ICC status register and can also allow the hardware to activate the card by setting the Startup bit. Note that the subsequent phases of the card session – data transmission and reception – should be handled solely by the software. But the deactivation sequence is carried out only by the hardware.

Clock stop mode is supported by the SCI PL131 and this is controlled within the SciCntl block. It is possible to stop the card clock by writing a 1 to bit [6], CLKDIS, within the SCICR0 register. This can be done when the SCI is

idle i.e. no transmission or reception in progress, and even during a transmission whereby the clock will stop when the TX FIFO is empty and the clock stop time has expired. The card clock is re-started by writing a 0 back into bit [6], CLKDIS, of the SCICR0 register. A secondary function of defining the state at which the clock stops is programmed by bit [7], CLKVAL in the SCICR0 register.

### 4.4.6 Transmit FIFO

The Transmit FIFO consists of an 8-deep 8-bit wide circular buffer. Reads and writes to the FIFO result in access to buffer locations addressed by a read pointer and a write pointer respectively. The pointer values are compared to deduce the FIFO fill level. The FIFO control logic also generates the transmit tide interrupt.

Writes to the Transmit FIFO occur through the APB interface in the PCLK domain. The SCIDATAWrEn signal is a decode of writes to the SCIDATA register. When the write enable signal SCIDATAWrEn is asserted and the MODE bit is set to 'Transmit', data present on the PWDATAIn [7:0] bus is written, on the rising edge of PCLK, into the FIFO location pointed to by the current value of the write pointer. At the end of every FIFO write, the write pointer is incremented.

Both the read pointer and the write pointer are clocked on PCLK. Reads from the transmit FIFO are done by the Transmit section, which runs on SCICLK. The TXFIFO always drives the RdData bus with the contents of the FIFO location pointed to by the current value of the Read Pointer. When the FIFO is non-empty, this is indicated by the TXDataAvlbl signal.

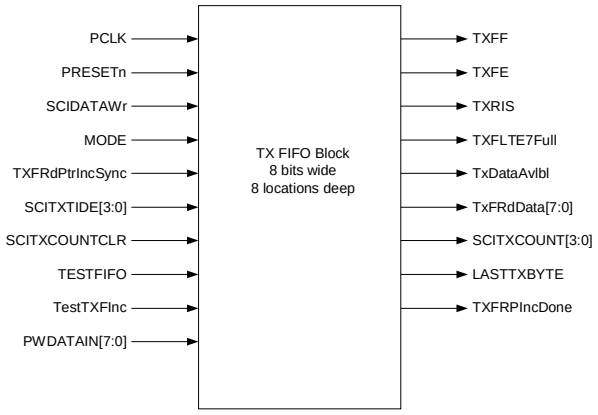


***Figure 4.4-9 Transmit FIFO***

After the Transmit section has completed transmitting one data byte, the TXFRdPtrInc signal is asserted by the transmit logic. This signal is synchronised to the PCLK domain (TXFRdPtrIncSync) and is used to increment the read pointer. When the increment operation is complete, the RdPtrIncDone signal is asserted to indicate the completion to the transmit section.

The interface between the FIFO control logic and the data buffer is shown in ***Figure 4.4-10***. The WrPtr[2:0] signal represents the Write pointer, which points to the next empty location in the data buffer into which the next write can occur. The RdPtr[2:0] represents the Read pointer, which points to the FIFO location whose contents will be driven onto the read databus. The RegFileWrEn signal is derived from the SCIDATAWrEn input to the FIFO and the MODE bit such that writes to the transmit FIFO are ignored if it is already full. PCLK is used to clock the storage elements in the register file. The Register bank includes the output multiplexer for reads. This multiplexer uses the RdAddr[2:0] bus to select the buffer location whose contents have to be driven out onto the FIFOData[7:0] bus.

The transmit tide mark interrupt TXRIS is generated by the Transmit FIFO control logic. The transmit interrupt is asserted when the FIFO fill level falls below the TXTIDE mark set in the SCITIDE register. This interrupt is cleared

when the condition that caused the interrupt is no longer present, i.e. when the FIFO fill level rises above the TXTIDE value.

The TXFF signal and TXFE signal are readable through the SCIFR register. When asserted, the TXFF signal indicates that the transmit FIFO is full. When the TXFE signal is asserted, it indicates that the transmit FIFO is empty. TXFIFO fill level is readable through the SCITXCOUNT register. A write to the SCITXCOUNT register with any value will reset both the read pointer and the write pointer to zero and SCITXCOUNT will become zero.

The TXFLTE7Full signal is asserted when there is less than or equal to seven full locations within the TXFIFO. It is used in conjunction with TXRIS to generate single and burst request information to the DMA interface.

As a test feature, when the TESTFIFO signal is asserted, by writing a 1 to the SCITCR bit [1], data can be read from the TX FIFO by performing reads from the SCITDR register (test data register).

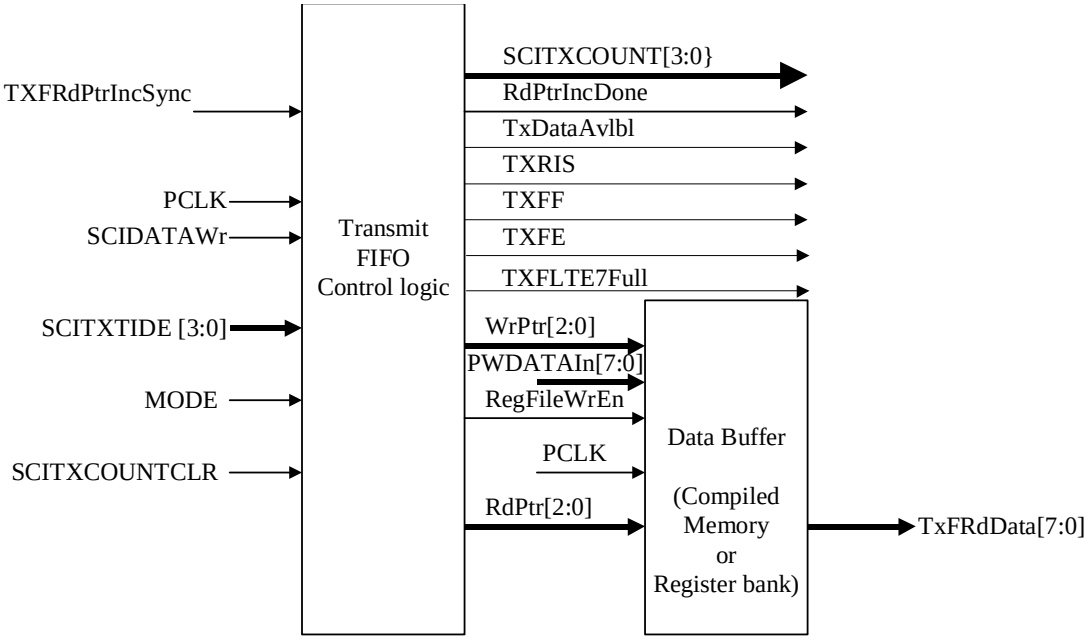


***Figure 4.4-10 Transmit FIFO Control logic interaction with Data buffer***

## 4.4.7 Receive FIFO

The receive FIFO is similar to the transmit FIFO in most respects. The receive FIFO is 9 bits wide and 8 locations deep. Writes to the receive FIFO occur from the Receive section. Reads from the receive FIFO occur through the APB interface.

For the purpose of evaluating the FIFOFillLevel, it would be convenient to have the read pointer and the write pointer operate on the same clock domain. Operating them on SCICLK would mean data cannot be served consistently onto the APB since two successive reads from the APB could occur with a separation of just one PCLK period (and the synchronisation of the read pointer increment signal in itself would consume two PCLK cycles). Hence both the read pointer and the write pointer operate on PCLK.

On receiving a full byte of data, the Receiver section drives data on the RXFIFOData [8:0] bus and asserts the write enable signal (RXFIFOWr) synchronous to SCICLK. This signal is synchronised to PCLK and is seen as the RXFIFOWrSync input of the receive FIFO. After every write, the write pointer is incremented.

On the read side, the content of the FIFO element selected by the current value of the read pointer is always driven on the RXFRdData [8:0] bus. The RXFRdPtrInc signal, which indicates when to increment the read pointer, is derived from a decode of read accesses to the SCIDATA register.

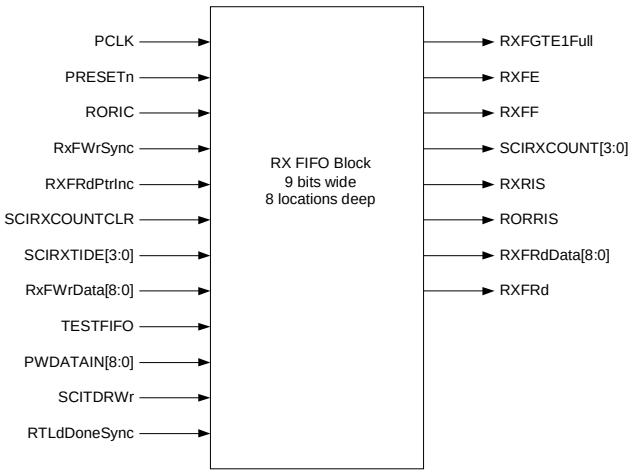


***Figure 4.4-11 Receive FIFO***

The RXRIS (Receive Interrupt) is asserted when the RX FIFO fill level is greater than or equal to the value of SCIRXTIDE programmed in the SCITIDE register. The interrupt is de-asserted when the FIFO contents have been read out such that the FIFO fill level is below the level programmed in the SCITIDE register.

The receive FIFO controller asserts the RXFRd signal when there is a read from the Receive FIFO. The synchronised version of this signal is used by the transmit and receive logic to reload the receive timeout value into a timer. When this timer counts down to zero, a Receive Timeout signal is generated.

The RXFE signal and the RXFF signal indicate the empty and full status of the receive FIFO. These signals are readable through the ‘RXFE’ bit and the ‘RXFF’ bit in the SCIFR register.

***Figure 4.4-12*** illustrates the interface between the FIFO control logic and the data buffer. Writes to the register bank occur to the location pointed to by the current value of the WrPtr[2:0] signal which indicates the value of the write pointer. In the case of reads, the contents of the location pointed to by the current value of the RdPtr[2:0] signal is always driven onto the RXFRdData[8:0] output lines. The control logic drives the RdPtr[2:0] lines with the value of the read pointer.

The Rx FIFO fill level is readable through SCIRXCOUNT register. A write to the SCIRXCOUNT register with any value resets both the write pointer and the read pointer. The Receive FIFIO fill level will then become zero.

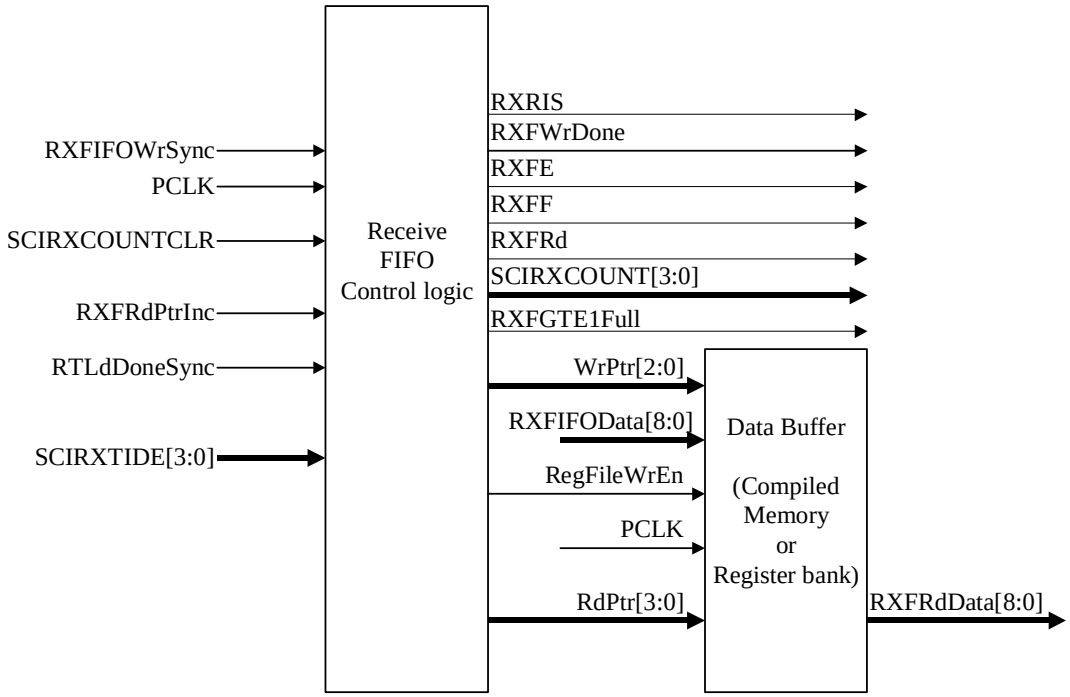


***Figure 4.4-12 Receive FIFO Control logic interaction with Data buffer***

The RXGTE1Full signal is asserted when there is one or more full locations within the RXFIFO. It is used in conjunction with RXRIS to generate single and burst request information to the DMA interface.

As a test feature, when the TESTFIFO signal is asserted, by writing a 1 to the SCITCR bit [1], data can be written to the RX FIFO by performing writes to the SCITDR register (test data register).

### 4.4.8 DMA Interface

This block provides an interface to connect to a DMA controller. It generates four DMA signals, two for the transmit request SCITXDMASREQ, SCITXDMABREQ, and two for the receive requests SCIRXDMASREQ, SCIRXDMABREQ. The block diagram is shown in ***Figure 4.4-13***.

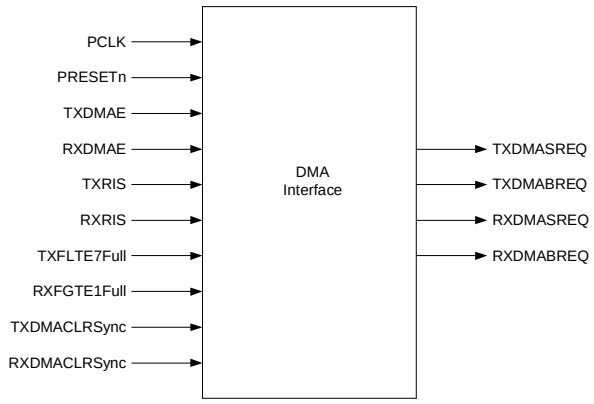


***Figure 4.4-13 DMA Transmit Interface***

#### DMA Transmit Interface

In normal operation, the DMA transmit interface is enabled when the transmit DMA is enabled i.e. when TXDMAE is high. It can also be enabled in test mode, in which case TESTFIFO also has to be high.

The level within the transmit FIFO is depicted by the signals TXRIS and TXFLTE7Full. The TXRIS is set when the data within the transmit FIFO is less than or equal to the programmable watermark level whilst the TXFLTE7Full is set when there is seven or less spaces within the transmit FIFO. The transmit DMA single request TXDMASREQ signal is set whenever TXFLTE7Full is asserted. The transmit DMA burst request TXDMABREQ is set when TXRIS is asserted. Therefore, it is possible for both TXDMASREQ and TXDMASREQ to be active together.

The DMA controller has to be programmed with the most suitable burst length implied by the programmed transmit FIFO watermark level.

The TXDMASREQ and TXDMABREQ signals are cleared by the application of the SCITXDMACLR signal from the DMA controller, which signifies that the end of a single/burst transfer is about to occur. They are also cleared when the transmit DMA function is disabled i.e. when TXDMAE is low.

#### DMA Receive Interface

In normal operation, the DMA receive interface is enabled when RXDMAE is high. It can also be enabled in test mode, in which case TESTFIFO also has to be high.

The level within the receive FIFO is depicted by the signals RXRIS and RXFGTE1Full. The RXRIS signal is set when the data within the receive FIFO is greater than or equal to the watermark level whilst the RXFGTE1Full is set when there is one or more full locations within the receive FIFO. The receive DMA single request RXDMASREQ signal is set when RXFGTE1Full is asserted. The receive DMA burst request RXDMABREQ is set when RXRIS is asserted. Therefore, it is possible for both RXDMASREQ and RXDMABREQ to be active together.

The DMA controller has to be programmed with the most suitable burst length implied by the programmed receive FIFO watermark level.

The RXDMASREQ and RXDMABREQ signals are cleared by the application of the SCIRXDMACLR signal from the DMA controller, which signifies that the end of a single/burst transfer is about to occur. They are also cleared when the receive DMA function is disabled i.e. when RXDMAE is low.

### 4.4.9 Test logic

The block diagram is shown in ***Figure 4.4-14***. It contains the test mode registers in the SCI and the logic for integration testing.

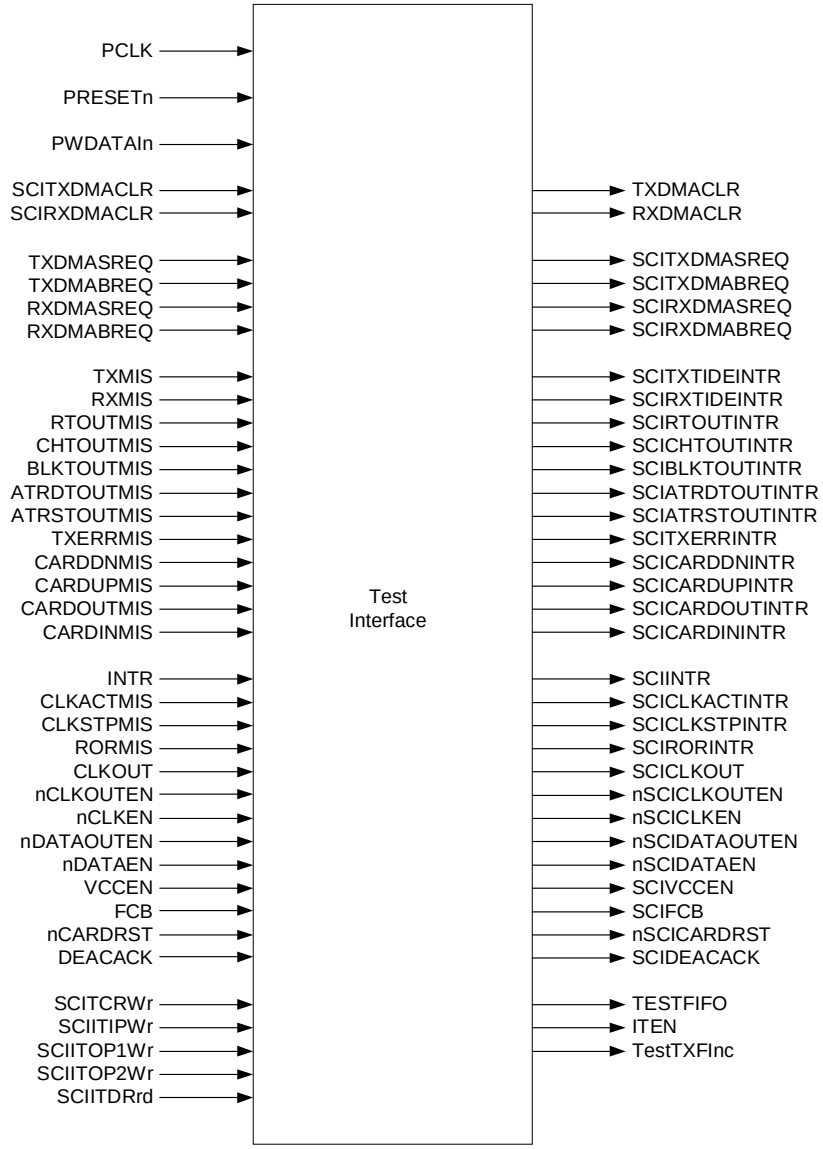


***Figure 4.4-14 Test Interface Block***

The integration test logic allows all inputs and outputs to be read from and written too.

The inputs can be read via the integration input (SCIITIP) register and the outputs can be written to via the integration output (SCIITOP1 and SCIITOP2) registers.

All the intra-chip inputs to the SCI enter the SCITest block and are multiplexed with the signals from the SCIITIP register under the control of the integration test enable (ITEN) signal.

Similarly, all the SCI intra-chip outputs are multiplexed with the signals from the SCIITOP1 and SCIITOP2 registers under the control of the ITEN signal.

Primary outputs can be written to, but the reading of them, when the peripheral is in isolation, has to be done through external logic back to the primary inputs. When connected in a typical system, the primary outputs are read by the terminating device, and similarly the inputs will be driven by a source device.

### 4.4.10 Synchronisers

All synchronisers for signals crossing clock domains have been grouped into two sub-blocks – one for signals entering the SCICLK domain (SciSynctoSCICLK) and the other for signals crossing over from the SCICLK domain into the PCLK domain (SciSynctoPCLK).

The signals crossing clock domains are double synchronised with flip-flops operating on the rising edge of the clock into which the signals get synchronised.

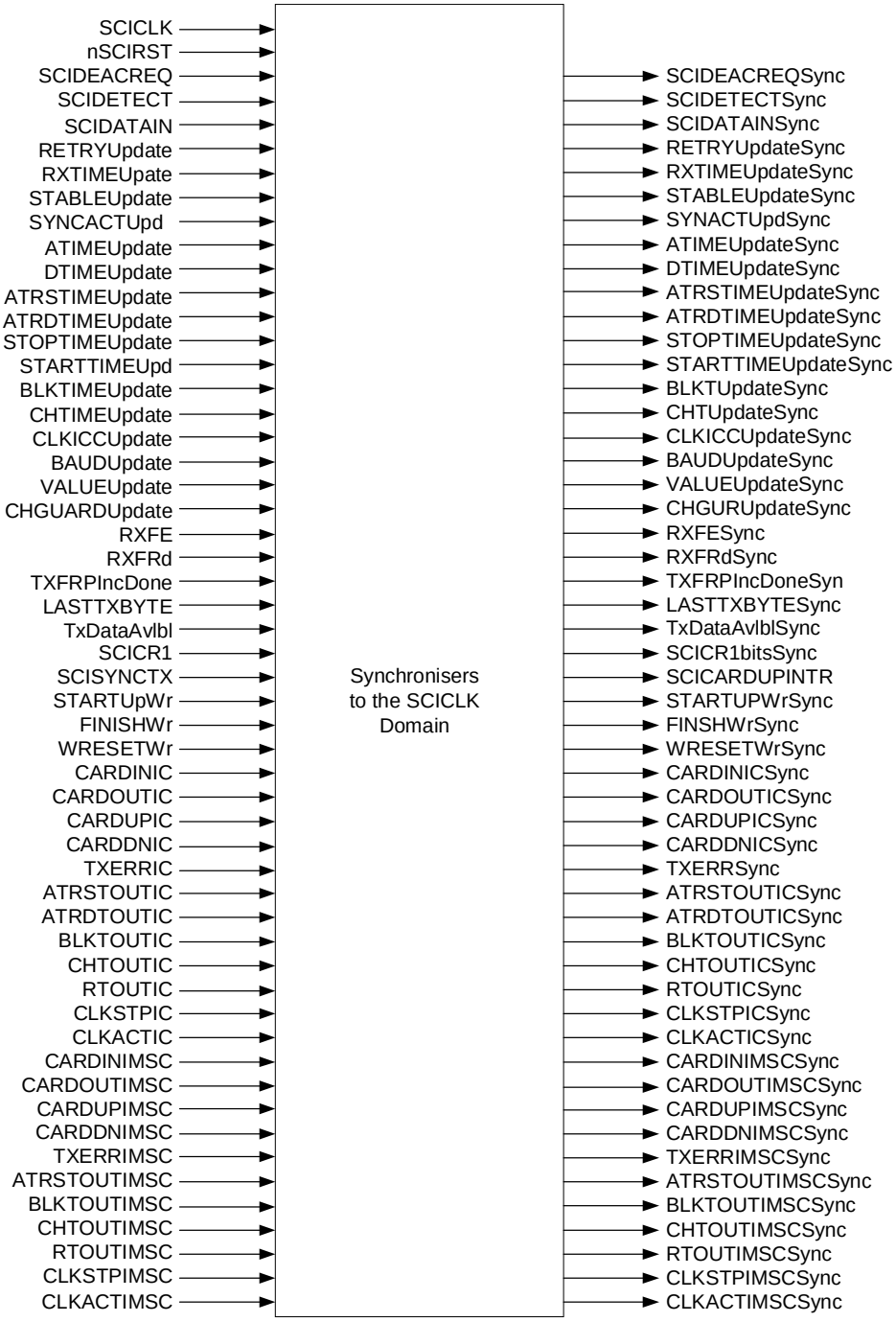


***Figure 4.4-15 Synchronisers for signals crossing into SCICLK domain***

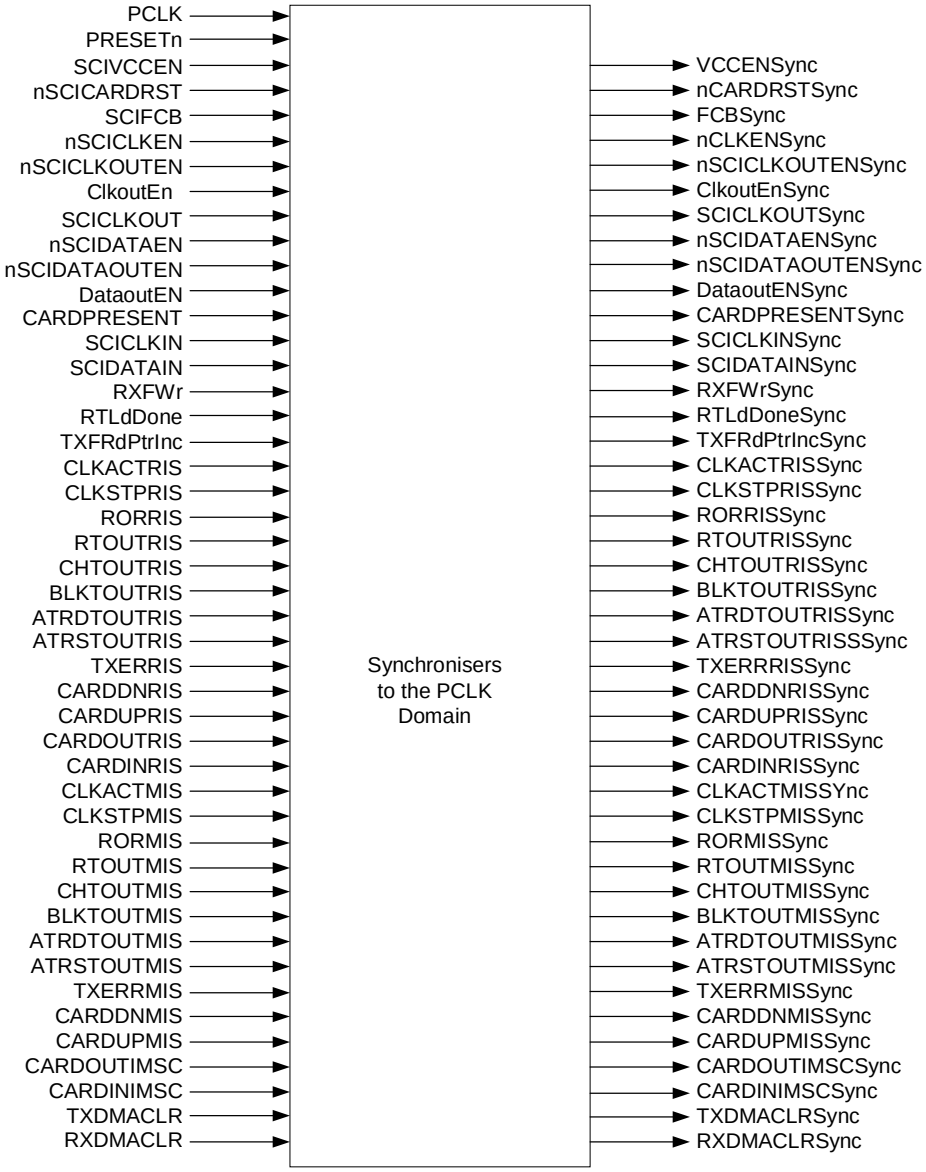


***Figure 4.4-16 Synchronisers for signals crossing into PCLK domain***

### 4.4.11 Revision Designator block

The block diagram is shown in ***Figure 4.4-17*****.** It contains a single AND gate to be used as a placeholder cell to mark the Revision of the controller. The 2 input pins are tied-off at the top level of the hierarchy. These "TieOffs" can be identified during layout and re-wired to "VDD" or "VSS" if needed.

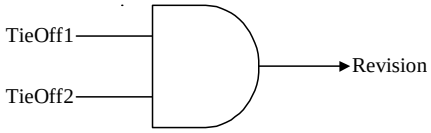


***Figure 4.4-17 Revision Designator Block***

# 5 SCI IMPLEMENTATION DETAILS

## 5.1 Design Hierarchy

The design hierarchy in the SCI is shown in ***Figure 4.4-2***. A short description about the functionality of each sub-block is also given within brackets.

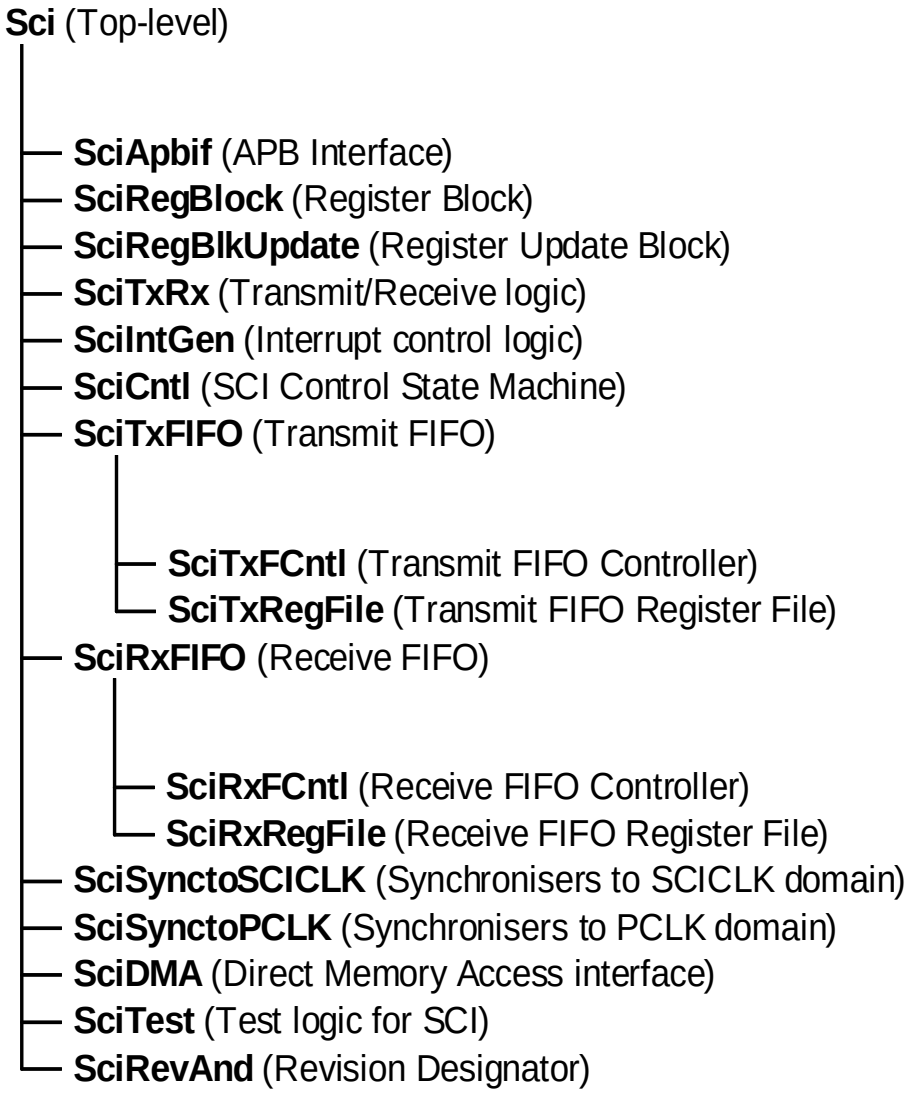


***Figure 5.1-1 SCI Design Hierarchy***

## 5.2 SCI Top Level Block (Sci)

The top-level block is a structural block which instantiates and interconnects the main functional blocks in the SCI.

## 5.3 SCI APB Interface (SciApbif)

The APB Interface generates decodes for writes into the registers in the address space of the SCI. It also contains the read interface for the SCI.

An internal write data bus, PWDATAIn[15:0], is generated by gating the write data bus input to the SCI, PWDATA[15:0], with PSEL and PWRITE. This prevents the data bus internal to the SCI from toggling when the device is not being written into or when it is not selected.

In order to reduce the load on the PADDR[11:2] bus, it is clocked internally into a set of d-types (LatchedPADDR[11:2]) on the rising edge of PCLK. Similarly, to reduce loading on the PENABLE input line, an internal signal, IntPENABLE, which is one PCLK-wide, is generated. PSEL is AND-ed with the inverted version of PENABLE and is clocked through a flip-flop clocked by PCLK to generate IntPENABLE.

### 5.3.1 Write Interface

For writes, an internal signal, Wr is generated. PSEL and PWRITE are AND-ed with the inverted version of PENABLE and this signal is clocked through a flip-flop clocked by PCLK to generate the Wr signal. The write strobe for each register is generated with a decode of LatchedPADDR[7:2] and Wr. Data is clocked into the a register on the rising edge of PCLK when the respective registers write enable is asserted.

### 5.3.2 Read Interface

The read interface involves an output multiplexer followed by a bank of flip-flops, which drive data onto the data bus. These flip-flops are clocked by the rising edge of PCLK. For reads, an internal signal, RdEn is generated by AND-ing PSEL with the inverted version of PWRITE and the inverted version of IntPENABLE. The output multiplexer is controlled by a combinatorial decode of RdEn and PADDR[11:2]. The resulting output is clocked into the output register on the rising edge of PCLK. Data is sampled by the APB Bridge on the next rising edge of PCLK when PENABLE is de-asserted.

The external data bus from multiple peripherals could be connected using one of many techniques so as to get a single read data bus driving the APB Bridge. The connection could be a multiplexer bus implementation or an OR bus implementation. For an OR bus implementation it is required that the peripheral drive zeros on the data bus when it is not selected. In the SCI, the output multiplexer is left such that, when the device is not selected, it drives zeroes onto the read data bus. This ensures that there is no restriction placed on the type of external data bus connection used.

## 5.4 SCI Register Block (SciRegBlock)

This block contains the normal mode registers in the SCI. In addition, it contains the PCLK-domain buffers for the synchronisation of some of the SCI registers to the SCICLK domain. This block also contains the logic that generates and removes the interrupt clear signals for interrupts that are asserted in the SCICLK domain but need to be de-asserted by writes from within the PCLK domain.

### 5.4.1 Generation of Interrupt Clear signals

This block also generates the signals that clear the interrupts and the corresponding bit locations in the SCIICR register.

These clear signals are generated when a ‘1’ is written into the corresponding bit location of the SCIICR register. For interrupts that are generated in the SCICLK domain, the clear signal that is generated in SciRegBlock is synchronised to the SCICLK domain. A delayed version of this signal is generated and a one SCICLK wide pulse is generated when a rising edge on the clear signal is detected. This pulse clears the raw interrupt. The raw

interrupt value gets synchronised back to the PCLK domain and once it has been de-asserted, the corresponding bit location in the SCIICR register is cleared to 0.

### 5.4.2 Synchronisation of SCI registers to the SCICLK domain

A 2-buffer technique is used to synchronise the contents of control registers like SCIATRSTIME to the SCICLK domain. Out of the two buffers, the first stage is located in the PCLK domain in the Register Block (SciRegBlock) and the second stage is located in the SCICLK domain in the Register Update block (SciRegBlkUpdate).

When there is a write to the SCIATRSTIME register, the first stage buffer is written with the new data. Also, the ATRSTIMEUpdate signal toggles to indicate to the SCICLK domain that the synchronisation process has to start. This signal is double synchronised to the SCICLK domain and is seen as the ATRSTUpdateSync signal in the SCICLK domain. A delayed version of this signal is generated by clocking this signal through a D-type clocked by SCICLK. By XOR-ing the ATRSTUpdateSync signal with its delayed version, a load pulse that is one-SCICLK wide is generated on the ATRSTIMEWrEn signal. When the ATRSTIMEWrEn signal is asserted, the contents of the first stage buffer are copied into the second stage buffer in the SCICLK domain. A similar mechanism is used to synchronise the other control registers to the SCICLK domain, namely SCIRETRY, SCIRXTIME, SCIATIME, SCIDTIME, SCIATRSTIME, SCIATRDTIME, SCICLKICC, SCIBAUD, SCIVALUE, SCICHGUARD, SCISTABLE, SCIBLKGUARD, SCIBLKTIMELS, SCIBLKTIMEMS, SCICHTIMELS, SCICHTIMEMS, SCISTOPTIME and SCISTARTTIME.

## 5.5 SCI Register Update Block (SciRegBlkUpdate)

This block contains the SCICLK domain buffers that form part of the two-buffer synchronisation mechanism used to synchronise control registers such as SCIATRSTIME etc. When the ATRSTUpdateSync signal toggles, it indicates that the second stage buffers need to be updated. A delayed version of ATRSTUpdateSync signal is generated by clocking it through a D-type clocked by SCICLK. An edge on the ATRSTUpdateSync signal is detected by XORing the ATRSTUpdateSync signal with its delayed version, giving a one SCICLK-wide pulse on the ATRSTIMEWrEn signal. When the ATRSTIMEWrEn signal is asserted, the contents of the first stage buffer are copied into the second stage buffer. A similar mechanism is used to synchronise the contents of the other control registers to the SCICLK domain, namely, SCIRETRY, SCIRXTIME, SCIATIME, SCIDTIME, SCISTABLE, SCIATRSTIME, SCIATRDTIME, SCICLKICC, SCIBAUD, SCIVALUE, SCICHGUARD, SCIBLKGUARD, SCIBLKTIMELS, SCIBLKTIMEMS, SCICHTIMELS, SCICHTIMEMS, SCISTOPTIME and SCISTARTTIME.

The 16 bit registers SCIBLKTIMELS and SCIBLKTIMEMS are combined within the SciRegBlkUpdate block to form the 32 bit register BLKTIME and this is only updated only when a write to its lower 16 bits is performed. Similarly, the 16 bit registers SCICHTIMELS and SCICHTIMEMS are combined within the SciRegBlkUpdate block to form the 32 bit register CHTIME and this is only updated only when a write to its lower 16 bits is performed.

## 5.6 SCI Transmit Receive logic (SciTxRx)

This block performs the main functionality of the SCI with the help of control information from the SCI control block (SciCntl). This block drives all card signals namely nSCIDATAOUTEN, nSCIDATAEN, SCICLKOUT, nSCICLKOUTEN, nSCICLKEN, nSCICARDRST, SCIFCB and SCIVCCEN. This block also generates the interrupt signals such as CARDINIS, CARDOUTIS, CARDUPIS, CARDDNIS, ATRSTOUTIS, ATRDTOUTIS, CHTOUTIS, BLKTOUTIS, RTOUTIS, CLKSTOPRIS and CLKACTRIS.

This block contains all the timers in the SCI namely SCIACTTIME, SCIDATATIME, SCICLKICCCNT, SCIBAUDCNT, SCIVALUECNT and SCIRTSTBPRECNT. The SCIACTTIME counter is used for seven different operations namely debounce period measurement, activation event time measurement, ATR start time measurement, ATR duration measurement, warm reset event measurement, deactivation event time measurement and clock stopping and starting control times.

The SCIDATATIME counter is used for four different operations namely character timeout generation, block timeout generation, character guard time measurement and block guard time measurement.

After reset, the Card Out interrupt (CARDOUTIS) and the Card Powered Down interrupt (CARDDNIS) interrupts are set. The SCIDETECT signal indicates that a Smart Card is present in the interface. When SCIDETECT is asserted, the SciCntl block asserts the DBLDEN (Debounce Timer Load Enable) and the DBEN (Debounce Timer Enable) signals to load the debounce time (SCISTABLE) into the SCIACTTIME timer and to enable the timer. Any writes to the SCISTABLE (Debounce time) register while the SCIACTTIME counter is counting down will not affect the current counting and will take effect only when the load event occurs next time. After the debounce time, the SCIACTTIME timer sends the DBTIMEOUT (Debounce Time over) signal to the SciCntl block to indicate that a valid card is present. On detecting the DBTIMEOUT signal, the SciCntl block asserts the CARDPRESENT signal, which is used to set the Card In interrupt (CARDINIS) and reset the CARDOUTIS interrupt in the SciTxRx block.

If the EXDBNCESync signal is cleared, the SCIRTSTBPRECNT timer will act as a pre-scaler for the SCIACTTIME counter when it is used to measure debounce time. In this case, the SCIACTTIME timer will count down only whenever SCIRTSTBPRECNT wraps from 0000 to FFFF. If the EXDBNCESync signal is set, the SCIRTSTBPRECNT is bypassed and SCIACTTIME timer counts down on every SCICLK.

The SCICLKICCCNT counter is used to generate a Smart card clock with a duty cycle of 50% to drive the external pad. This clock is in turn used by the Card. This counter is enabled only when there is a valid card present in the interface. The value of the counter can be changed by writing to the SCICLKICC register.

### 5.6.1 Card Activation Sequence

After software writes into the STARTUP bit, the SciCntl block asserts the ACTSEQLDEN signal and the ACTSEQEN signal. The ACTSEQLDEN signal is used to load the activation event time into the SCIACTTIME timer and a value of 2 into the internal ActCnt timer. Also, this signal is used to reset the Card Powered down interrupt (CARDDNIS).The ACTSEQEN signal enables the activation sequence. During the activation sequence, whenever the activation event timer (SCIACTTIME) counts down to zero, the internal ActCnt timer is decremented. The SCIACTTIME counter counts down with every pulse on the CLKICCEN signal. The CLKICCEN signal is a stream of pulses with a period corresponding to the value programmed into the SCICLKICC register and with a pulse width of one SCICLK period.

When ActCnt is 2 and the SCIACTTIME counter has counted down to 0 and the D-input of CLKICCEN is asserted, IOEN and SCIVCCEN are asserted. When the IOEN signal, which is one REFCLK wide, is asserted, the nSCIDATAOUTEN signal and the nSCIDATAEN signal, which drive the output enable signals of the pads, are de-asserted to place the data lines at high impedance. The assertion of the SCIVCCEN signal enables power to the card.

When ActCnt is 1 and the SCIACTTIME counter has counted down to 0 and the D-input of CLKICCEN is asserted, ClkoutEn is asserted. When ClkoutEn is asserted, the SCI enables the off-chip Clock buffer through nSCICLKEN and drives the 50% duty cycle Smart Card clock through either the nSCICLKOUTEN output or the SCICLKOUT output depending on the CLKZ1 bit in the SCICR1 register.

When ActCnt is 0 and the SCIACTTIME counter has counted down to 0 and the D-input of CLKICCEN is asserted, the SCI de-asserts the Card reset (nSCICARDRST) and asserts the Activation sequence over (ACTSEQOVER) signal to the SciCntl block. The ACTSEQOVER signal is also used to set the Card Powered Up interrupt (SCICARDUPIS).

When the SciCntl block detects the ACTSEQOVER signal, it asserts ATRSLDEN to load the ATR start time into the SCIACTTIME timer. The count enable for the ATR start time counter (SCIACTTIME) is generated within the SciTxRx block, by latching this load enable. Software has to set the MODE bit to zero (Reception) to start ATR reception prior to the start of the activation sequence. The ATR start time counter is disabled when it receives a valid start bit after the end of the activation sequence. This timer counts down with every pulse of CLKICCEN. If the timer reaches zero before it is disabled, the ATR start timeout (ATRSTIMEOUT) signal is set to generate the ATR start timeout (SCIATRSTOUTIS) interrupt.

The SciCntl block asserts the ETULDEN signal when it sees a low on the data input line and if the MODE bit is set for reception. This signal is used in this block to load half of the SCIVALUE into the SCIVALUECNT counter and the SCIBAUD value into the SCIBAUDCNT counter. When the SCIBAUDCNT counter counts down to 0 and when the SCIVALUECNT counter counts down to 1, a one SCICLK-wide pulse is created on the ETU signal to the SciCntl block. This indicates that half an etu has elapsed. Thus, the mid point of the start bit is detected. If the data input line is low after this half etu period, then a valid start bit is said to have been detected.

When the SCIBAUDCNT counter and the SCIVALUECNT counter reach their minimum values, they are reloaded with the values programmed into the SCIBAUD register and the SCIVALUE register respectively. From the time when the ETULDEN signal is detected, the first pulse on the ETU signal comes after a time duration of half an etu. Subsequent pulses on the ETU signal are one etu apart. Any change in the values programmed into the SCIBAUD register and the SCIVALUE register takes effect immediately.

### 5.6.2 Receive Logic

After a valid start bit is detected, the SciCntl block asserts the RXDATAEN signal and the ATRDCNTEN signal. The RXDATAEN signal enables the Receive logic of the SciTxRx block. The ATRDCNTEN signal enables the ATR duration timer (SCIACTTIME). The ATR duration time is loaded into the SCIACTTIME counter by detecting the leading edge on the ATRDCNTEN signal. The ATR duration counter (SCIACTTIME) decrements on every etu pulse. The counter is disabled when software resets the ATRDEN bit in the SCICR1 register after all the data in the ATR have been received. The counter is also disabled if the SciCntl block detects a warm reset or if the MODE bit is set to ‘Transmit’ by clearing the ATRDCNTEN input. If the ATR duration time expires before any of these events, the ATR duration timeout (ATRDTIMEOUT) signal will be set to generate the ATR duration timeout interrupt (SCIATRDTOUTIS).

After a valid start bit is detected, the DataActCnt counter is loaded with a value of 7 (given that there are 8 data bits in a character) and then decrements on every etu. The receive logic samples the data line on every etu and stores the sampled data bit in the MSBit of an internal shift register (ShiftReg). With every etu, the shift register is shifted right by one bit position.

If the SENSE bit is set, the sampled value of the data bit is inverted and then stored in the shift register. When the internal DataActCnt counter becomes zero, the parity logic is enabled.

The Receive parity error is determined by XORing all the data bits in the shift register with the RX parity control bit and the actual parity bit received. The parity error information is latched onto the RxParityErr signal. When the Parity error bit (RxParityErr) is updated, the contents of the Receive shift register (ShiftReg[7:0]) are copied into the 8 LSBits of another buffer (RxFWrData[8:0]).

One etu after the RxParityErr bit has been updated, the RxParityErr bit is copied into bit 8 of the RxFWrData[8:0] buffer and a write to the FIFO is initiated. If the ORDER bit is set, the control logic swaps the data bits when the received data is copied from the Receive Shift register into the RxFWrData[8:0] buffer.

Take as an example a byte value of 0xAF.

Reception with SENSE = 0, ORDER = 1, RXPARITY = 1 [odd] and RXNAK = 0 [T1 mode] will cause the value written to the FIFO to be 0X0AF, where the most significant bit is the Parity Error bit.

If ORDER = 0, with all other settings remaining the same, the received data written into the FIFO would be 0x0F5.

If SENSE = 1, with all other settings remaining the same, the data bits and the parity bit are inverted before being shifted into the shift register. The data written to the FIFO in that case would be 0x150.

If there is a parity error, the interface requests the card to retransmit the same data again. It does this by pulling the data line low for 2 etus on the next etu after the parity bit is sampled. A 3-bit counter, RETRYCNT, is used to keep track of the number of times retry requests are issued for a character. The request for retransmission is applicable only if the device is in the T0 mode and if the RETRYCNT counter has a value less than the value programmed into the retry register. The RETRYCNT counter is common to both the Transmit and the Receive sections. After error-free reception of each character, this counter is cleared to zero. When the retry limit is

exceeded, a parity error is registered and the RETRYCNT counter is cleared. When a parity error is registered, the received data is stored in the FIFO with the Parity error bit set.

If the RXRETRY value is programmed as zero or if T1 mode is programmed, a parity error is registered the first time it is encountered and the received data is written into the FIFO with the Parity error bit set.

If the card transmits correct data within the RX RETRY limit, the RXFIFOWr signal is set for one SCICLK period to toggle the RXFWrEn signal. When the RXFWrEn signal toggles, it indicates to the Receive FIFO that a write has to occur.

The transfer of received data into the Receive FIFO is initiated 10.5 etus after the start bit is detected. In the case where there is a retry, the data line is pulled low for 2 etus starting from 10.5 etus from the time when the start bit was detected. Hence reception is said to be complete 12.5 etus after the start bit was detected. After the completion of reception at 10.5 etus or 12.5 etus, the character time programmed into the SCICHTIME register, is loaded into the SCIDATATIME counter. This counter counts down with every etu. If the start bit of the next character is not received within the programmed time, then this counter expires, and generates the CHTIMEOUT signal, which is used to generate the SCICHTOUTIS interrupt.

On completion of reception, the SciTxRx block asserts the RXOVER output signal to the SciCntl block. The SciCntl block transitions out of the Receive state and asserts the CHTIMEEN signal to enable the SCIDATATIME counter in the SciTxRx block. The CHTIMEEN signal is cleared when card sends the next valid start bit or the interface is set to transmit. Before this event, if the timer expires the character timeout indication signal will be set to generate character timeout signal.

The SCIRTSTBPRECNT counter also used in the determination of the Receive Read Timeout interrupt generation. It is loaded with the read timeout value programmed in the SCIRXTIME register. This counter will start counting down only if there is a byte in the Receive FIFO, as indicated by synchronised RX empty signal (RXFESync). When the SCIRTSTBPRECNT counter counts down to one, the RTIMEOUT signal is generated which is used to generate the Receive Read TimeOut interrupt (SCIRTOUTIS). This counter is reloaded when there is a read from the Receive FIFO. In addition, reload occurs when the condition common to all timeout counters, occurs.

RTLdDone toggles whenever the SCIRTSTBPRECNT counter is reloaded due to a read from the Receive FIFO.

### 5.6.3 Transmit Logic

When switching from reception to transmission, two requirements must be met:

- The minimum number of etus between the leading edge of the last received start bit and the leading edge of the first transmitted start bit. (11 etu in the case of T1 mode, 12 etu in the case of the T0 mode)
- The minimum number of etus etu between the end of reception and the start of transmission corresponding to the Block guard value programmed in the SCIBLKGUARD register.

If the MODE bit is set for transmission, the SciCntl block asserts the BLKGDLDEN signal and the ETULDEN signal. On detecting the ETULDEN signal, half the value of SCIVALUE is loaded into the SCIVALUECNT counter and the SCIBAUD value is loaded into the SCIBAUDCNT counter. These counters are used to time the duration of an etu. The BLKGDLDEN signal is used to load the Block Guard value into the SCIDATATIME counter.

After the BLKGDLDEN signal is asserted, the SciCntl block asserts the BGCNTEN signal. If the BGTEN bit of SCICR1 is reset or zero block guard value is programmed in the SCIBLKGUARD register, the internal logic asserts the BLKGUARDOVER signal immediately. If a non-zero Block guard value is programmed and the BGTEN bit is set then, the Block Guard timer (SCIDATATIME) counts down on every etu. When this timer expires, the BLKGUARDOVER signal is asserted. On detecting the BLKGUARDOVER signal, the TxDataAvlblSync signal and the ETU signal, the SciCntl block asserts the TXSTRBITLDEN signal. This signal is used by the Transmit logic to drive the start bit.

The interface finishes reception 10.5 etu after the start bit is detected.

In the case of T1 mode, the transmit logic then waits for 0.5 etu to meet the minimum character time requirement of 11 etu. In addition, the transmit logic waits for block guard time before transmission begins.

In the case of T0 mode the transmit logic waits for 1.5 etu after the end of reception to meet the minimum character time requirement of 12 etu. In addition, it waits for block guard time before transmission recommences. The transmit logic goes to the block guard state even if block guard is disabled or if zero block guard time is programmed. This is done in order to meet the minimum time requirement between the leading edge of the last received start bit and the leading edge of the next transmitted start bit.

After ensuring that the minimum time requirement between the end of reception and the start of transmission has been met, transmission starts if the TxDataAvlblSync signal is set. When the SciCntl block asserts the TXSTRBIT signal, the SciTxRx block transmits the start bit. At the same time, the data on the TxFRdData[7:0] input is transferred into the internal shift register and the internal DataActCnt counter is loaded with a value of 7. If the ORDER bit is set, the data bits in the TxFRdData[7:0] input are swapped and then loaded into the shift register. Then, the SciCntl block enables the transmit logic of the SciTxRx block by asserting the TXDATAEN signal. After the transmitter logic is enabled, with every pulse on the ETU signal, data bits are shifted out serially and the internal DataActCnt counter is decremented. The transmit data bit is XORed with the SENSE bit and the resultant signal drives the nSCIDATAOUTEN line. When the DataActCnt counter becomes zero, the parity logic is enabled. The parity bit is calculated by XORing all data bits in the internal shift register with the Transmit parity control bit. After driving the parity bit, the data line is placed in the high impedance state by de-asserting the nSCIDATAEN signal. If the transmission mode is set to T0, on the next etu, the data input line is sampled for a low to check for a retry request. A retry request is said to have been detected if the line is sampled low.

Take as an example a data byte value of 0x50 and consider that the SCI is programmed with SENSE = 1, ORDER = 1, TXPARITY = 0 [even] and TXNAK = 0 [T1 mode].

Since ORDER=1, the data bits are swapped and loaded into the Transmit Shift register as 0x0A. Finally, since SENSE=1, the data bits are inverted before being driven on to the nSCIDATAOUTEN line.

In the case of T0 mode, if a non-zero TXRETRY value has been programmed into the SCIRETRY register, a retransmission request from the card can occur one etu after the parity bit has been transmitted. When a retransmission request arrives, the interface waits for a time duration corresponding to the amount of time

required to satisfy the minimum character duration specifications. Then the interface waits for 4 etu, irrespective of the character guard time value programmed. Then retransmission starts.

At the end of retransmission if another retransmission request arrives, the interface first waits for the amount of time required to satisfy the minimum character duration specifications. Then the interface waits for 4 etu irrespective of the character guard time value programmed. Then retransmission starts.

If TXRETRY is programmed for a non-zero value and a retry request is detected, the interface retransmits the data and the internal RETRYCNT counter will be incremented. This process of retrying is repeated upto TXRETRY times. Each time data is retransmit and the RETRYCNT counter is incremented. When there is an error-free transmission, the RETRYCNT counter will be reset. If the card requests a retry for more number of times than that programmed in the SCIRETRY register, the interface sets the TXERR signal which is used to generate the SCITXERRIS interrupt. Software has to clear the Transmit FIFO for further transmission or reception to occur. The TXFIFOCLR signal indicates that the Transmit FIFO has been cleared. If the SCIRETRY register has been programmed with 0, the SciTxRx block sets the TXERR signal immediately after the first retry request is detected. In the case of T1 mode, TXERR is not generated.

In the case of error-free transmission, the start, data and the parity bits are transmitted. Then the interface waits for the amount of time required to satisfy the minimum character duration specifications. After this, the interface waits for a time duration corresponding to the character guard time before the next transmission.

This block uses the SCIDATATIME counter to wait for the character guard time period in the case of normal transmission and for 4 etus in the case of retransmission. The read pointer increment trigger to the transmit FIFO is issued only after error-free transmission. The SciCntl block waits for the read pointer increment done (TXFRPIncDoneSync) signal from the Transmit FIFO control logic to load the next data into the internal shift register.

Transmission proceeds until the MODE bit is cleared (Receive) and the Transmit FIFO becomes empty. After the last byte has been transmitted, the interface does not wait for character guard time. Instead, it asserts the transmission completion (TXOVER) signal to the SciCntl block. Using this signal, the SciCntl block enables the block timeout-generating timer (SCIDATATIME) by setting the BLKTIMEEN signal. The block time is loaded into the block timeout counter when the leading edge on the BLKTIMEEN signal is detected. The interface then waits for data from the card.

The block timeout counter is disabled when any of the following conditions occurs:

- The BLKEN bit in the SCICR1 register is cleared
- A valid start bit is received
- The MODE bit is set again for transmission.

If the SCIDATATIME counter expires before any of these events occurs, the block timeout (BLKTIMEOUT) indication signal will be set to generate the block timeout (SCIBLKTOUTIS) interrupt.

### 5.6.4 Clock Stop Mode

Envisaged as a power saving mode, the card clock can be stopped (held high or low) when it is perceived that there will be no further transmissions from the card until the next request for data is issued. The card clock can then be restarted prior to the next transaction occurring.

When clock stop is requested, by setting the CLKDIS bit within the SCICR0 register, the CLKSTOPLDEN signal is asserted for one SCICLK period from within the SciCntl block. It is used to load the SCIACTTIME counter with the SCISTOPTIME value within the SciTxRx block. In parallel, the SciCntl block CLKSTOPEN signal is asserted and subsequently causes the SCIACTIME counter to decrement on successive smart card clock cycles. On expiry, the CLKSTPEXP signal is asserted and in conjunction with the desired static clock level, defined by the CLKVAL bit within SCICR1, it is used to cleanly stop the card clock and assert the CLKSTOPPED signal. This latter signal is

fed back to the SciCntl block and is the source of the SCICLKSTPINTR interrupt. The card clock will remain static until a request to restart is issued or there are such events as a warm reset, activation or deactivation.

When clock restart is issued, by clearing the CLKDIS bit within the SCICR0 register, the CLKSTARTLDEN signal is asserted for one SCICLK period from the SciCntl block. It is used to load the SCIACTTIME counter with the SCISTARTTIME value within the SciTxRx block. In parallel, the SciCntl block CLKSTARTEN signal is asserted which clears the CLKSTPEXP signal, allowing the card clock to become active again. It also allows the SCIACTIME counter to decrement on successive smart card clock cycles and on its expiry, the SCICLKACTINTR interrupt is asserted, informing the system that further card transactions can continue.

### 5.6.5 Card Activation / Deactivation control signal generation

In this block, the signals that initiate activation, warm reset and deactivation are generated. Whenever a write occurs to the corresponding locations in Control Register 2 (SCICR2), the corresponding control signals toggle. These control signals are synchronised to the SCICLK domain. The synchronised signals are XORed with their delayed versions.

The pulses that result are latched within the SciTxRx block only when there is a valid card present. The latched signals will be held high till the next smart card clock. The latched signals are used in the SciCntl block to initiate activation, warm reset or deactivation.

Card deactivation is not only due to writes to the FINISH bit location of the SCICR2 register, but also due to other factors such as physical removal of the card, ATR start timeout after warm reset and detection of a deactivation request from the PMU. All the sources except the ATR start timeout are ORed to generate a single source – CARDDOWN. This signal is used in the SciCntl block to initiate card deactivation at any point of time after the presence of the card has been validated. The ATR start timeout after warm reset (WATRSTIMEOUT) signal is used in the SciCntl block to initiate card deactivation only from the state where it waits for the ATR start bit.

### 5.6.6 Card Deactivation Sequence

When the SciCntl block asserts the Deactivation Load Enable (DEACTLDEN) signal the deactivation time programmed into the SCIDTIME register is loaded into the SCIACTTIME timer and a value of 2 is loaded into the internal ActCnt timer. At the same time, the Card reset (nSCICARDRST) line is pulled low and the SCICARDUPIS interrupt is reset. During the deactivation sequence, whenever the activation event timer (SCIACTTIME) counts down to zero, the internal ActCnt timer is decremented. The SCIACTTIME counter counts down with every pulse on the CLKICCEN signal.

When ActCnt is 2 and the SCIACTTIME counter has counted down to 0 and the D-input of CLKICCEN is asserted, the SCI de-asserts the ClkoutEn signal. This, in turn, pulls the nSCICLKOUTEN , nSCICLKOUT and nSCICLKEN lines low so that the external clock is forced low.

When ActCnt is 1 and the SCIACTTIME counter has counted down to 0 and the D-input of CLKICCEN is asserted, the DataoutEn signal is de-asserted. This drives the external data lines (nSCIDATAOUTEN and nSCIDATAEN) low.

When ActCnt is 0 and the SCIACTTIME counter has counted down to 0 and the D-input of CLKICCEN is asserted, the SciTxRx block de-asserts the power line(SCIVCCEN) and also asserts the deactivation completion(DEACTOVER) signal. On the assertion of DEACTOVER and the de-assertion of SCIDETECTIntSync signal, the SciCntl block de-asserts the CARDPRESENT signal and starts another card session. Using the de-assertion of the CARDPRESENT signal, the SCICARDOUTIS interrupt is set and the SCICARDINIS interrupt is reset. Using the DEACTOVER signal, the SCICARDDNIS interrupt is set. When the DEACTOVER signal, the NOCARDEN signal and the DEACREQIntSync signal are asserted, the acknowledgement (SCIDEACACK) signal is asserted to the PMU. The SCIDEACACK is de-asserted immediately after the SCIDEACREQ is de-asserted.

All timeout counters remain at their minimum value once they have counted down fully. The counters are reloaded when the respective reload event occurs. When the corresponding reload value registers are written to the counter, reload occurs on the next SCICLK provided the counter has not already counted down to its minimum

value. All counters operate in the nibble mode when their respective nibble mode control bit is set. In this mode, the counters count down as if they are split into nibble wide counters.

Writing into the SCIICR register clears the interrupts corresponding to the bit in the write data that are set. The SCIRTOUTIS (Receive FIFO Read TimeOut) interrupt is an exception in that it cannot be cleared by software. The clear signals for the individual interrupts are generated by XOR-ing the synchronised version of the clear triggers with their delayed versions. The Receive FIFO read timeout interrupt will be set when the Receive read timeout counter (SCIRTSTBPRECNT) expires. This interrupt will be cleared when there is a read from the Receive FIFO or when the Receive FIFO is cleared through a write to the SCIRXCOUNT register.

For cards that are not compliant with the EMV standard, the clock and data lines are driven by software after the clock line and data line are activated. When the SYNCCARD bit in the SCICR1 register is set, the nSCICLKEN output and the Smart Card clock are controlled by the WCLKEN bit and the WCLK bit respectively, of the SCISYNCTX register. Similarly, when the SYNCCARD bit in the SCICR1 register is set, the nSCIDATAEN output and the nSCIDATAOUTEN output are controlled by the WDATAEN bit and the WDATA bit respectively, of the SCISYNCTX register. The two remaining signals, SCIFCB and nSCIRST can also be controlled by writing the appropriate values to the WFCB and WRST bits within the SCISYNCTX register when the SYNCCARD bit within the SCICR1 register is set.

## 5.7 SCI Interrupt Generation Block (SciIntGen)

All of the interrupts i.e. SCICARDININTR, SCICARDOUTINTR, SCICARDUPINTR, SCICARDDNINTR, SCITXERRINTR, SCIATRSTOUTINTR, SCIATRDTOUTINTR, SCIBLKTOUTINTR, SCICHTOUTINTR, SCIRTOUTINTR, SCIRORINTR, SCICLKSTOPINTR, SCICLKACTINTR, SCIRXTIDEINTR and SCITXTIDEINTR are created in this block by AND-ing their raw interrupts with their respective interrupt mask set clear bits and the inverse of their clear bits. The SCIINTR is the OR of the individual device interrupts.

## 5.8 SCI Control State Machine (SciCntl)

This block controls the operation of the SCI taking as inputs, the timeout signals and other control signals from the SciTxRx block.

The states in this state machine are:

| | |
|---|---|
| STNOCARD | : Idle state |
| STDEBOUNCE | : State where card debounce is done |
| STWAITFORSTART | : State to wait for software to initiate activation |
| STACTIVESEQ | : State where Card Activation sequence is carried out |
| STTXRX | : State to wait for transmission / reception to begin after card activation |
| STDEACT | : State where Card Deactivation sequence is carried out |
| STWRESET | : State where Warm Reset sequence is carried out |
| STCHKFORSTRBIT | : State where the Start bit is validated |
| STRXDATA | : State where data reception is carried out |
| STBLKGUARD | : State to wait for block guard time |
| STWAITFORDATAAV | : State to wait for transmit data to become available |
| STTXDATA | : State where data transmission is carried out |

STCLKSTOP : State where the card clock is stopped

STCLKSTART : State where the card clock is active, but no data transactions should occur.

After reset, the state machine stays in the STNOCARD state. The state machine transitions into this state from the STDEACT state after the deactivation sequence is completed. If the deactivation request is from the PMU, the SCIDEACACK output is asserted by the SciTxRx block when the state machine is in the STNOCARD state. The PMU, on detecting the SCIDEACACK signal, de-asserts its deactivation request (SCIDEACREQ).

When the SCIDETECTIntSync signal is high and the PMU has de-asserted its deactivation request, the state machine transitions to the card debounce state (STDEBOUNCE). On the transition, it asserts the DBLDEN signal for one SCICLK period to load the debounce time into the SCIACTTIME timer in the SciTxRx block. The decode of the STDEBOUNCE state is used as the count enable (DBEN) for the SCIACTTIME counter. When the state machine is in this state, if the SCIDETECTIntSync signal goes low or the PMU asserts its deactivation request again, the state machine transitions back to the STNOCARD state. If the state machine receives the debounce time over (DBTIMEOUT) signal from the SciTxRx block it enters the STWAITFORSTART state. On the transition, it generates the CARDPRESENT signal, which is held high till the deactivation sequence is complete.

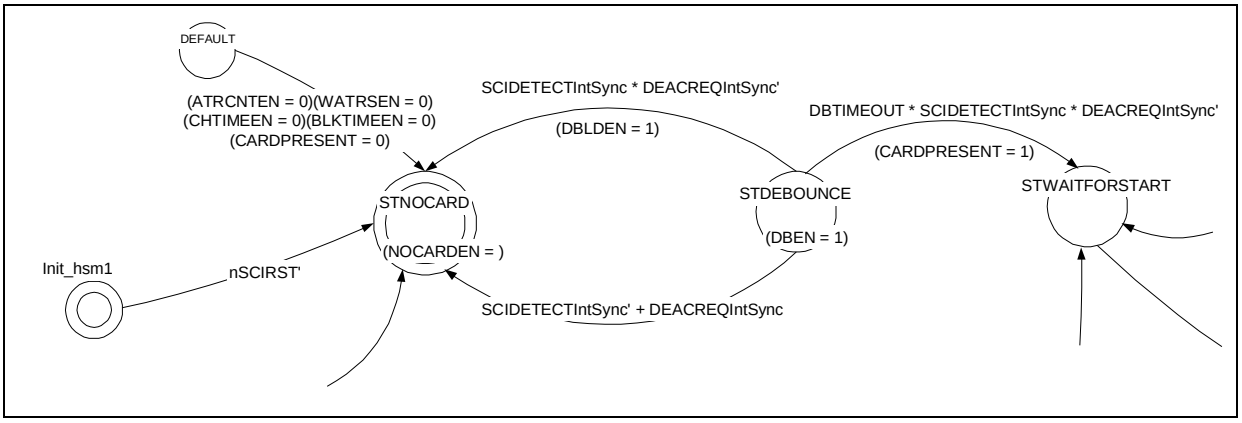


***Figure 5.8-1 A section of the SciCntl state machine controlling card debounce***

When the STARTUP input signal from the SciTxRx block is asserted, the state machine transitions to the activation sequence state (STACTIVESEQ). The decode of the STACTIVESEQ state, is driven as the ACTSEQ output signal to the SciTxRx block, where it triggers the control logic used to activate the card. After receiving the activation sequence over (ACTSEQOVER) signal from the SciTxRx block, the state machine transitions to the STTXRX state. On the transition, a signal is generated to load the ATR start timeout value into the SCIACTTIME counter in the SciTxRx block. An ATR start timeout interrupt is generated if the SCIACTTIME timer expires. Before the activation sequence is over, software has to program the MODE bit to 0 for reception.

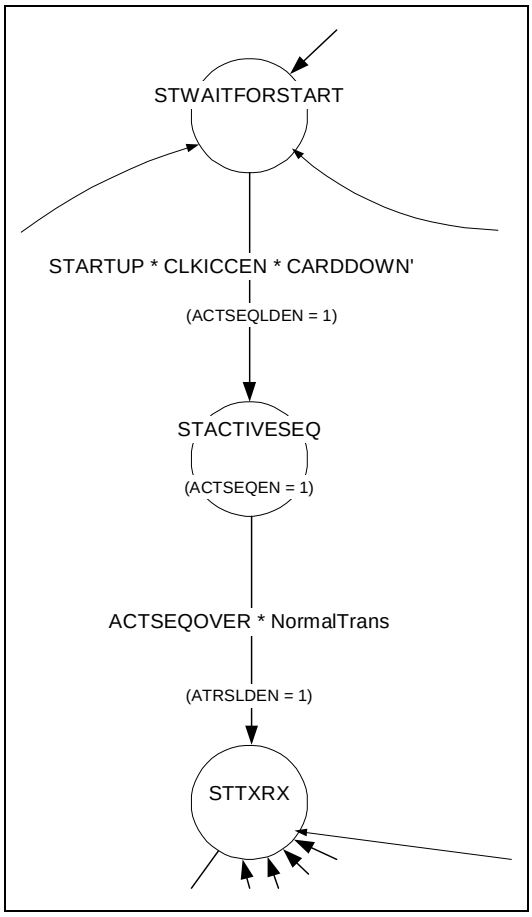


***Figure 5.8-2 A section of the SciCntl state machine controlling Card activation***

When the state machine is in the STTXRX state, if the MODE bit is set to 0 and a low is detected on the data input line, the state machine transitions to the STCHKFORSTRBIT state. On the transition, it generates the ETULDEN signal to load half the SCIVALUE into the SCIVALUECNT counter and the SCIBAUD value into the SCIBAUDCNT counter in the SciTxRx block, to detect the mid-point of the start bit. After half an etu, the input data line (SCIDATAINIntSync) is sampled again for a 0. If the data line is sampled low, a valid start bit is said to have been detected and the state machine enters the STRXDATA state else it transitions back to the STTXRX state to wait for another low on the data line. On the transition, the ATRDCNTEN signal is issued to disable the ATR start time counter and to load the ATR duration time into the same timer. The ATRDCNTEN signal is also used as count enable for the ATR duration timer.

The decode of the STRXDATA state, is driven as the RXDATAEN output signal to the SciTxRx block, where it enables the receive logic. After reception is complete, the SciTxRx block asserts the RXOVER input signal, upon which, the state machine transitions to the STTXRX state. On the transition, the character timeout signal (CHTIMEEN) Is asserted. This signal is kept asserted till the next valid start bit is detected or the MODE bit is set (Transmit) or card deactivation is initiated.

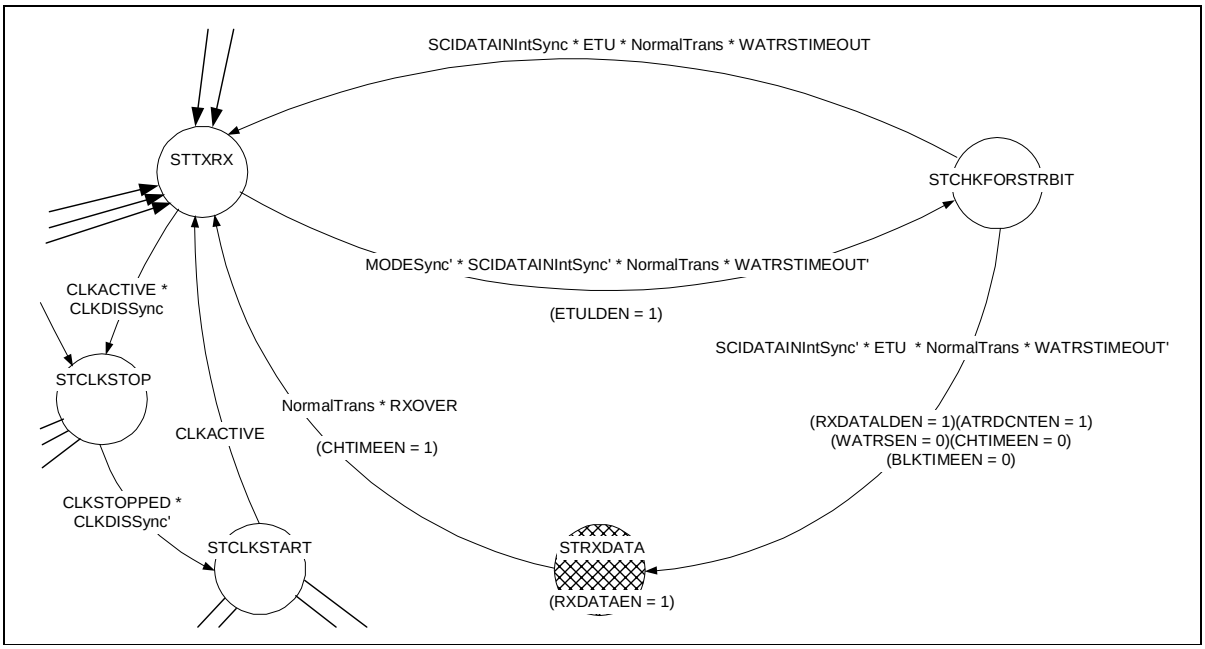


***Figure 5.8-3 Section of the state machine controlling data reception***

When the state machine is in the STTXRX state and the MODE bit is cleared to zero, if the CLKDIS bit is set to one, the state machine transitions to the STCLKSTOP state and the CLKSTOPLDEN and CLKSTOPEN signals are asserted. The SciTxRx block uses these to load and time the number of smart card clock cycles before the clock to the card becomes inactive. When in the STCLKSTOP state, if the CLKDIS bit is cleared, then the CLKSTARTLDEN and CLKSTARTEN signals are asserted and the state machine transitions to the STCLKSTART state. The SciTxRx block uses these signals to immediately restart the card clock and then time the number of smart card clock cycles before further card transactions can proceed. This is signified by the assertion of the SCICLKACTINTR interrupt.

When the state machine is in the STTXRX state, if the MODE bit is set to one, the state machine transitions to the STBLKGUARD state to ensure the minimum inactive period between the end of reception and the start of transmission. On the transition, the state machine generates the ETULDEN signal that is used by the SciTxRx block to time half an etu duration.

When the state machine is in the STBLKGUARD state, it transitions to the STTXDATA state when all of the following conditions occur:

- The BLKGUARDOVER signal from the SciTxRx block is asserted
- The TxDataAvlblSync signal from the Transmit FIFO is asserted indicating that valid data is available in the Transmit FIFO.
- The RXNAKSync bit is set to T1 mode

On the transition, the TXSTRBITLDEN signal to the SciTxRx block is asserted to indicate that the start bit is to be driven.

When the state machine is in the STBLKGUARD state, it transitions to the STWAITFORDATAAV state if the BLKGUARDOVER signal from the SciTxRx block is asserted and if any of the following conditions occurs:

- The TxDataAvlblSync signal from the Transmit FIFO is de-asserted

- The RXNAKSync bit is set to T0 mode

When the state machine is in the STWAITFORDATAAV state, if the TxDataAvlblSync signal is asserted, the state machine transitions to the STTXDATA state to initiate transmission of data. On the transition the TXSTRBITLDEN signal is asserted. This signal is used by the SciTxRx block to drive the start bit.

When the state machine is in the STWAITFORDATAAV state, if the MODE bit is cleared (receive) and the TxDataAvlblSync signal is de-asserted, it indicates that there is no data to transmit and the state machine transitions to the STTXRX state. On the transition, the SCIDATATIME counter in the SciTxRx block is enabled by asserting the BLKTIMEEN signal. This signal is also used to load the block time into the SCIDATATIME counter.

When the state machine is in the STTXDATA state, if the TXOVER signal from the SciTxRx block is asserted, it indicates that transmission is over and the state machine transitions to the STTXRX state. On the transition, the BLKTIMEEN signal is asserted to enable the block timeout counter in the SciTxRx block.

When the state machine is in the STTXDATA state, if the CHGUARDOVER signal is asserted and the TxDataAvlblSync signal is not set, and if either of the following conditions is satisfied, the state machine transitions to the STWAITFORDATAAV state:

- The TXFRPIncDoneSync signal is asserted
- The TXFIFOCLR signal is asserted

When the state machine is in the STTXDATA state, the TXSTRBITLDEN signal is asserted and the state machine transitions back to the STTXDATA state when the CHGUARDOVER signal is asserted and the TxDataAvlblSync signal is set and any of the following conditions occurs:

- The TXFRPIncDoneSync signal is asserted
- The TXFIFOCLR signal is asserted
- The RETX signal (retransmit indication) from the SciTxRx block is asserted.

-

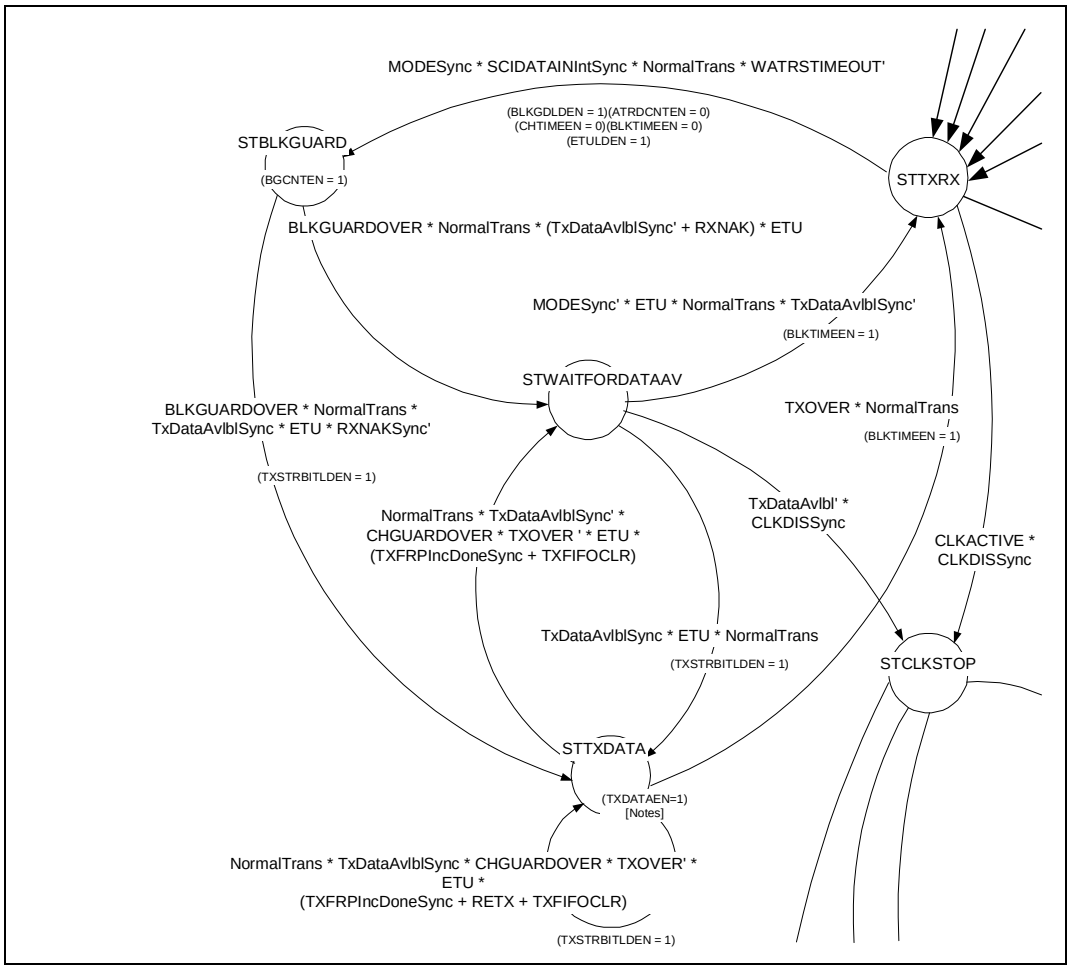


***Figure 5.8-4 Section of the state machine controlling data transmission***

When the state machine is in the STWAITFORDATAAV state, if the MODE bit is set to one and TxDataAvlbl is cleared to zero, then if the CLKDIS bit is set, the state machine transitions to the STCLKSTOP state and the CLKSTOPLDEN and CLKSTOPEN signals are asserted. Note that the setting of the CLKDIS bit can occur when the TX FIFO is not empty and in this case the state machine will wait until the TX FIFO becomes empty before entering the clock stop sequence. The SciTxRx block uses the CLKSTOPLDEN and CLKSTOPEN to load and time the number of smart card clock cycles before the clock to the card becomes inactive. When in the STCLKSTOP state, if the CLKDIS bit is cleared, then the CLKSTARTLDEN and CLKSTARTEN signals are asserted and the state machine transitions to the STCLKSTART state. The SciTxRx block uses these signals to immediately restart the card clock and then time the number of smart card clock cycles before further card transactions can proceed. This is signified by the assertion of the SCICLKACTINTR interrupt.

When the state machine is in the STTXRX, STCHKFORSTRBIT, STRXDATA, STBLKGUARD, STWAITFORDATAAV, STTXDATA, STCLKSTOP or STCLKSTART state, if the WRESET signal from the SciTxRx block is asserted, it indicates that the warm reset sequence has to be initiated and the state machine transitions to the STWRESET state.

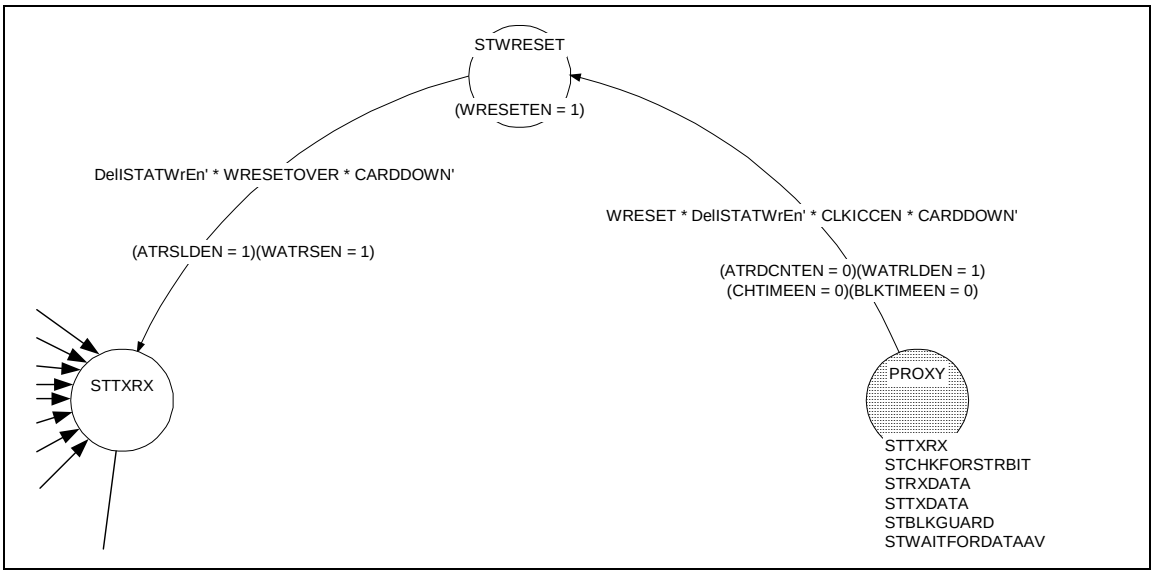


***Figure 5.8-5 Section of SciCntl state machine controlling initiation of Warm Reset sequence***

On the transition, the WATRLDEN signal is asserted. This signal is used in the SciTxRx block :

- To assert the nSCICARDRST signal
- To hold the data line at high impedance state
- To load the activation event time value into the SCIACTTIME timer to determine when to de-assert the card reset.

In the STWRESET state, the SCIACTTIME counter in the SciTxRx block counts down with every pulse on the CLKICCEN signal. After this counter expires, the WRESETOVER signal is asserted and the state machine transitions to the STTXRX state to start data reception. On the transition, the ATR start time is loaded into the SCIACTTIME counter. If the timer expires before a valid start bit is detected, the SciTxRx block asserts the WATRSTIMEOUT signal. On detecting this signal the state machine transitions to the STDEACT state. If the state machine is in the STCHKFORSTRBIT state, and if the WATRSTIMEOUT signal is asserted, the state machine transitions to the STDEACT state.

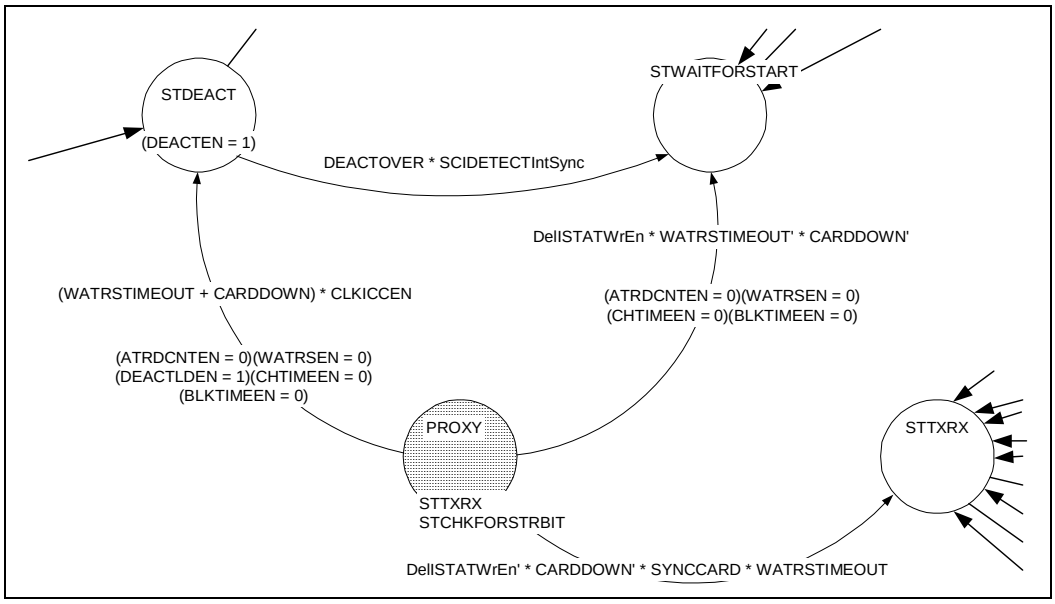


***Figure 5.8-6 Section of the SciCntl state machine initiating deactivation sequence due to ATR start timeout after warm reset***

When the state machine is at any state other than the STNOCARD state, the STDEBOUNCE state and the STDEACT state, if the CARDDOWN signal is asserted, the state machine transitions to the STDEACT state to deactivate the card. On the transition, it asserts the DEACTLDEN signal to load the deactivation time into the SCIACTTIME counter. The decode of STDEACT state (DEACTEN) is used to enable the deactivation logic in the SciTxRx block. On the assertion of the DEACTOVER input signal and the de-assertion of the SCIDETECTIntSync signal, the state machine transitions to STNOCARD state. On the transition, it de-asserts the CARDPRESENT signal. On the assertion of the DEACTOVER signal and the SCIDETECTIntSync signal, the state machine transitions to STWAITFORSTART state.

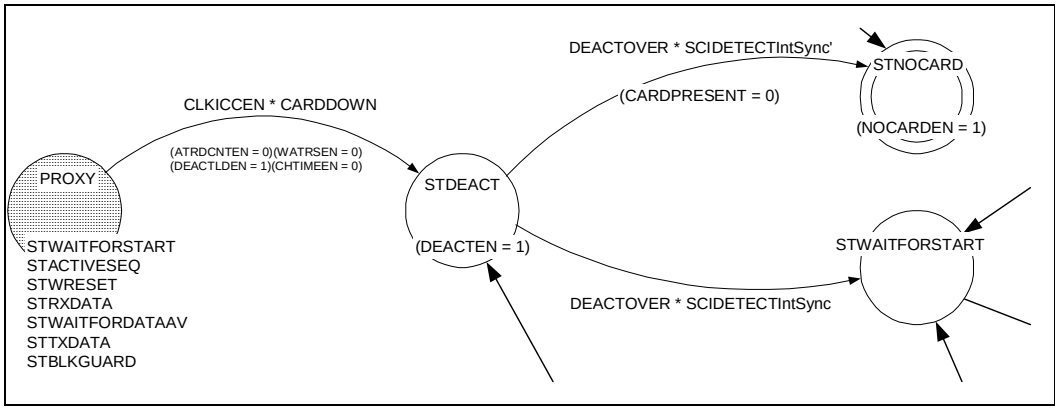


***Figure 5.8-7 Section of the SciCntl state machine showing initiation of deactivation sequence***

A write to the SCISYNCACT register indicates that the card is not compliant with the EMV standard. In this case, the internal logic in the SCI is bypassed during card activation and data transfer. The card control signals are

driven by software through writes to the SCISYNCACT register. Deactivation is still done by the internal control logic of the SCI. When the state machine is at any state other than the STNOCARD state, STDEBOUNCE state, STWAITFORSTART state or the STDEACT states, if a write occurs to the SCISYNCACT register, the state machine transitions to the STWAITFORSTART state.

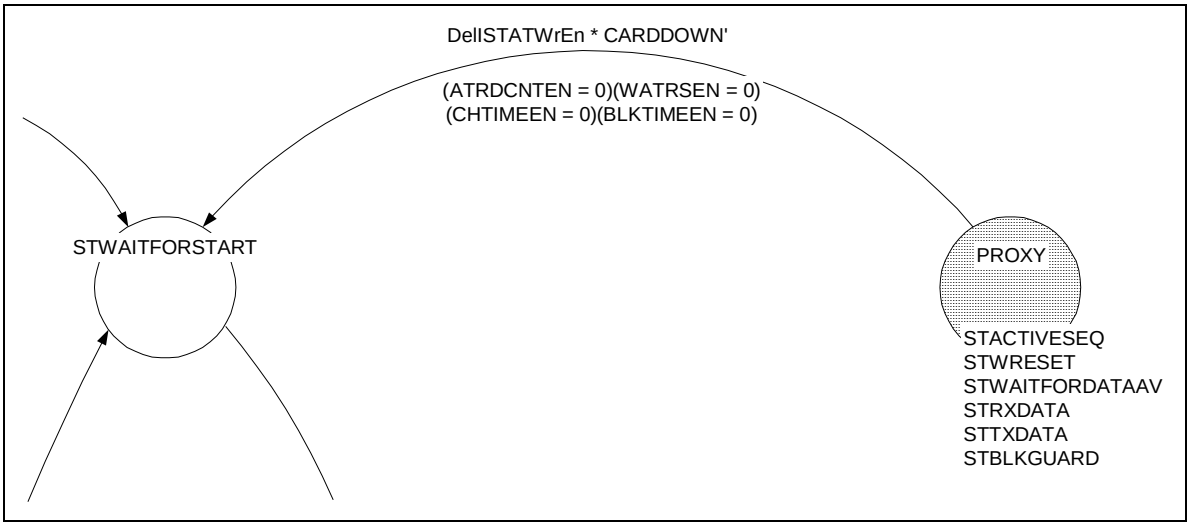


***Figure 5.8-8 Section of the SciCntl state machine controlling non-EMV-compliant cards***

***Note : During EMV-compliant card sessions software is not expected to write to the SCISYNCACT register. If the SCISYNCACT register is written to, the state machine aborts reception / transmission and transitions to the STWAITFORSTART state.***

When the state machine is in the STCHKFORSTRBIT, STRXDATA, STBLKGUARD, STWAITFORDATAAV or the STTXDATA state, if the SYNCCARD bit in the SCICR1 register is set, the state machine transitions to the STTXRX state. The state machine remains in this state as long as there is no write to the SCISYNCACT register and the CARDDOWN signal from the SciTxRx block remains de-asserted. This is to allow transmission and reception to proceed under software control through writes to the SCISYNCTX register and reads from the SCISYNCRX register.

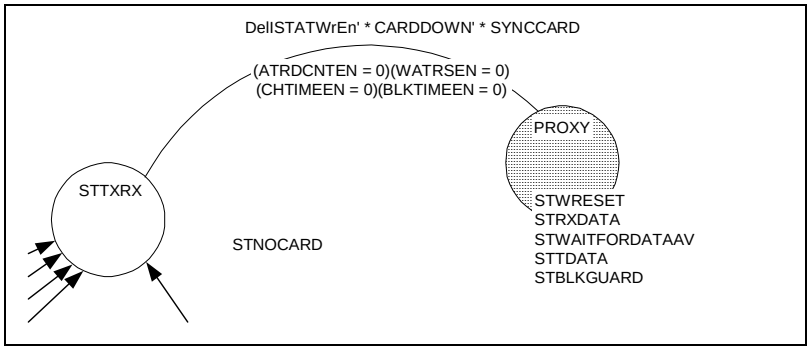


***Figure 5.8-9 Section of the SciCntl state machine showing card session through the SCICR1 register and the SCISYNCTX AND SCISYNCRX registers***

***Note : During EMV-compliant card sessions software is not expected to write to the SCISYNCACT register. If the SCISYNCACT register is written to, the state machine aborts reception / transmission and transitions to the STTXRX state.***

## 5.9 Transmit FIFO (SciTxFIFO)

The Transmit FIFO block instantiates the transmit FIFO control logic (SciTxFCntl) and the data storage elements (SciTxRegFile). The transmit FIFO is 8-bits wide and 8-deep. Writes to the transmit FIFO occur from the APB through the APB interface, which operates on PCLK. Reads from the Transmit FIFO occur from the Transmit logic, which operates on SCICLK.

For writes to the FIFO, the APB interface in the SCI drives the SCIDATAWrEn input to the transmit FIFO with decodes of writes to the SCIDATA register. The SCIDATAWrEn input pulse is one PCLK wide. When the SCIDATAWrEn input is asserted, the data on the PWDATAIn[7:0] input is clocked into the Register file (SciTxRegFile) location pointed to by the current value of the write pointer, on the rising edge of PCLK. The write pointer is controlled by the SciTxFCntl block. The Read pointer is also generated by the same block. For reads from the FIFO, the content of the Transmit Register file pointed to by the Read pointer, is driven onto the TXFRdData[7:0] output of the FIFO.

The TxDataAvlbl output signal indicates to the transmit logic that valid data is available in the FIFO for transmission. On detecting this signal, the transmit section starts transmission and when a byte has been transmitted, it asserts the Read pointer increment signal which is seen by the FIFO control logic as the TXFRdPtrIncSync input signal. After the Transmit FIFO control logic has completed the incrementing operation, it asserts the TXFRPIncDone (Read pointer increment done) signal. On detecting this signal, if there is further valid data available in the FIFO, the transmit section starts transmitting the data driven onto the TXFRdData[7:0] lines which correspond to the next data byte in the FIFO.

Thus writes, which are from the PCLK domain, occur without any handshake signals, but reads from the FIFO, which occur from the SCICLK domain, operate with suitable handshake signals through synchronisers.

The TXFF (Transmit FIFO Full) signal indicates the fill status of the FIFO. The assertion of the TXFF signal indicates that there are 8 valid bytes in the transmit FIFO. This signal is readable through the ‘TXFF’ bit in the SCIFR register. The Transmit FIFO Interrupt, TXRIS, is generated in the SciTxFCntl block. The transmit interrupt, TXRIS, is asserted if the FIFO fill level becomes less than the watermark level programmed in the SCITXTIDE register. The interrupt is de-asserted when the condition creating the interrupt is no longer present. The TXRIS signal is gated with its interrupt enable bit in the APB interface block (SciApbif) to generate the final SCITXTIDEINTR interrupt.

The contents of the TX FIFO can be read through the APB interface by performing read access to the test register SCITDR when the TESTFIFO signal is active.

## 5.10 Transmit FIFO control logic (SciTXFCntl)

The Transmit FIFO control logic controls the read and write pointers for the FIFO, besides controlling accesses to the FIFO register file. It also generates the FIFO fill level status signals and the Transmit FIFO service request, TXRIS.

The SCIDATAWrEn input is a one-PCLK-wide pulse driven by the write decode to the SCIDATA address. The TxFRdPtrIncSync signal is an indication originating from the transmit / receive section (SciTxRx) that the transmit FIFOs read pointer can be incremented. This signal remains asserted for multiple PCLK cycles. To detect a rising edge on this signal, a delayed version (DelRdPtrInc) of this signal is generated by clocking it through a D-type flip-flop clocked by PCLK. A rising edge on TxFRdPtrIncSync is said to have been detected when TxFRdPtrIncSync is HIGH and DelRdPtrInc is LOW. In normal operation, the RdPtrIncValid signal is asserted when there is data available in the TX FIFO and a rising edge is detected on TxFRdPtrIncSync, whilst in test mode, it can be asserted when TESTFIFO is enabled and TestTXFInc is pulsed during a read access to the SCITDR test register.

The SCITXTIDE[3:0] signal indicates the watermark level for the transmit FIFO. The SCITXCOUNTCLR input indicates that the Transmit FIFO is to be cleared.

The WrPtr[2:0] output signal and the RdPtr[2:0] output signal indicate the current value of the write pointer and the read pointer respectively. The TXFF signal indicates whether the transmit FIFO is full or not. The TXFE signal when asserted, indicates that the transmit FIFO is empty i.e. the FIFO does not contain any valid data. The TxDataAvlbl signal indicates whether data is available in the transmit FIFO for transmission. The TXRIS signal is the Transmit FIFO service request signal that is asserted when the Transmit FIFO contains fewer entries than the number programmed as the watermark. The SCITXCOUNT[3:0] output from the block indicates the fill level of the transmit FIFO. The LASTTXBYTE output is set when the FIFO has one last valid byte left. This signal is used in the Transmit/Receive section to transition to the reception mode immediately after the last character has been transmitted. The TXFRPIncDone output is used to indicate to the Transmit section that the Read pointer increment operation is complete.

The WrPtrIncValid signal is asserted when a valid write access occurs to the FIFO. The following considerations apply when generating this signal:

- When there is a write access to the Transmit FIFO, the write data is clocked into the FIFO only if the FIFO is not already full. If the FIFO is full when the write access occurs, a simultaneous read should also occur for the write data to be clocked in. In normal operation, the Interrupt service routine that writes data into the FIFO is not expected to write more data than is required to fill the FIFO. This check is done to protect from such writes in case they occur.
- Writes to the Transmit FIFO are allowed to proceed only if the SCI is programmed to be in the Transmit mode i.e. only when the MODE bit is set.

The WrPtr[2:0] signal is used as the write pointer signal. The Write pointer is incremented every time the WrPtrIncValid signal is asserted.

The RdPtrIncValid signal is asserted when there is a rising edge detected on the TxFRdPtrIncSync signal and the FIFO is not already empty.

The RdPtr[2:0] signal is the Read pointer for the FIFO. The Read pointer is incremented every time the RdPtrIncValid signal is asserted.

There exists a possibility that the write pointer increments through “111” to “000”. The direct difference between the pointers will not give the correct fill level. To take this into account, the ‘Wrap’ signal has been introduced.

The Wrap signal is set when the Write pointer has wrapped around but the read pointer hasn’t. This signal is reset when the read pointer also subsequently wraps around. The Wrap signal is prepended to the write pointer and from the resultant value, the read pointer is subtracted to determine the fill level of the FIFO. The SCITXCOUNT[3:0] signal indicates the number of locations in the FIFO that contain valid data. This signal takes a range of values starting with zero, when the FIFO is empty, one when there has been one write to the transmit FIFO and up to eight when the FIFO has all its eight locations filled.

The following considerations apply when generating the Wrap signal:

- When the write pointer is at “111” and the WrPtrIncValid signal is asserted and the RdPtrIncValid signal is not asserted then set the Wrap.
- When the read pointer is at “111” and the RdPtrIncValid signal is asserted, and the WrPtrIncValid signal is not asserted, clear Wrap. To combine this condition with the previous condition, toggle Wrap when either of the two conditions occurs.

When both the pointers are at “111” with the FIFO being full (Wrap already set) and both the WrPtrIncValid signal and the RdPtrIncValid signal are asserted, then the write is allowed to complete and the FIFO is still full, but retain the value of the Wrap bit i.e. let it remain set, thus indicating that the FIFO is still full.

When the SCITXCOUNTCLR signal is asserted, the pointers and the Wrap signal are cleared so that the FIFO becomes empty. The Transmit Interrupt (TXRIS) and the status signals (TXFF, TxDataAvlbl and TXFE) are also set to their reset values, either directly or indirectly. Only the FIFO control and status signals are cleared and the actual contents of the FIFO Register File are not cleared.

The TXRIS interrupt signal is generated on the basis of the FIFO fill level. The SCITXTIDE[3:0] input indicates the Watermark level programmed. When the fill level in the FIFO becomes less than or equal to this programmed water mark level, the Transmit Interrupt, TXRIS, is asserted. The interrupt is de-asserted when sufficient number of bytes are written into the FIFO and the fill level becomes greater than the programmed water mark level. The assertion and de-assertion of TXRIS occurs one PCLK after the respective conditions are satisfied.

When there is a write to the SCITXCOUNTCLR register, the control signals in the FIFO namely WrPtr[2:0], RdPtr[2:0], Wrap, TxDataAvlbl and TXFF are set to their reset values. The FIFO then becomes empty.

## 5.11 Transmit FIFO Register File (SciTXRegFile)

The Register File is implemented as a bank of D-types. This block can be replaced with a compiled memory if required. The Register File also contains a 3-to-8 decoder to decode the Write pointer and a 8-to-1 bus multiplexer for the read data bus.

## 5.12 Receive FIFO (SciRxFIFO)

The Receive FIFO instantiates the Receive FIFO control logic (SciRxFCntl) and the Register File (SciRxRegFile). Writes to the Receive FIFO occur from the Receive section, which operates on SCICLK. Reads from the Receive FIFO occur through the APB interface operating on PCLK.

The Receive FIFO is similar to the transmit FIFO in most respects. The difference is that each element in the Receive FIFO is 9 bits wide. It is implemented as a circular buffer with read and write pointers.

When a full byte of data has been received, the Receive section drives the data plus parity bit on RxFWrData[8:0]. It then asserts the RxFWrEn signal, which gets synchronised to PCLK and is seen as the RxFWrEnSync signal. Write data is clocked into the FIFO on the rising edge of PCLK on which the RxFWrEnSync signal is high. For reads, the RXFRdData[8:0] output from the FIFO is driven with the contents of the location pointed to by the current value of the read pointer.

The SCIRXTIDE[3:0] input indicates the water mark level for the Receive FIFO. When the FIFO fill level becomes greater than or equal to this watermark level, the RXRIS interrupt is asserted. The SCIRXCOUNT[3:0] output indicates the fill level of the Receive FIFO. When the SCIRXCOUNTCLR input is asserted, the FIFO control logic is cleared and the FIFO becomes empty.

The Receive timeout counter in the SCICLK domain is reloaded whenever there is a read from the Receive FIFO as indicated by the RxFRd output signal from the Receive FIFO. Once the reload operation is complete, the RTLdDoneSync input to the Receive FIFO is asserted.

In test mode, the RX FIFO can be written through the APB interface by performing writes accesses to the SCITDR register.

## 5.13 Receive FIFO Controller (SciRXFCntl)

The Receive FIFO controller controls the read pointer and the write pointer. The receive interrupt, RXRIS is also generated by this block.

For writes, the RXFWrEnSync signal is clocked through a flip-flop clocked by PCLK to generate a delayed version DelRXFWrEnSync, which is used to detect a rising edge on the RXFWrEnSync signal. Writes to the FIFO and increment of the write pointer occur on the basis of the rising edge on the RXFWrEnSync signal.

In the case of reads from the APB interface, the RXFRdPtrInc signal is a one-PCLK-wide pulse used to increment the read pointer. The FIFOFillLevel signal is derived using the same mechanism as used for the transmit FIFO.

The Receive interrupt, RXRIS, is asserted when the receive FIFO is filled to a level equal to or greater than the level specified by the SCIRXTIDE[3:0] input. The interrupt is cleared when the FIFO contents are read out so that the fill level falls below the SCIRXTIDE[3:0] watermark.

When the RXFWrEnSync signal is asserted and the DelRXFWrSync is not asserted, the rising edge on the RXFWrSync signal is said to have been detected. If the FIFO is not already full, the data on the RxFWrData[8:0] input is clocked into the register file location currently being pointed to by the write pointer. At the end of the write, the write pointer is incremented. The Wrap signal is set when the write pointer has wrapped around but the read pointer has not. The Wrap signal is used to calculate the FIFO fill level.

When the FIFO fill level is greater than or equal to the programmed water mark level, the Receive interrupt (RXRIS) is asserted after one PCLK duration.

The RxFRdPtrInc signal is a decode of PADDR[11:2], PSEL, PWRITE and IntPENABLE in the APB interface block. When the RXFRdPtrInc input is asserted, the Read pointer is incremented if the FIFO is not already empty. Read data from the FIFO is latched into the PRDATA register in the APB interface on the rising edge of PCLK on which SCIDRRd is high. The FIFO status signals such as RXFE get updated on the same clock edge on which the pointers are updated, while the RXRIS interrupt gets updated one PCLK duration after the assertion/de-assertion condition is satisfied.

The Receive FIFO can be cleared by writing to the SCIRXCOUNTCLR register. When there is a write to the SCIRXCOUNTCLR register, the control signals in the FIFO namely WrPtr[2:0], RdPtr[2:0], Wrap, RXFE and RXFF are set to their reset values. The FIFO then becomes empty.

The RxFRd output signal indicates to the Transmit/Receive section that there has just been a read from the receive FIFO and that the Receive timeout counter should be reloaded. The RxFRd signal is toggled whenever the RTLdEn signal, which is generated as an AND of RxRd and RTLd, is asserted. The RxRd signal stores the information that there has been a valid read from the FIFO. The RTLd signal stores the information that the Receive timeout counter has just reloaded. When a read from the Receive FIFO occurs, the toggling of the RxFRd signal is delayed till the previous reload command to the Receive timeout counter is complete, as indicated by the setting of the RTLd signal.

When TESTFIFO is asserted and the RX FIFO is not full, then the SCITDWR signal is used to increment the RX FIFO write pointer on each successive write access through the APB interface. This allows the RX FIFO to be accessed for test purposes.

## 5.14 Receive FIFO Register File (SciRxRegFile)

The Receive FIFO Register file is similar to the transmit register file except for the fact that the receive register elements are 9 bits wide. Write data is clocked into the location pointed to by the current value of the write pointer (WrPtr[2:0]) on the rising edge of PCLK on which the write enable (RegFileWrEn) is sampled high. The RXFRdData[8:0] bus is driven with the contents of the FIFO location pointed to by the current value of the read pointer (RdPtr[3:0]).

## 5.15 Synchronisers to the SCICLK domain (SciSynctoSCICLK)

This block contains all synchronisers in the design that synchronise signals entering the SCICLK domain. Classical double synchronisation has been used to synchronise signals crossing clock domains.

Primary inputs such as SCIDETECT, SCIDEACREQ and SCIDATAIN are synchronised and fed to the SciTxRx block.

The write update signals for the second stage buffers are from the Register block (SciRegBlk). On synchronisation, these signals are fed to the SciRegBlkUpdate block.

The FIFO-related input signals to this block are from the Transmit FIFO and the Receive FIFO control logic. On synchronisation, these signals are fed to the SciTxRx block.

The Card activation/de-activation control signals are from the Register block (SciRegBlk). On synchronisation, these signals are fed to the SciTxRx block and the SciCntl block.

The Card control input signals to this block are from the Register block (SciRegBlk). These signals correspond to the bits in the SCICR0, SCICR1 AND SCISYNCTX registers. On synchronisation these signals are fed to the SciTxRx block and the SciCntl block.

The interrupt clear trigger inputs to this block are from the Register block (SciRegBlk). On synchronisation, these signals are fed to the SciTxRx block.

The interrupt mask set clear signals are from the Register block (SCiRegBlk). On synchronisation, these signals are fed to the interrupt generation block (SciIntGen) and combined with raw interrupt status bits and clear signals to provide the final interrupts.

## 5.16 Synchronisers to the PCLK domain (SciSynctoPCLK)

This block contains all synchronisers for signals entering the PCLK domain. Classical double synchronisation has been used to synchronise signals crossing clock domains.

The non-AMBA output signals such as SCICLKOUT, nSCIDATAEN etc. that are inputs to this block, are from the SciTxRx block. On synchronisation, these signals are fed to the APB interface block (SciApbif). These signals are readable through registers in the APB interface.

The FIFO-related input signals to this synchroniser block are from the SciTxRx block. On synchronisation, these signals are fed to the Transmit FIFO and the Receive FIFO respectively.

The raw interrupt status input signals to this block are from the SciTxRx block. On synchronisation, these signals are fed to the Register block (SciRegBlk) and the APB Interface block (SciApbif).

The masked interrupt input signals to this block are from the SciIntGen block. On synchronisation, these signals are fed to the APB interface block (SciApbif).

The DMA clear signals are intra-chip inputs and on synchronisation, these signals are fed to the DMA block (SciDMA).

## 5.17 DMA Interface (SciDMA)

This block provides an interface to connect to a DMA controller. The DMA operation of the SCI is controlled through the SCI DMA Control Register (SCIDMACR).

Requests for data to be written to the transmit FIFO or read from the receive FIFO in single or burst accesses are controlled by the four registered outputs, TXDMASREQ, TXDMABREQ, RXDMASREQ and RXDMABREQ. These outputs are asynchronously cleared to zero on application of PRESETn and subsequent next values are clocked through by PCLK.

Setting the TXDMAE and RXDMAE bits within the SCIDMACR register enables the respective transmit and receive DMA functions. When not enabled, the respective single and burst request signals are held low.

If there is one or more spaces available in the transmit FIFO, signified by the TXFLTE7Full signal being set, then TXDMASREQ will be asserted on the next PCLK rising edge.

If the transmit FIFO level is less than or equal to that of the programmed transmit FIFO watermark level, signified by TXRIS being set, then the TXDMABREQ will be asserted on the next PCLK rising edge.

If there is data available within the receive FIFO, signified by the RXGTE1Full signal being set, then RXDMASREQ will be asserted on the next PCLK rising edge.

If the receive FIFO level is greater than or equal to that of the programmed receive FIFO watermark level, signified by RXRIS being set, then the RXDMABREQ will be asserted on the next PCLK rising edge.

When the respective DMA transfer is about to complete, the DMA controller will raise the relevant SCITXDMACLR or SCIRXDMACLR signal, which clears the burst request signals, thereby terminating the transfer. After termination, if more accesses are required, then the relevant request signals will be clocked out on the next PCLK rising edge after removal of the respective clear signal.

## 5.18 Test logic (SciTest)

The Test block performs the following functions:

- Test mode control via SCITCR register
- Integration testing using SCIITIP, SCIITOP1 and SCIITOP2 read/set registers.
- Test read and write access of the Tx and Rx FIFOs respectively.

The SCITCR test control register contains two programmable bits:

- ITEN, the integration test enable. When set, the SCI is placed into integration test mode. This mode allows point to point connections to be checked between other modules when the SCI is instantiated within a system. Three registers, SCIITIP (Integration Test Input), SCIITOP1 (Integration Test Output 1) and SCIITOP1 (Integration Test Output 2) are used to implement this test mode.
- TESTFIFO, the test FIFO enable bit. When set, data can be read from the Tx FIFO or written to the Rx FIFO via the SCITDR test data register.

### 5.18.1 Integration testing of block inputs

The following sections describe the integration testing for the block inputs:

- *Intra-chip inputs.*
- *Primary inputs.*

**Intra-chip inputs**

***Figure 5.18-1*** explains the implementation details of the input integration test harness. The ITEN bit is used as the control bit for the multiplexor, which is used in the read path of the SCITXDMACLR and SCIRXDMACLR intra-chip inputs. If the ITEN control bit is deasserted, the SCITXDMACLR and SCIRXDMACLR intra-chip inputs are routed as the internal SCITXDMACLR and SCIRXDMACLR inputs respectively, otherwise the stored register values are driven on the internal line.

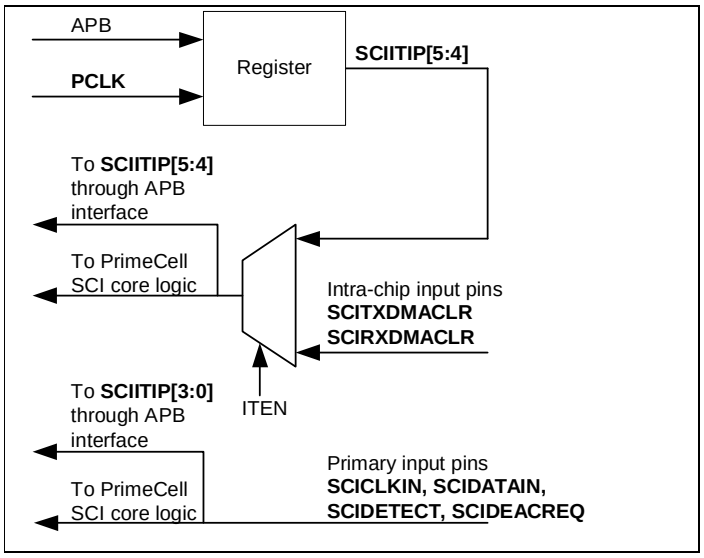


***Figure 5.18-1 Input integration test harness***

When you run integration tests with the PrimeCell SCI in a standalone test setup:

- Write a 1 to the ITEN bit in the control register. This selects the test path from the SCIITIP[5:4] register bits to the SCIRXDMACLR and SCITXDMACLR signals.
- Write a 1 and then a 0 to each of the SCIITIP[5:4] register bits, and read the same register bits to ensure that the value written is read out.

When you run integration tests with the PrimeCell SCI as part of an integrated system:

- Write a 0 to the ITEN bit in the control register. This selects the normal path from the external SCIRXDMACLR pin to the internal SCIRXDMACLR signal, and the path from the external SCITXDMACLR pin to the internal SCITXDMACLR pin.
- Write a 1 and then a 0 to the internal test registers of the DMA controller to toggle the SCIRXDMACLR signal connection between the DMA controller and the PrimeCell SCI. Read from the SCIITIP[4] register bit to verify that the value written into the DMA controller, is read out through the PrimeCell SCI. Similarly, write a 1 and then a 0 to the internal registers of the DMA controller to toggle the SCITXDMACLR signal connection

between the DMA controller and the PrimeCell SCI. Read from the SCIITIP[5] register bit to verify that the value written into the DMA controller, is read out through the PrimeCell SCI.

**Primary inputs**

The following primary inputs are tested using the integration vector trickbox:

- SCICLKIN
- SCIDATAIN
- SCIDEACREQ
- SCIDETECT

### 5.18.2 Integration testing of block outputs

The following sections describe the integration testing for the block outputs:

- *Intra-chip outputs.*
- *Primary outputs.*

**Intra-chip outputs**

Use this test for the following outputs:

- SCIINTR
- SCICLKACTINTR
- SCICLKSTPINTR
- SCIRORINTR
- SCITXDMASREQ
- SCITXDMABREQ
- SCIRXDMASREQ
- SCIRXDMABREQ
- SCIRORINTR
- SCITXTIDEINTR
- SCIRXTIDEINTR
- SCIRTOUTINTR
- SCICHTOUTINTR
- SCIBLKTOUTINTR
- SCIATRDTOUTINTR
- SCIATRSTOUTINTR
- SCITXERRINTR
- SCICARDDNINTR
- SCICARDUPINTR
- SCICARDOUTINTR
- SCICARDININTR

When you run integration tests with the PrimeCell SCI in a standalone test setup:

- Write a 1 to the ITEN bit in the control register. This selects the test path from the SCIITOP1[15:0] and SCIITOP2[12:9] register bits to the intra-chip output signals.
- Write a 1 and then a 0 to the SCIITOP1[15:0] and SCIITOP2[12:9] register bits, and read the same register bits to verify that the value written is read out.

When you run integration tests with the PrimeCell SCI as part of an integrated system:

- Write a 1 to the ITEN bit in the control register. This selects the test path from the SCIITOP1[15:0] and SCIITOP2[12:9] register bits to the intra-chip output signals.
- Write a 1 and then a 0 to the SCIITOP1[15:0] and SCIITOP2[12:9] register bits to toggle the signal connections between the DMA controller/interrupt controller and the PrimeCell SCI. Read from the internal test registers of the DMA controller/interrupt controller to verify that the value written into the SCIITOP1[15:0] and SCIITOP2[12:9] register bits is read out through the PrimeCell SCI.

***Figure 5.18-2*** explains the implementation details of the output integration test harness for intra-chip outputs.

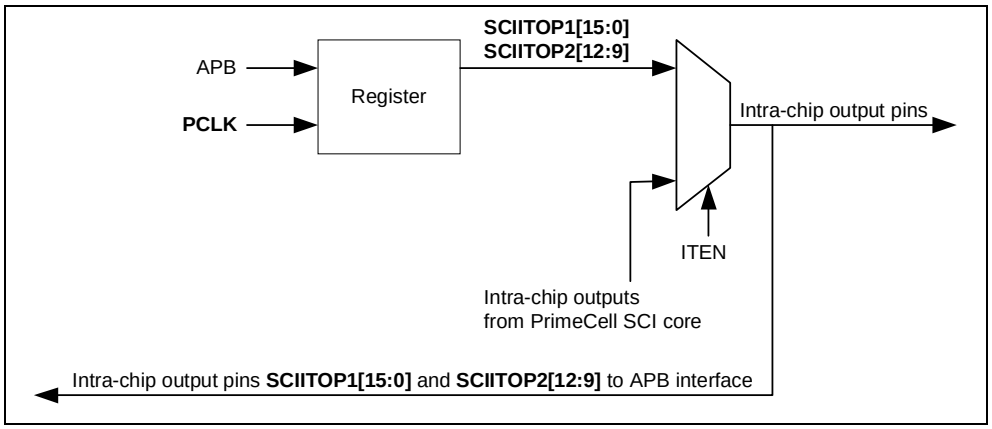


***Figure 5.18-2 Output integration test harness, intra-chip outputs***

**Primary outputs**

Integration testing of primary outputs and primary inputs is carried out using the integration vector trickbox. Use this test for the following outputs:

- nSCICLKEN
- nSCICLKOUTEN
- SCICLKOUT
- nSCIDATAEN
- nSCIDATAOUTEN
- SCIDEACACK
- SCIVCCEN
- nSCICARDRST
- SCIFCB

**Note** Only the SCICLKOUT, nSCIDATAOUTEN, SCIDEACACK, SCIVCCEN, SCIFCB and nSCICARDRST signals are available at the output pads of a typical configuration. The nSCICLKEN, nSCICLKOUTEN and nSCIDATAEN signals are internal connections to the pad.

Verify the primary input and output pin connections as follows:

The primary outputs, SCIDEACACK, nSCIDATAOUTEN and SCICLKOUT are directly connected to the SCIDEACREQ, SCIDATAIN and SCICLKIN respectively by the integration trickbox shown in *Figure 5.18-3*. To test the nSCICLKEN and nSCIDATAEN connections, you can apply a weak pull down/pull to the tristate pins through the trickbox. It is not possible to test the nSCICLKOUTEN connection in this configuration.

- The SCIVCCEN, nSCICARDRST and SCIFCB signals are exclusive ORed within the integration trickbox. The output is connected to the SCIDETECT input.
- All the primary outputs can be accessed through the SCIITOP2[8:0] register. Different data patterns are written to the output pins using the SCIITOP2[8:0] register bits.
- The looped-back data is read back through the SCIITIP[3:0] register bits.

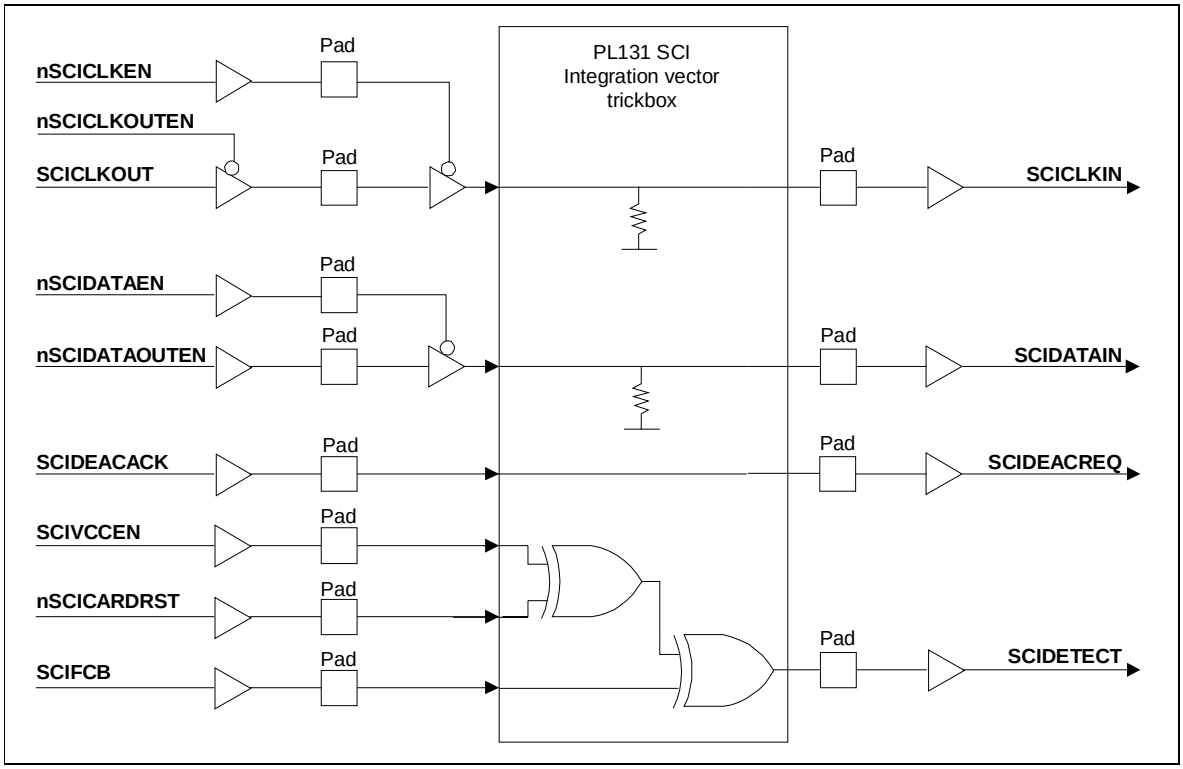


***Figure 5.18-3 Primary outputs routed to primary inputs***

***Figure 5.18-4*** shows implementation details of the output integration test harness in the case of primary outputs.

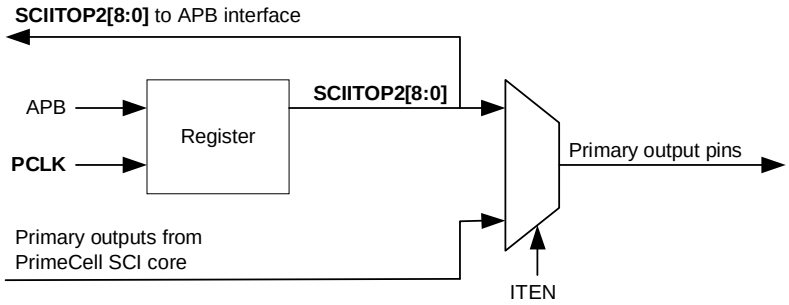


***Figure 5.18-4 Output integration test harness, primary outputs***

# 6 DESIGN ASSUMPTIONS

## 6.1 Software Assumptions

### 6.1.1 Response to the reception of the ATR initial (TS) character

The design assumes that on reception of the initial character within the Answer-To-Reset stream that the associated software routine can read the character, interpret it and reconfigure the SCI registers, if need be, prior to the start of the next character. For example, if the SCI is programmed to receive data in direct data format, but the incoming initial character is in inverse data format, then it is assumed that the software can realise this. It will ignore any parity errors and configure the SCI interface into inverse data format prior to the start of the next character.

## 6.2 Hardware Assumptions

### 6.2.1 Selection of Card Operating voltages is through external hardware

The design assumes that the hardware that is used to provide the correct operating voltage controlled by external logic and that the SCI interface only provides the enable signal.

# 7 SCI FUNCTIONAL VERIFICATION DETAILS

The SCI Functional Verification testbench is an APB BusTalk testbench. The VHDL and Verilog BusTalk testbench files reside in the ***verification/vhdl*** and ***verification/verilog*** directories respectively. The test vectors that are applied to the BusTalk testbenches are in the ***verification/bustest*** directory.

## 7.1 SCI VHDL Verification Testbench

The ARM PrimeCell SCI VHDL test world uses a variant of a top-level testbench that is based on a bus compliance testbench from the AMBA Compliance Test Suite. For further information refer to the ARM Micropack AMBA Compliance Test Suite User Guide.

The testbench has been designed such that it can be used within most common VHDL simulators, and it is suited to AMBA system designers who want to ensure the portability of their blocks to any other AMBA designs. This simplifies the ASIC integration task and exchange of peripherals between projects.

The testbench is “programmable”, meaning that a description of the transfers to be performed on the **Unit Under Test** (UUT) is given without a need to modify the test bench implementation, hence the “generic” property. This transfer description takes the form of a text file that can be read by the specific test bench to which it is being targeted. ***Figure 7.1-1*** illustrates the BusTalk VHDL testbench for running test vectors.

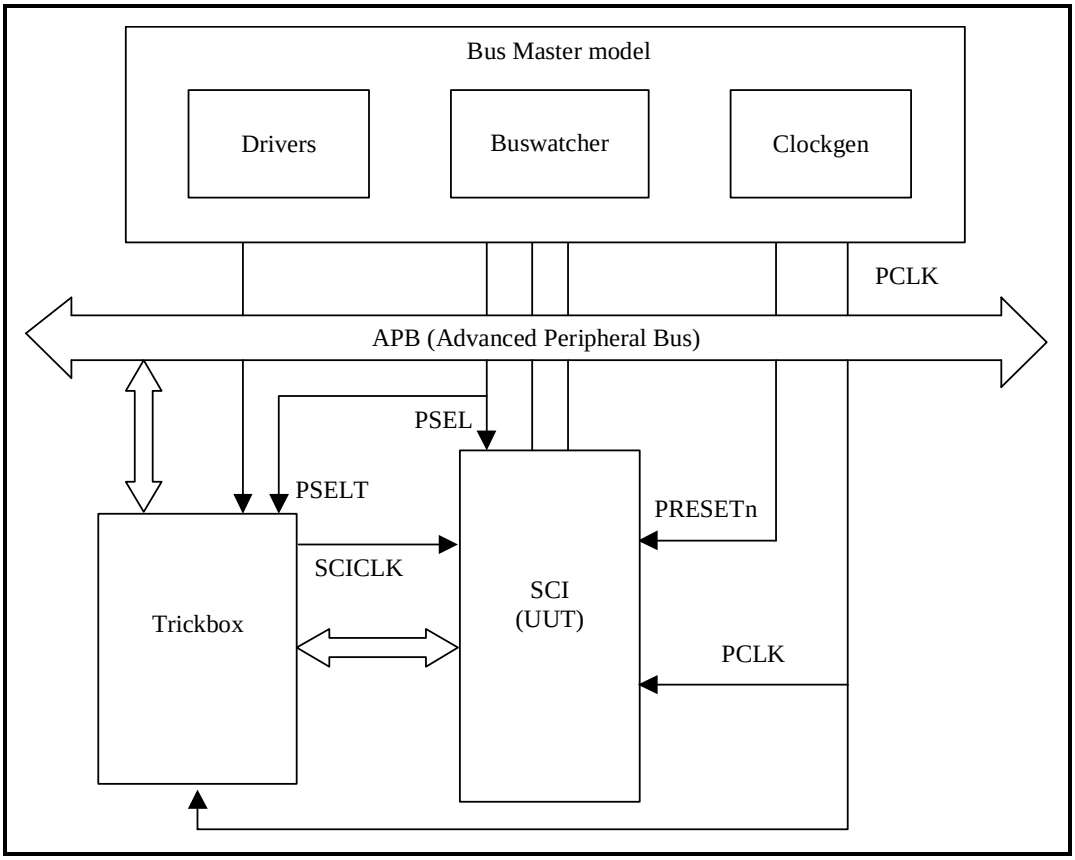


***Figure 7.1-1 VHDL testbench for simulating test vectors***

***Figure 7.1-2*** illustrates the hierarchy for the VHDL testbench.

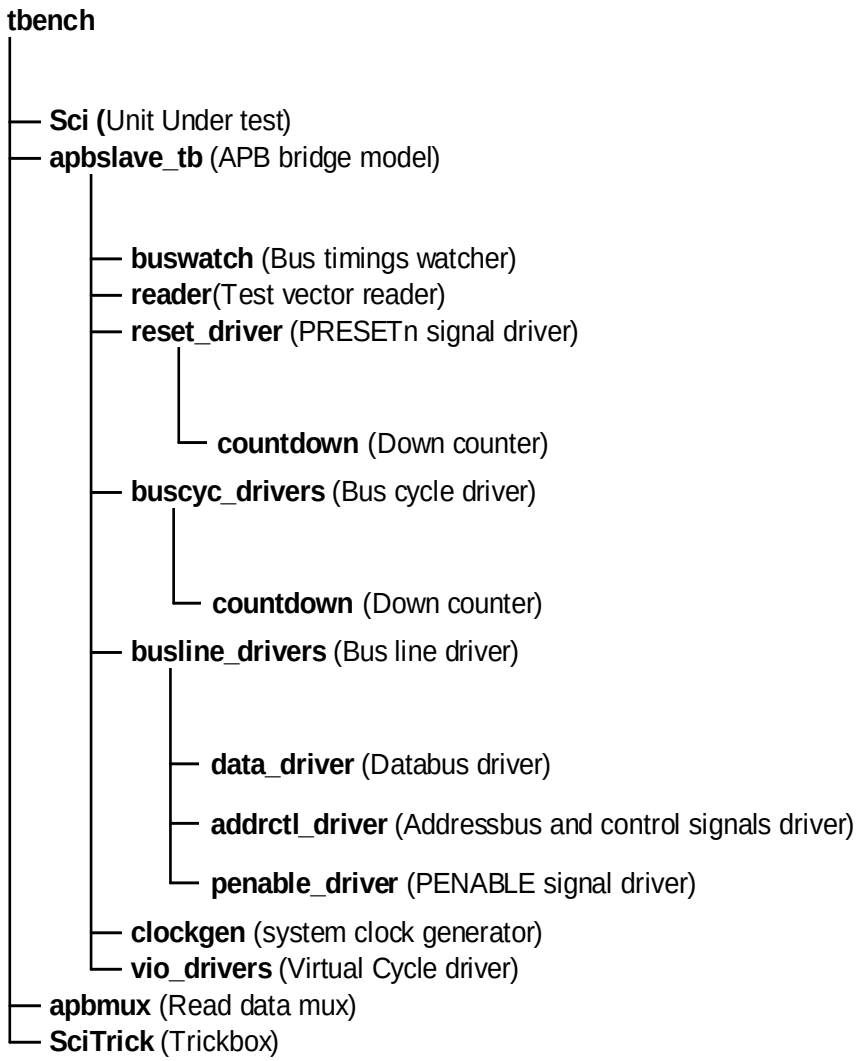


***Figure 7.1-2 VHDL Testbench hierarchy***

The testbench, **tbench.vhd** is used for functional simulations of the SCI. This module instantiates the SCI (Unit Under Test), the behavioural model of the APB Bridge, the trickbox and the APB Read mux. The SCICLK is generated by the trickbox to facilitate frequency modifications from within the test code.

The SCICLK and its domain inputs to the SCI are driven by the trickbox. All the Scan input pins are tied low to keep them from interfering with the test process.

The default test frequency is set at 100MHz. This is the same frequency to which the SCI was constrained during synthesis. PCLK is set to the frequency defined in tbench.vhd using the tclkh and tclkl generic values when instantiating apbslave_tb.vhd.

### 7.1.1 Unit Under Test

The Unit Under Test may be one of the following

- SCI VHDL RTL
- SCI VHDL netlist from VHDL RTL Source

One out of these two versions may be referenced via the corresponding link to uut directory in ***verification/vhdl*** .

### 7.1.2 APB Bridge Model

The UUT is stimulated primarily by the APB bridge behavioural model. This module issues commands on the APB bus under program control.

The APB bridge model, apbslave_tb.vhd, instantiates the buswatcher, the reader, the reset driver, the bus cycle drivers, the bus line drivers and the clock generator. The reader module reads the test vectors from the infile.bif file ( from the ***verification/bustest/invec*** directory ) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the PRESETn signal with the correct timings. The buscyc_drivers instantiates an APB cycle driver for each cycle type, i.e. PSW, PSR, PI, PO, etc. The busline_drivers drives the databus, the address and control signals with the correct timings. The buswatch module checks the Read/Write Databus timings. The clockgen generates the system clock, PCLK.

The APB bridge model also provides the Select line for the trickbox.

Any access to the address range

***0x60000000 to 0x600000FF***

will access the TrickBox via the PSELT select line.

Accesses to any address outside the above range will select the UUT via the PSEL select line.

While the default behaviour of the Bridge in the testbench is like a Rev E device, there is an option to force the bridge to issue commands with Rev D timings.

### 7.1.3 Trickbox

The functionality of the SCI is verified via a trickbox. The trickbox is an APB Slave that mimics the behaviour of an SCI Slave device. Like the SCI, the trickbox also contains receive and transmit FIFOs. The trickbox has a serial to parallel converter to line up data received serially from the SCI and it has a parallel to serial converter to transmit the data stored in its FIFO. The trickbox has a checker module that flags off any violations in the SCI transmit/receive protocol. The trickbox communicates with the APB through the APB-Interface module. All the registers in the trickbox are maintained in a separate register module.

### 7.1.4 Protocol Checker

The testbench includes a protocol checker that reports any protocol or timing violations during accesses to any APB slave. Timing parameters can be set using the timings.vhd file in the ***verification/vhdl/tbench*** directory. The buswatch module which is in the ***verification/vhdl/buswatcher*** directory checks the Read/Write Databus timings and reports any violations in the timings.

### 7.1.5 APB Multiplexer

The read data buses from the SCI and the trickbox are muxed together in this module before being connected to the APB Bridge model.

### 7.1.6 Test Vectors

The test vectors for the verification of the SCI are coded into separate C files depending on the logical region of the SCI that these vectors are testing. Make options have been provided to use all of the test vectors together or to use them individually.

### 7.1.7 Generics

The SCI testbench provides some configurability options using generics and constants. These generics and constants are configurable through the tbench.vhd file.

**XonPSEL**

**XonPSEL** is used in the tbench.vhd file that resides in ***verification/vhdl/tbench***.

This generic is used to eliminate the 'X's that appear on the PSEL, PWRITE and PADDR lines when these lines are not being actively driven. If **XonPSEL** is set to **0** in the top level of the testbench, these signals will go to '0' when the corresponding line driver is inactive. If set to **1**, the signals go to 'X' when the line driver is idle.

**Verbosity**

**Verbosity** is used in the data_driver.vhd, the reset_driver.vhd and the buscyc_drivers.vhd which reside in ***verification/vhdl/bid***.

This generic is used to control the verbosity of the transcript generated during simulation. If **Verbosity** is set to **1**, every command issued will have a corresponding message in the transcript file. Every poll operation in a POLL command sequence is also reported. If **Verbosity** is set to **0**, a message is issued only if an error occurs on a read. Similarly during a POLL command, a message is flagged only if a POLL time-out occurs.

**HaltOnMismatch**

This is used in the data_driver.vhd that resides in ***verification/vhdl/bid***.

This generic is used to stop the simulation if a read is unsuccessful. If set to **1**, the simulation halts after an error is reported. If set to **0**, the error is flagged, but the simulation continues.

**SuppressOnReset**

**SuppressOnReset** is used in the buswatch.vhd that resides in ***verification/vhdl/buswatcher.***

This generic is used to suppress the checking of the PRDATA timings when the PRESETn signal is asserted. If **SuppressOnReset** is TRUE, the PRDATA set-up and hold timing violations are not flagged when PRESETn is asserted. If **SuppressOnReset** is set to FALSE, PRDATA set-up and hold timing violations are flagged irrespective of PRESETn.

**Mesg_enb**

**Mesg_enb** is used in the buswatch.vhd which resides in ***verification/vhdl/buswatcher.***

This generic is used to enable or disable the flagging of error messages when PRDATA does not go to zero when the UUT is not selected. The flagging of these error messages is disabled when this generic is set to FALSE. If **Mesg_enb** is set to TRUE, the error messages are flagged whenever PRDATA goes to a non-zero value when the UUT is not selected. This generic will have to be set to FALSE in testbenches which include more than one slave.

**Data_check**

**Data_check** is used in the apbmux.vhd which resides in ***verification/vhdl/tbench***.

This constant is used to enable or disable the flagging of error messages when the PRDATA driven by a slave has a non-zero value when it is not selected. When **Data_check** has a value **1**, these error messages are flagged. When it has a value **0**, the error messages are not flagged. This constant is used in testbenches which include more than one slave.

### 7.1.8 Running Simulations

The ARM PrimeCell Generic Peripheral User Guide [Ref 8] gives step by step instructions for running simulations using the SCI test vectors on the VHDL source code and the VHDL netlist.

Run time logs for the RTL simulations are:

- verification/vhdl/Sci_rtl/Sci_rtl_modelsim.log

Run time logs for the netlist simulations are:

- verification/vhdl/Sci_scaninsert_vhdl_net_max/Sci_scaninsert_vhdl_net_max.log

## 7.2 SCI Verilog Verification Testbench

The ARM PrimeCell SCI Verilog test world uses a variant of a top-level testbench which is based on a bus compliance testbench from the AMBA Compliance Test Suite. For further information refer to the ARM Micropack AMBA Compliance Test Suite User Guide.

The test bench is designed to be used with most common Verilog simulators, and it is suited to AMBA system designers who want to ensure the portability of their blocks to any other AMBA designs. This simplifies the ASIC integration task and exchange of peripherals between projects.

The test bench is “programmable”, meaning that a description of the transfers to be performed on the **Unit Under Test** (UUT) is given without a need to modify the test bench implementation, hence the “generic” property. This transfer description takes the form of a text file that can be read by the specific test bench to which it is being targeted. ***Figure 7.2-1*** illustrates the BusTalk Verilog testbench for simulating test vectors.

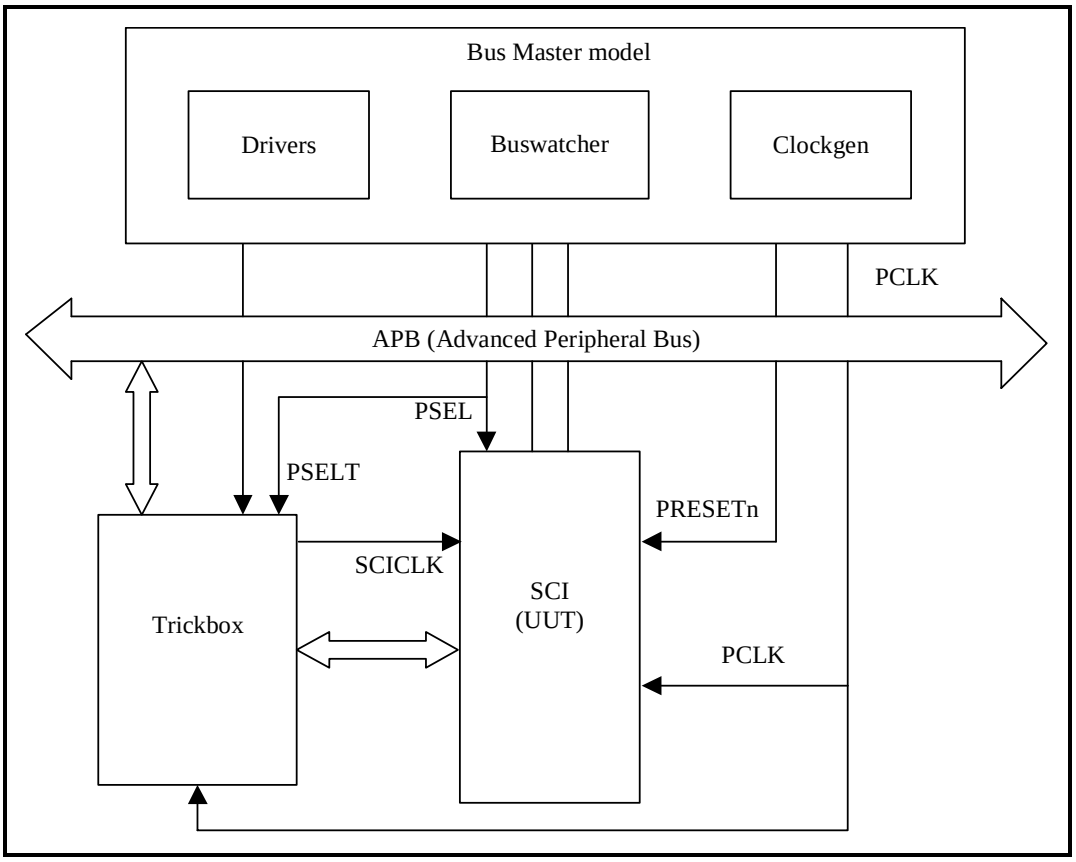


***Figure 7.2-1 Verilog Testbench for simulating test vectors***

***Figure 7.2-2*** illustrates the hierarchy for the Verilog testbench.

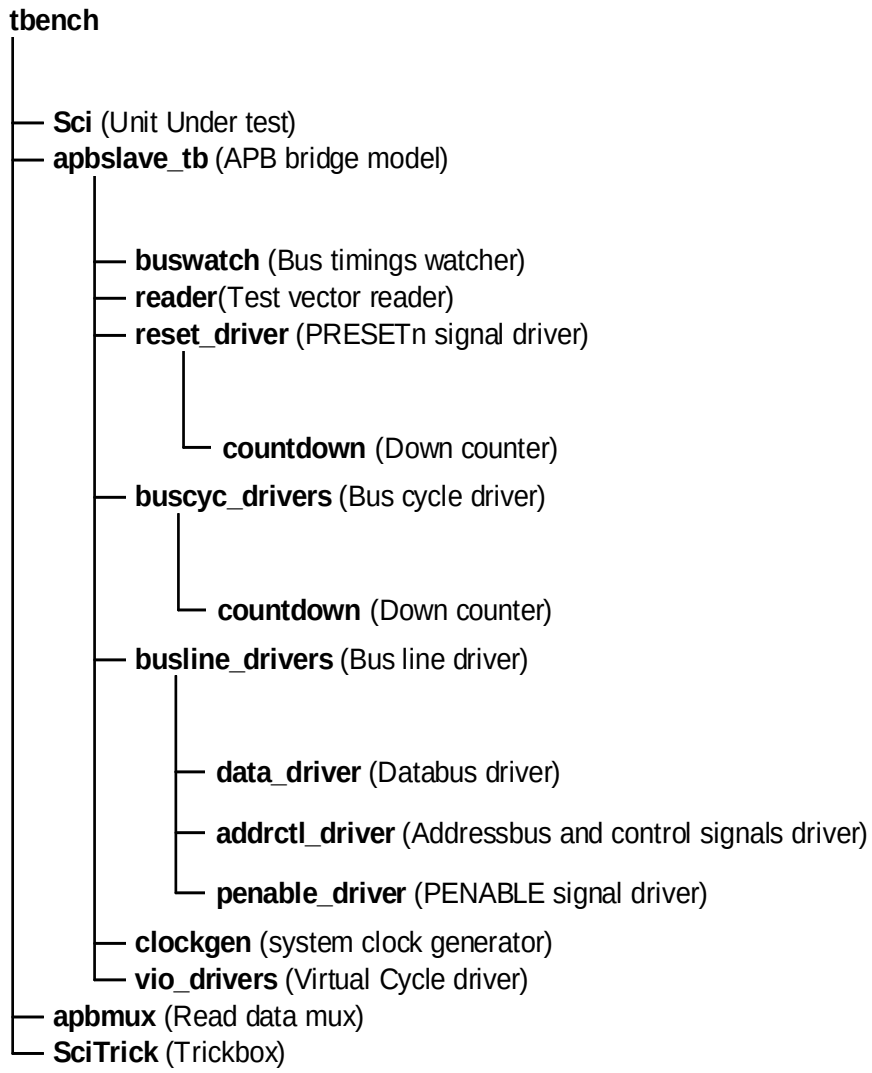


***Figure 7.2-2 Verilog Testbench hierarchy***

The testbench, **tbench.v** is used for functional simulations of the SCI. This module instantiates the SCI (Unit Under Test), the behavioural model of the APB Bridge, the trickbox and the APB Read mux. The SCICLK is generated by the trickbox to facilitate frequency modifications from within the test code.

All the SCICLK and SCICLK domain inputs to the SCI are driven by the trickbox. All the scan input pins are tied low to keep them from interfering with the test process.

The default test frequency is set at 100 MHz. This is the same frequency to which the SCI was constrained during synthesis. SCICLK is set to the PCLK frequency defined in timing.v using the Tclkh and Tclkl ‘define values when instantiating apbslave_tb.v in tbench.v. Note that the ‘define values override the local parameters defined in clockgen.v in the ***verification/verilog/common*** directory. All the non-AMBA inputs to the SCI, except the scan data inputs, are driven by the Trickbox.

### 7.2.1 Unit Under Test

The Unit Under Test may be one of the following

- SCI Verilog RTL
- SCI Verilog netlist from Verilog Source
- SCI Verilog netlist from VHDL Source

One out of these three versions may be referenced via the corresponding links to uut directory in ***verification/verilog*** directory***.***

### 7.2.2 APB Bridge Model

The UUT is stimulated primarily by the APB bridge behavioural model. This block issues commands on the APB bus under program control.

The APB bridge model, apbslave_tb.v instantiates the buswatcher, the reader, the reset driver, the bus cycle drivers, the bus line drivers and the clock generator. The reader block reads the test vectors from the bif.sim file (from the ***verification/bustest/invec*** directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the PRESETn signal with the correct timings. The buscyc_drivers instantiates an APB cycle driver for each cycle type, i.e. PSW, PSR, PI, PO, etc. The busline_drivers drives the databus, the address and control signals with the correct timings. The buswatch block checks the Read/Write data bus timings. The clockgen generates the system clock, PCLK.

The APB bridge model also provides the Select line for the Trickbox.

Any access to the address range

***0x60000000 to 0x600000FF***

will access the Trickbox via the PSELT select line.

Accesses to any address outside the above range will select the UUT via the PSEL select line.

**The Poll Command in APB**:

Usage: PO(expected value, mask , addr, [limit], [tag]);

On encountering a PO command in the vector file, the APB bridge model issues a read cycle (PSR) to the specified address (addr). If the read value matches the (expected) value, the next command in the vector file is executed. If the values do not match, reads are repeatedly issued to the same address until either the expected value is sampled or the number of reads issued exceeds the [limit] parameter. If the [limit] is not specified, the Polling continues indefinitely until the expected value is returned. One limitation of the POLL command is that two successive PO commands cannot be issued back-to-back. They should be separated by at least one command like PSR , PSW, PI etc.

### 7.2.3 Trickbox

The functionality of the SCI is verified via a Trickbox model. The trickbox is an APB Slave that mimics the behaviour of an SCI Slave device. Like the SCI, the trickbox also contains receive and transmit FIFOs. The trickbox has a serial to parallel converter to line up data received serially from the SCI and it has a parallel to serial converter to transmit the data stored in its FIFO. The trickbox has a checker module that flags off any violations in the SCI transmit/receive protocol. The trickbox communicates with the APB through the APB-Interface module. All the registers in the trickbox are maintained in a separate register module.

### 7.2.4 Protocol Checker

The testbench includes a protocol checker that reports any protocol or timing violations during accesses to any APB slave. Timing parameters can be set using the timings.v file in the ***verification/verilog/tbench*** directory. The buswatch block which is in the ***verification/verilog/buswatcher*** directory checks the Read/Write data bus timings and reports any violations in the timings.

### 7.2.5 APB Multiplexer

The read data buses from the SCI and the Trickbox are multiplexed together in this block before being connected to the APB Bridge model.

### 7.2.6 Vector Sets

The test vectors for the verification of the SCI are coded into separate C files depending on the logical region of the SCI which these vectors are testing. Make options have been provided to use all the test vectors together or to use them individually.

### 7.2.7 Parameters

The SCI testbench provides some configurability options via parameters and defines. All the parameters, except MESG_ENB and SUPPRESSONRESET are configurable through the tbench.v file. MES_ENB and SUPPRESSONRESET are configurable through the buswatch.v that resides in ***verification/verilog/buswatcher.*** The define, DATA_CHECK is configurable through the apbmux.v that resides in ***verification/verilog/tbench.***

**XonPSEL**

**XonPSEL** is used in the tbench.v file that resides in ***verification/verilog/tbench***.

This define is used to eliminate the ‘X’s that appear on the PSEL, PWRITE and PADDR lines when these lines are not being actively driven. If **XonPSEL** is set to **0** in the top level of the testbench, these signals will go to ‘0’ when the corresponding line driver is inactive. If set to **1**, the signals go to ‘X’ when the line driver is idle.

**Verbosity**

**Verbosity** is used in the data_driver.v, the reset_driver.v and the buscyc_drivers.v which reside in ***verification/verilog/bid***.

This parameter is used to control the verbosity of the transcript generated during simulation. If **Verbosity** is set to **1**, every command issued will have a corresponding message in the transcript file. Every poll operation in a POLL command sequence is also reported. If **Verbosity** is set to **0**, a message is issued only if an error occurs on a read. Similarly during a POLL command, a message is flagged only if a POLL time-out occurs.

**HaltOnMismatch**

This is used in the data_driver.v that resides in ***verification/verilog/bid***.

This parameter is used to stop the simulation if a read is unsuccessful. If set to **1**, the simulation halts after an error is reported. If set to **0**, the error is flagged but the simulation continues.

**SUPPRESSONRESET**

**SUPPRESSONRESET** is used in the buswatch.v which resides in ***verification/verilog/buswatcher.***

This parameter is used to suppress the checking of the PRDATA timings when PRESETn signal is asserted. If **SUPPRESSONRESET** is **1**, the PRDATA set-up and hold timing violations are not flagged when PRESETn is asserted. If **SUPPRESSONRESET** is set to **0**, PRDATA set-up and hold timing violations are flagged irrespective of PRESETn.

**MESG_ENB**

**MESG_ENB** is used in the buswatch.v which resides in ***verification/verilog/buswatcher.***

This parameter is used to enable or disable the flagging of error messages when PRDATA does not go to zero when the UUT is not selected. The flagging of these error messages is disabled when this parameter is set to **0**. If **MESG_ENB** is set to **1**, the error messages are flagged whenever PRDATA goes to a non-zero value when the UUT is not selected. This generic will have to be set to **0** in testbenches that include more than one slave.

**DATA_CHECK**

**DATA_CHECK** is used in the apbmux.v that resides in ***verification/verilog/tbench***.

This define is used to enable or disable the flagging of error messages when the PRDATA driven by a slave has a non-zero value when it is not selected. When **DATA_CHECK** has a value **1**, these error messages are flagged. When it has a value **0**, the error messages are not flagged. This define is used in testbenches which include more than one slave.

### 7.2.8 Running Simulations

The ARM PrimeCell Generic Peripheral User Guide [Ref 8] gives step by step instructions for running simulations using the SCI test vectors on the Verilog source code and the Verilog netlist.

Run time logs for the RTL simulations are:

- verification/verilog/Sci_rtl/Sci_rtl_modelsim.log
- verification/verilog/Sci_rtl/Sci_rtl_LDV.log

Run time logs for the netlist simulations are:

- verification/verilog/Sci_noscan_vhdl_net_min/Sci_noscan_vhdl_net_min_modelsim.log
- verification/verilog/Sci_scanready_verilog_net_typ/Sci_scanready_verilog_net_typ_LDV.log

The code coverage results are:

- verification/verilog/Sci_rtl/vnavigator.sumary

## 7.3 SCI Trickbox

The Trickbox is a programmable APB device designed to provide a test engineer a means to drive the non-APB input pins of the SCI and sample the non APB responses of the same. Both VHDL and Verilog versions of the Trickbox exist. The VHDL version resides in ***BusTalk/vhdl/trickbox*** while the Verilog version resides in ***BusTalk/vlog/trickbox***.

The Trickbox reads the data transmission line of the SCI, performs width checking on the data line and transfers the serial data into a FIFO. The Trickbox is also capable of driving serial data on the data reception lines of the SCI. The transmit and receive paths of the Trickbox are buffered with internal FIFO memories allowing up to 16 bytes to be stored independently in both the transmit and receive modes. The reference clock SCICLK and nSCIRST is also provided to the unit under test(UUT) by the Trickbox.

The following features of the Trickbox are programmable.

- Independent Tx and Rx function enables.
- Programmable number of retries in character handshake mode in both transmit and receive directions.
- Facility to drive the SCICLK to simulate synchronous SmartCards.
- Insertion of jitter in the data stream transmitted to the interface.
- Frequency of the reference clock which can be routed to the SCICLK pin
- Either PCLK or SCICLK can be routed to the SCICLK output

The following sections detail the functionality of the Trickbox.

### 7.3.1 Trickbox Signal Description

The following table summarises the SCI signal list. ***Table 7.3-1*** lists the APB interface signals while ***Table 7.3-2*** lists the module specific non-APB signals.

| Signal Name | Type | Source / Destination | Description |
|---|---|---|---|
| PCLK | IN | clock generator | APB Bus Clock |
| PRESETn | IN | reset generator | Bus reset (active low). |
| PADDR [7:2] | IN | APB Bridge | Subset of APB address bus |
| PWDATA[15:0] | IN | APB Bridge | Unidirectional APB write data bus |
| PRDATA[15:0] | OUT | APB Bridge | Unidirectional APB read data bus |
| PENABLE | IN | APB Bridge | APB enable signal. PENABLE is asserted HIGH for one cycle of PCLK to enable a bus transfer cycle |
| PWRITE | IN | APB Bridge | When HIGH, this signal indicates a write into the peripheral and when LOW, a read from the peripheral. |
| PSELT | IN | APB Bridge | Trickbox select signal from the APB Bridge. When HIGH, this signal indicates that the APB Bridge has selected the Trickbox. |

***Table 7.3-1 APB signal descriptions***

| Signal Name | Type | Source/ Destination | Description |
|---|---|---|---|
| SCICLK | OUT | To SCI | SCI Reference clock |
| nSCIRST | OUT | To SCI | SCI Reset signal for clocked elements in the SCICLK domain. |
| SCICARDININTR | IN | From SCI | SCI Card in interrupt (active HIGH). |
| SCICARDOUTINTR | IN | From SCI | SCI Card out interrupt (active HIGH). |
| SCICARDUPINTR | IN | From SCI | SCI Card powered up interrupt (active HIGH). |
| SCICARDDNINTR | IN | From SCI | SCI Card powered down interrupt (active HIGH). |
| SCITXERRINTR | IN | From SCI | SCI Character transmission error interrupt (active HIGH). |
| SCIATRSTOUTINTR | IN | From SCI | SCI ATR start timeout interrupt (active HIGH). |
| SCIATRDTOUTINTR | IN | From SCI | SCI ATR duration timeout interrupt (active HIGH). |
| SCIBLKTOUTINTR | IN | From SCI | SCI Block timeout between blocks interrupt (active HIGH). |
| SCICHTOUTINTR | IN | From SCI | SCI Character timeout between characters interrupt (active HIGH). |
| SCIRTOUTINTR | IN | From SCI | SCI Receive FIFO Read time out interrupt (active HIGH) |
| SCIRORINTR | IN | From SCI | SCI Receive Over Run interrupt (active HIGH) |
| SCICLKSTPINTR | IN | From SCI | SCI Clock Stopped interrupt (active HIGH) |
| SCICLKACTINTR | IN | From SCI | SCI Clock Active interrupt (active HIGH) |

| | | | |
|---|---|---|---|
| SCIRXTIDEINTR | IN | From SCI | SCI Receive FIFO tide mark reached interrupt (active HIGH). |
| SCITXTIDEINTR | IN | From SCI | SCI Transmit FIFO tide mark reached interrupt (active HIGH). |
| SCIINTR | IN | From SCI | SCI interrupt (active HIGH). A single combined interrupt generated as an OR function of the twelve individually maskable interrupts above. |
| SCANMODE | OUT | To SCI | Not currently utilised. |
| SCICLKIN | IN | From SCI | SCI clock input. |
| SCIDATAOUT | OUT | To SCI | Serial data output |
| SCIDATAIN | IN | From SCI | Serial data input. |
| SCICLKOUT | OUT | To SCI | SCI Clock output |
| SCIDETECT | OUT | To SCI | Card detect signal. |
| SCIDEACREQ | OUT | To SCI | Card deactivation request. |
| SCIDEACACK | IN | From SCI | Card deactivation acknowledgement. |
| SCITXDMASREQ | IN | From SCI | Transmit DMA Single Request |
| SCITXDMABREQ | IN | From SCI | Transmit DMA Burst Request |
| SCIRXDMASREQ | IN | From SCI | Receive DMA Single Request |
| SCIRXDMABREQ | IN | From SCI | Receive DMA Burst Request |
| SCITXDMACLR | OUT | To SCI | Transmit DMA Clear |
| SCIRXDMACLR | OUT | To SCI | Receive DMA Clear |

***Table 7.3-2 Block specific signal description***

## 7.3.2 Trickbox functional description

The architectural behaviour of the Trickbox is similar to that of the SCI itself. The trick box contains the following major modules

**APB Interface and the Register Block**

The APB is a local secondary bus which provides a low-power extension to the higher AHB (or ASB) within the AMBA system hierarchy. The APB interface generates read and write decodes for accesses to status/control registers and transmit/receive FIFO memories. The register block stores data written or to be read across the APB interface.

***Trickbox register summary***

The SCI registers are shown in ***Table 7.3-3***. The base address is implementation dependent. The registers located at offset 0x80 to 0xFC are reserved for test purposes and should not be used during normal operation.

| Address (offset from Trickbox Base) | Type | Width | Reset Value | Name | Description |
|---|---|---|---|---|---|
| 0x00 | R/W | 9 | 0x00 | SCITrDATA | Data Register |
| 0x04 | R/W | 16 | 0x0000 | SCITrCR | Control Register |
| 0x08 | R/W | 8 | 0x00 | SCITrFiLCR | FIFO fill level control register. |
| 0x0C | R | 6 | 0x00 | SCITrSR0 | Flag register |
| 0x10 | R | 13 | 0x0000 | SCITrSR1 | SCI interrupt status register |
| 0x14 | R | 1 | 0x0 | SCITrSR2 | Deactivation Acknowledge |
| 0x18 | R/W | 4 | 0x0 | SCITrTXPC | Transmit retry register |
| 0x1C | R/W | 4 | 0x0 | SCITrRXPC | Receive retry register |
| 0x20 | R/W | 8 | 0x0 | SCITrCTRL | Error message control register |
| 0x24 | --- | --- | --- | Reserved | -- |
| 0x28 | --- | --- | --- | Reserved | -- |
| 0x2C | R/W | 16 | 0x0000 | SCITrCKICC | SCI clock register |
| 0x30 | R/W | 16 | 0x0000 | SCITrBAUD | Baud register |
| 0x34 | R/W | 8 | 0x00 | SCITrVALUE | Value register |
| 0x38 | R/W | 8 | 0x00 | SCITrTXCHG | Tx.Character guard register |
| 0x3C | R/W | 8 | 0x00 | SCITrTXBG | Tx Block Guard register |
| 0x40 | R/W | 8 | 0x00 | SCITrRXCHG | Rx Character guard register |
| 0x44 | R/W | 8 | 0x00 | SCITrRXBG | Rx Block guard register |
| 0x48 | R/W | 16 | 0x0014 | SCITrRFCK | SCICLK period register |
| 0x4C | R/W | 8 | 0x00 | SCITrWV | Width verification margin register |
| 0x50 | R/W | 16 | 0x0000 | SCITrJit | Jitter value register |
| 0x54 | R/W | 9 | 0x000 | SCITrJitPt | Jitter control register |
| 0x58 | R/W | 3 | 0x0 | SCITrRFCNTL | SCICLK control register |
| 0x5C | R/W | 6 | 0x00 | SCITrDMA | DMA control register |

***Table 7.3-3 SCI Trickbox Register map***

**Transmit and Receive FIFOs**

The transmit FIFO is a First-In, First-Out circular buffer, 8-bit wide, 16 word deep. Any data written to the Trickbox data register is stored in this FIFO till it is read out by the transmit logic. The receive FIFO is a First-In, First-Out circular buffer, 9-bit wide, 16 word deep. The received parity bit is the MSBit of the 9-bit word.

**Transmitter Logic**

This section converts the data in the TXFIFO into a serial bit stream with the duration of each bit determined by the following formula.

Duration = 1 etu = (SCIBAUD + 1) x (SCIVALUE)

Where,

etu = Elementary Time Unit

SCIBAUD = Baud rate clock.

SCIVALUE = Number of Baud Cycles per etu.

If character handshaking is enabled, retransmission is performed according to the programmed value.

**Receiver Logic**

The incoming bit stream is stored into a shift register before being transferred into the RXFIFO. Parity checking is performed on the received data. If character handshaking is enabled, retransmission is requested according to the programmed value.

**Synchroniser logic**

There are two clock domains in the Trickbox – SCIREFCK domain and PCLK domain. Some signals need to cross over from one domain to another. Thus two set of synchronisers – SynctoREFCLK and SynctoPCLK are provided. These synchronisers are implemented using the double synchronisation technique.

**Clock and Reset generation**

The Trickbox generates the reference clock which will act as the SCICLK of the UUT. It also generates the nSCIRST from PRESETn and applies it to the UUT.

The nSCIRST is a reset signal which is controlled by the “device reset” bit of the SCITrCR register. Whenever this bit is made one, it drives the nSCIRST line low (assertion of reset). Writing a zero into this bit de-asserts the reset.

The SCICLK is also produced by the Trickbox. It uses the register – SCITrRFCK for determining the frequency of the reference clock. The Trickbox can also be programmed to pass the PCLK directly as SCICLK. The hardware for facilitating glitch-free switching over from SCICLK to PCLK and vice versa is provided in this module.

### 7.3.3 Trickbox Register Descriptions

The base address of the Trickbox is not fixed and may be different for any particular system implementation. However, the offset of any particular register from the base address is fixed.

**SCITrDATA: SCITrickbox Data Register [9] (+0x00)**

The SCITrDATA register is used for both transmitting and receiving characters. When the SCITrDATA register is read, the entry in the receive FIFO pointed to by the current FIFO pointer is accessed. When the SCITrDATA is written to, the entry in the transmit FIFO pointed to by the write pointer, is written to.

| Bit | Name | R/W | Reset value | Function |
|---|---|---|---|---|
| 7- 0 | DATA | 0x00 | R/W | Eight data bits which correspond to the character being read or written. |
| 8 | PARITY | 0x0 | R | Parity flag transmitted by the SCI |

**SCITrCR: SCITrickbox Control Register [16] (+0x04)**

| Bit | Name | R/W | Reset value | Function |
|---|---|---|---|---|
| 0 | TrBoxEn | R/W | 0x0 | 0 = Trickbox disable<br>1 = Trickbox enable |

| | | | | |
|---|---|---|---|---|
| 1 | TrTXEn | R/W | 0x0 | 0 = Trickbox Transmitter disable<br>1 = Trickbox Transmitter enable |
| 2 | TrRXEn | R/W | 0x0 | 0 = Trickbox Receiver disable<br>1 = Trickbox Receiver enable |
| 3 | TrDebugEn | R/W | 0x0 | 0 = Debug message enable<br>1 = Debug message disable |
| 4 | TrCDETEn | R/W | 0x0 | 0 = Card detect signal disable<br>1 = Card detect signal enable |
| 5 | TrTXPEn | R/W | 0x0 | 0 = Transmit wrong parity disable<br>1 = Transmit wrong parity enable |
| 6 | TrRXPEn | R/W | 0x0 | 0 = Receive wrong parity simulation disable<br>1 = Receive wrong parity simulation enable |
| 7 | TrTXPStEn | R/W | 0x0 | 0 = Transmit even parity<br>1 = Transmit odd parity |
| 8 | TrRXPStEn | R/W | 0x0 | 0 = Program the receiver for even parity<br>1 = Program the receiver for odd parity |
| 9 | TrTXNAKEn | R/W | 0x0 | 0 = Transmit hand shaking disable – T1 mode<br>1 = Transmit hand shaking enable – T0 mode |
| 10 | TrRXNAKEn | R/W | 0x0 | 0 = Receive hand shaking disable – T1 mode<br>1 = Receive hand shaking enable – T0 mode |
| 11 | SCIDEACREQ | R/W | 0x0 | 0 = Deactivation request to the SCI disable<br>1 = Deactivation request to the SCI enable |
| 12 | SCANMODE | R/W | 0x0 | 0 = Scan mode disable<br>1 = Scan mode enable |
| 13 | nSCIRST | R/W | 0x0 | 0 = Reset enable. (Active low)<br>1 = Reset disable |
| 14 | TrSENSE | R/W | 0x0 | 0 = I/O line direct convention<br>1 = I/O line inverse convention |
| 15 | TrSCICLKEn | R/W | 0x0 | 0 = SCICLK generated inside the Trickbox disable<br>1 = SCICLK generated inside the Trickbox enable |

**SCITrFiLCR: SCITrickbox FIFO Flag Control Register [8] (+0x08)**

| Bit | Name | R/W | Reset value | Function |
|---|---|---|---|---|
| 3 - 0 | TXFiLCR | R/W | 0x0 | Trickbox TX FIFO water mark level |
| 7 - 4 | RXFiLCR | R/W | 0x0 | Trickbox RX FIFO water mark level |

**SCITrSR0: SCITrickbox status Register 0 [6] (+0x0C)**

| Bit | Name | R/W | Reset value | Function |
|---|---|---|---|---|
| 0 | SCITrTFR | R | 0x0 | Set when the Trickbox TX FIFO contains equal or less number of data as TXFiLCR |
| 1 | SCITrTFE | R | 0x0 | Set when the Trickbox transmit FIFO becomes empty |
| 2 | SCITrTFF | R | 0x0 | Set when the Trickbox transmit FIFO becomes full |
| 3 | SCITrRFR | R | 0x0 | Set when the receive RX FIFO contains equal or more number of data as RXFiLCR |
| 4 | SCITrRFE | R | 0x0 | Set when the Trickbox receive FIFO becomes empty |
| 5 | SCITrRFF | R | 0x0 | Set when the Trickbox receive FIFO becomes full |

**SCITrSR1 : Trickbox status register 1 [13] (+0x10)**

| Bits | Name | R/W | Reset value | Function |
|---|---|---|---|---|
| 0 | SCIINTR | R | 0x00 | SCI combined interrupt status |
| 1 | SCIRTOUTINTR | R | 0x00 | SCIRTOUTINTR status |
| 2 | SCIRXTIDEINTR | R | 0x00 | SCIRXTIDEINTR status |
| 3 | SCITXTIDEINTR | R | 0x00 | SCITXTIDEINTR status |
| 4 | SCICHOUTINTR | R | 0x00 | SCICHOUTINTR status |
| 5 | SCIBLKOUTINTR | R | 0x00 | SCIBLKOUTINTR status |
| 6 | SCIATRDOUTINTR | R | 0x00 | SCIARTDOUTINTR status |
| 7 | SCIATRSTOUTINTR | R | 0x00 | SCIATRSTOUTINTR status |
| 8 | SCITXERRINTR | R | 0x00 | SCITXERRINTR status |
| 9 | SCICARDDNINTR | R | 0x00 | SCICARDDINTR status |
| 10 | SCICARDUPINTR | R | 0x00 | SCICARDUPINTR status |
| 11 | SCICARDOUTINTR | R | 0x00 | SCICARDOUTINTR status |
| 12 | SCICARDININTR | R | 0x00 | SCICARDININTR status |

**SCITrSR2 : SCI Trickbox status register 2 [1] (+0x14)**

| Bit | Name | R/W | Reset value | Function |
|---|---|---|---|---|
| 0 | SCIDEACACK | R | 0x0 | SCI deactivation acknowledgement status |

**SCITrTXPC : SCI Trickbox transmit count register [4] (+0x18)**

| Bits | Name | R/W | Reset value | Function |
|---|---|---|---|---|
| 3 – 0 | TXPC | R/W | 0x00 | Specifies number of retransmissions required |

**SCITrRXPC: SCI Trickbox receive count register [4] (+0x1C)**

| Bits | Name | R/W | Reset value | Function |
|---|---|---|---|---|
| 3 - 0 | RXPC | R/W | 0x00 | Specifies number of retransmission request |

**SCITrCTRL : Error message control register [8] (+0x20)**

| Bit | Name | R/W | Reset value | Function |
|---|---|---|---|---|
| 0 | SCICLKErEn | R/W | 0x0 | 0 = Error message disable<br>1 = Display error message if the SCICLK width changes from the programmed value |
| 1 | TXPtimErEn | R/W | 0x0 | 0 = Error message disable<br>1 = Display error message if the retransmit request from the SCI comes at the wrong time |
| 2 | Reserved | | | Reserved bit |
| 3 | TXPtimWdErEn | R/W | 0x0 | 0 = Error message disable<br>1 = Display error message if the retransmit request timing is different from the expected 2 etu |
| 4 | RXPErEn | R/W | 0x0 | 0 = Error message disabled<br>1 = Display error message if parity error occurred in the data received by the Trickbox. |
| 5 | RXCtimErEn | R/W | 0x0 | 0 = Error message disabled<br>1 = Display error message if there is any timing violation between the start bit of two data received continuously by the Trickbox. |
| 6 | Reserved | | | Reserved |
| 7 | Reserved | | | Reserved |

**SCITrCKICC : Smart card clock frequency [16] (+0x2C)**

| Bits | Name | R/W | Reset value | Function |
|---|---|---|---|---|
| 15 - 0 | SCITrCKICC | R/W | 0x00 | Defines the Smart card clock frequency |

**SCITrBAUD : Trickbox baud rate clock [16] (+0x30)**

| Bit | Name | R/W | Reset value | Function |
|---|---|---|---|---|
| 15 - 0 | SCITrBAUD | R/W | 0x0 | Defines the Baud clock |

**SCITrVALUE : Trickbox baud cycles [9] (+0x34)**

| Bits | Name | R/W | Reset value | Function |
|---|---|---|---|---|
| 7 - 0 | SCITrVALUE | R/W | 0x00 | Defines number of baud cycles per etu |

**SCITrTXCHG : Trickbox transmitter character to character Extra Guard time [8] (+0x38)**

| Bits | Name | R/W | Reset value | Function |
|---|---|---|---|---|
| 7 - 0 | SCITrTXCHG | R/W | 0x0 | Defines the duration between the leading edges of the start bit of two consecutive characters transmitted by the Trickbox |

**SCITrTXBG : Trickbox transmitter Block Guard time [8] (+0x3C)**

| Bits | Name | R/W | Reset value | Function |
|---|---|---|---|---|
| 7 - 0 | SCITrTXBG | R/W | 0x00 | Define time between the Trickbox transmitter enabled and start of the transmission |

**SCITrRXCHG : Trickbox receiver character to character Extra Guard time [8] (+0x40)**

| Bits | Name | R/W | Reset value | Function |
|---|---|---|---|---|
| 7 - 0 | SCITrRXCHG | R/W | 0x0 | Define the expected time between the leading edges of the start bit of two consecutive characters received by the Trickbox |

**SCITrRXBG : Trickbox receiver Block Guard time [8] (+0x44)**

| Bits | Name | R/W | Reset value | Function |
|---|---|---|---|---|
| 7 - 0 | SCITrRXBG | R/W | 0x00 | Define time between the Trickbox receiver enabled and the expected reception of the start bit. |

**SCITrRFCK : Reference clock frequency [16] (+0x48)**

| Bits | Name | R/W | Reset value | Function |
|---|---|---|---|---|
| 15 – 0 | SCITrRFCK | R/W | 0x00 | Defines the reference clock frequency |

**SCITrWV : Error Margin value [8] (+0x4C)**

| Bits | Name | R/W | Reset value | Function |
|---|---|---|---|---|
| 7 - 0 | SCITrWV | R/W | 0x00 | Defines allowed margin in the timing calculation |

**SCITrJit : Jitter value register [8] (+0x50)**

| Bits | Name | R/W | Reset value | Function |
|---|---|---|---|---|
| 7 – 0 | SCITrJit | R/W | 0x00 | Defines the amount of jitter inserted in one bit time |

**SCITrJitPt : Jitter pattern register [10] (+0x54)**

| Bits | Name | R/W | Reset value | Function |
|---|---|---|---|---|
| 8 - 0 | JitterEn | R/W | 0x0 | Jitter enable for the particular bit which is one |
| 9 | JitterDir | R/W | 0x0 | 1 = jitter in positive direction<br>0 = Jitter in negative direction |

**SCITrCTRL : Clock control register [3] (+0x58)**

| Bit | Name | R/W | Reset value | Function |
|---|---|---|---|---|
| 0 | Select pin of REFCLK mux | R/W | 0x0 | 0 = Internally generated REFCLK connected to SCICLK pin of SCI<br>1 = PCLK connected to SCICLK pin of SCI |
| 1 | RFCLKEn | R/W | 0x0 | REFCLK mask |
| 2 | PCLKEn | R/W | 0x0 | PCLK mask |

**SCITrDMA : DMA control register [6] (+0x5C)**

The SCITrDMA register provides the DMA clear signals and a mechanism for checking the status of the four DMA request lines. Writing a “1” to bit 1 or bit 0 will generate a 2 PCLK wide pulse on the SCITXDMACLR and SCIRXDMACLR lines respectively. For single transfers this should be asserted one clock cycle after the request has gone high, and for bursts it should be asserted during the transmission of the last word in the burst. The DMA clear signals should be de-asserted 1 clock cycle after the request line has been de-asserted.

| Bit | Name | R/W | Reset value | Function |
|---|---|---|---|---|
| 0 | SCIRXDMACLR | W | 0x0 | Clears the transmit DMA requests when set. |
| 1 | SCITXDMACLR | W | 0x0 | Clears the receive DMA requests when set. |
| 2 | SCIRXDMABREQ | R | 0x0 | Receive DMA burst request status |

| | | | | |
|---|---|---|---|---|
| 3 | SCIRXDMASREQ | R | 0x0 | Receive DMA single request status |
| 4 | SCITXDMABREQ | R | 0x0 | Transmit DMA burst request status |
| 5 | SCITXDMASREQ | R | 0x0 | Transmit DMA single request status |

## 7.4 Functional Verification Tests

The following sections describe all the free running test cases that validate the function of the SCI module. These test cases use the SCI Trickbox for their execution.

### 7.4.1 Register Tests

#### Reset value Test

The SCIDATA, SCICR0, SCICR1, SCICR2, SCICLKICC, SCIVALUE, SCIBAUD, SCITIDE, SCIDMACR, SCISTABLE, SCIATIME, SCIDTIME, SCIATRSTIME, SCIATRDTIME, SCISTOPTIME, SCISTARTTIME, SCITXCOUNT, SCIRXCOUNT, SCIRETRY, SCICHTIMELS, SCICHTIMEMS, SCIBLKTIMELS, SCIBLKTIMEMS, SCICHGUARD, SCIBLKGUARD, SCIRXTIME, SCIFIFOSTATUS, SCITXCOUNT, SCIRXCOUNT, SCIIMSC, SCIRIS, SCIMIS, SCIICR, SCISYNCACT, SCISYNCTX, SCISYNCRX, SCIPeriphID0, SCIPeriphID1, SCIPeriphID2, SCIPeriphID3, SCIPCellID0, SCIPCellID1, SCIPCellID2, SCICellID3 and the SCITCR registers are read immediately after reset to verify that they initialise to the values as per the specification.

#### Pattern Read / Write Test

Read /Writeable registers SCICR0, SCICR1, SCICLKICC, SCIVALUE, SCIBAUD, SCITIDE, SCIDMACR, SCISTABLE, SCIATIME, SCIDTIME, SCIATRSTIME, SCIATRDTIME, SCISTOPTIME, SCISTARTTIME, SCIRETRY, SCIIER, SCIRXTIME, SCICHTIMELS, SCICHTIMEMS, SCIBLKTIMELS, SCIBLKTIMEMS, SCICHGUARD, SCIBLKGUARD, SCIRXTIME, SCIIMSC, SCISYNCACT and SCISYNCTX are written with patterns of 0xFF followed by patterns of 0x00. Data is read back and compared with the expected pattern.

#### Read only Register Test

The SCIFIFOSTATUS, SCITXCOUNT, SCIRXCOUNT, SCIRIS, SCIMIS, SCIPeriphID0, SCIPeriphID1, SCIPeriphID2, SCIPeriphID3, SCIPCellID0, SCIPCellID1, SCIPCellID2, SCICellID3 are written with a pattern that is different from the reset value. Data is read back from the same register. An error is flagged if the read data is different from the reset value.

### 7.4.2 Debounce Test

This test checks the debounce mechanism, at the time of card detection by the SCI. The test keeps the SCIDETECT signal high for a time which is sufficient to generate SCICARDININTR and checks whether the interrupt is generated. After the SCICARDININTR generation, it makes the SCIDETECT signal low and makes the SCI to start deactivation. After the deactivation time, it reads the status of SCICARDOUTINTR and SCICARDDNINTR and expects a high value on these pins. After deactivation the SCICARDININTR signal is sampled and checked for a low value The same test is repeated with zero and non-zero debounce time.

### 7.4.3 Activation-Deactivation sequence Test

This test verifies the activation and deactivation sequence of the SCI. After the detection of the SCICARDININTR, enable the activation sequence by writing one to the start bit of the SCI control register 2. It verifies the activation sequence through the SCISYNCACT register. It checks the SCICARDUPINTR after the activation sequence. The deactivation sequence is started by writing a one to the finish bit of the SCICR2. It verifies the deactivation sequence through the SCISYNCACT register of SCI.

### 7.4.4 ATR Test

This test checks the ATR sequences, SCIATRSTOUTINTR, SCIATRDTOUTINTR, SCIRTOUTINTR, warm reset and self-deactivation of SCI. The test sequence is as follows. Initialise the SCI in receive mode and poll for the SCICARDININTR. The start bit is delayed more than the ATR start value programmed and the status of SCIATRSTOUTINTR is checked. The ATR sequence time is kept more time than programmed ATR duration time and the condition of SCIATRDTOUTINTR is checked. After the reception of SCIATRDTOUTINTR the warm reset sequence is enabled by writing a one to the warm reset enable bit of the SCI control register. After the completion of warm reset, it is verified that the SCI starts self deactivation, by delaying the ATR start bit by a value that is greater than the programmed value. This test also checks the SCIRTOUTINTR by not reading the data within the receive FIFO until the programmed value of SCIRXTIME is exceeded.

### 7.4.5 Receive non back-to-back Test

This test checks the SCI receiver by transmitting a single data. After the initialisation, enable the SCI in receive mode. Write one data into the transmit FIFO of the Trickbox and send the data to the SCI. Read the receive FIFO and check whether the SCI receives the data properly. The same test is repeated for all four possible combinations of ORDER and SENSE of the data with different jitter values and jitter patterns. Different values of BAUD and SCIVALUE are used.

### 7.4.6 Receive back to back Test

In this test the trickbox transmits more than 50 data words continuously to the SCI receiver and checks the receiver by reading the data from the receive FIFO of the SCI. The same test is repeated for all combinations of ORDER and SENSE of the data as well as with different jitter values and jitter patterns.

### 7.4.7 Receive interrupt flag Test

This test checks RXFF, RXFE flags and SCIRXTIDEINTR. The SCIRXTIDEINTR is checked with different values of RXTIDE.

### 7.4.8 Receive parity Test

This test checks the receiver parity counter and the retry mechanism of the SCI receiver. It is possible to make the Trickbox transmit a wrong parity bit by enabling the parity error enable bit in the Trickbox control register. The number of times the wrong parity transmission is required can also be programmed. The test sequence is as follows. Program the SCI receiver retry counter with a particular value and make the Trickbox transmit the wrong parity bit. In T0 mode of operation SCI request retransmission if it received a wrong parity bit. The test checks the retransmission request assertion time and the width of the assertion. If the Trickbox parity error counter is programmed with a value that is greater than the SCI receive retry counter, the Trickbox transmits more number of wrong parity bits than the retry value programmed and it causes the SCI receiver to set the parity error bit. In case of T1 mode, the test expects the SCI to assert parity error bit immediately after it received the wrong parity bit. Additionally, it checks that the T1 mode does not support retransmission.

### 7.4.9 Character timeout interrupt Test

This test checks the SCICHTOUTINTR. Program the Trickbox transmitter character Guard register to a value, which is greater than the SCICHTIME register value. This makes the time gap between two data transmissions greater than the programmed SCICHTIME and the SCI to assert SCICHTOUTINTR.

### 7.4.10 Block time out interrupt Test

This test checks the SCIBLKTOUTINTR, and the interlock mechanism of the SCI transmitter. The test sequence is as follows. Enable the Trickbox transmitter and write one data into the transmit FIFO of the SCI. After that change the SCI to the receive mode, and verify that the SCI transmitter continues to transmit independent of the Mode value. This verifies the interlock mechanism. The test causes the SCI to assert SCIBLKTOUTINTR by delaying the start bit by a duration that is greater than the programmed value of SCIBLKTIME, and check whether the SCI asserts the BLKTOUTINTR.

### 7.4.11 Synchronous mode transmit Test

This test checks the operation of the SCI transmitter in synchronous mode. After the initialisation of the SCI, enable the synchronous mode by writing a value of 0x03 to the SCISYNCCR register. Transmit SCICLK and data through the SCISYNCTX register and verify it by reading the SCISYNCRX register.

### 7.4.12 Synchronous mode receive Test

This test checks the operation of the SCI receiver in synchronous mode. After the initialisation of the SCI, enable the synchronous mode by writing a value of 0x03 to the SCISYNCCR register. In this mode of operation SCICLK and data are driven by the Trickbox and correct reception is verified by reading the SCISYNCRX register.

### 7.4.13 Non EMV Test

This test checks the Non-EMV operation of the SCI. In Non-EMV mode of operation the activation sequence is controlled through the SCISYNCACT register. The test sequence is as follows. Make the SCIDETECT signal high and poll for the SCICARDININTR. After the detection of SCICARDININTR, start the activation sequence through SCISYNCACT register. It contains three steps

Step 1. Write one to the power and data enable bit of SCISYNCACT register and after synchronisation, read back its output pin value by reading the SCISYNCACT register.

Step 2. Write one to the clock enable bit of SCISYNCACT register and after synchronisation, read back its output pin value by reading the SCISYNCACT register.

Step 3. Write one to the reset enable bit of SCISYNCACT register and after synchronisation, read back its output pin value by reading the SCISYNCACT register.

The function code bit SCIFCB is also tested at this point in a similar manner. After the completion of the activation sequence, enable the synchronous mode by writing 0x03 to the SCISYNCCR register. Transmit SCICLK and data through the SCISYNCTX register and verify it by reading the SCISYNCRX register.

### 7.4.14 Scan mode Test

This test is for validating the SCI scan test hold input. In this, known data patterns are written into two writable registers. Reset is asserted by setting and clearing the TESTRST bit in the SCITCR of the SCI. After this test reset, the two writable register values should be changed to their reset values. Now the same test is conducted with SCANMODE pin driven high. This time the registers should retain the known data pattern.

### 7.4.15 CLKZ1 Test

This test is for checking the SCICLK output configuration. SCICLK output can be configured as buffer output or pulled down (open drain) mode. If the CLKZ1 bit of the SCI control register 2 is high (buffer output), then SCICLKOUT will be zero and clock will be driven by nSCICLKOUTEN. If the CLKZ1 is zero (pulled down mode), clock will be driven by SCICLKOUT and nSCICLKOUTEN will be zero. The test sequence is as follows. Enable the SCI in synchronous mode and drive the clock through the WCLK pin of the SCISYNCTX register. Check the

status of SCICLKOUT and nSCICLKOUTEN by reading the SCISYNCACT register in both cases (CLKZ1 is high and low).

### 7.4.16 Transmit non back to back Test

This test checks the SCI transmitter by sending a single data. After the initialisation, enable the SCI in transmit mode. Write one data into the transmit FIFO of the SCI. Configure the Trickbox in receive mode, so the data send by the SCI transmitter will be received by the Trickbox, and it can be verified by reading the receive FIFO of the Trickbox. The same test is repeated for all four possible combinations of ORDER and SENSE of the data. Different SCIBAUD and SCIVALUE are used.

### 7.4.17 Transmit back to back Test

This test checks continuous transmission of data by the SCI. This test causes the SCI to transmit more than 50 data words continuously to the Trickbox. Read the Trickbox receive FIFO and check whether the data is transmitted by the SCI properly. The same test is repeated for all four combinations of ORDER and SENSE of the data.

### 7.4.18 Transmit Interrupt Flag Test

This test checks TXFF, TXFE flags and SCITXTIDEINTR. The SCITXTIDEINTR is checked with different values of TXTIDE.

### 7.4.19 Transmit parity Test

This test checks the transmit retry mechanism and the transmit retry counter of the SCI. If the RXPStEn is enabled, the Trickbox receiver can simulate a situation of reception of a wrong parity bit and request for the retransmission of the received data. The number of retransmission requests can also be programmed by writing the required value to the TrRXPC register. Program the SCI transmit retry counter and program the Trickbox receive retry counter with a lesser value. This makes the SCI transmitter to retry the same data as many number of times as is programmed in the Trickbox TrRXPC register without generating the SCITXERRINTR. Transmitter retry mechanism is allowed only in T0 mode.

### 7.4.20 Transmit character guard Test

This test validates the transmitter character guard time by causing the transmitter to send two data continuously with the programmed value of GUARD time between the data transmission.

### 7.4.21 TXERRINTR Test

This test checks the generation of TXERRINTR. It programs the Trickbox receiver retry register with a value that is greater that the value in the SCI transmit retry register. This causes the Trickbox to request for retransmission more number of times than the value programmed in the SCI transmit retry register. This causes the SCI to assert TXERRINTR. In case of T1 mode, SCITXERRINTR should not be asserted.

### 7.4.22 DMA Interface Transmit tests

The SCI and Trickbox are configured with the TESTFIFO mode and transmit DMA enabled.

The Tx FIFO tide level is programmed to midway i.e. at a depth of 4 locations, the SCI is put into TX mode and the Tx FIFO interrupt is enabled.

The single and burst transmit requests, SCITXDMASREQ and SCITXDMABREQ, are checked as being asserted, signifying that there is space in the SCI Tx FIFO for single characters and a burst.

Seven words are written to the SCI Tx FIFO, in one burst length of four and three single transfers. The requests are cleared by writing a “1” to bit 0 of the SciTrDMA register during the transfer of the last data in the burst and after each single write. While there is still at least one burst length space in the SCI Tx FIFO, both requests are asserted. When there is less than a burst length of space in the SCI Tx FIFO, just the single request is asserted.

One word is successively read from the SCI Tx FIFO, whilst the burst request is checked to be asserted after three words have been read out.

The Transmit DMA is disabled and the request checked that it has become de-asserted. The transmit DMA is re-enabled and the request is checked that it has become re-asserted.

The remaining words are read out of the SCI Tx FIFO and the single and burst requests are checked for assertion.

The SCI, Trickbox, Transmit DMA and TESTFIFO mode are then disabled.

### 7.4.23 DMA Interface Receive tests

The SCI and Trickbox are configured with TESTFIFO mode and receive DMA enabled.

The Rx FIFO tide level is programmed to midway i.e. at a depth of 4 locations, the SCI is put into Rx mode and the Rx FIFO interrupt is enabled.

The single and burst receive requests, SCIRXDMASREQ and SCIRXDMABREQ, are checked for de-assertion, because the FIFO’s are currently empty.

Seven words are written in test mode to the SCI Rx FIFO. When there are less than four words in the Rx FIFO, only the single request is set, and that when there are more than 4 words in the Rx FIFO both the single and burst receive requests are set.

The DMA controller would service the burst requests. The data is read out of the Rx FIFO with a single burst of 4 words. The request is cleared by writing a “1” to bit 1 of the SciTrDMA register during the transfer of the last data in the burst. The remaining words are read as single characters, clearing the request after each transfer.

Four words are written into the RXFIFO and the receive single and burst request are checked for assertion. The receive DMA is disabled the requests are checked to be de-asserted. The receive DMA is re-enabled and the requests are checked to be re-asserted. The remaining data is read out of the Rx FIFO.

The SCI, Trickbox, Receive DMA and TESTFIFO mode are then disabled.

### 7.4.24 Clock Stop Mode tests

These test that the SCI card clock can be stopped (low or high) and started when no transactions are in progress. It also checks that if a transmission is in progress, then the clock stopping will not take effect until there is no more data available within the transmit FIFO. If the clock is stopped and the Tx FIFO is written with new data, then the data will not be transmitted until the clock has restarted. Clock stop and resumption of card transaction times are programmed through the SCISTOPTIME and SCISTARTTIME registers.

After an initialisation sequence, whilst in Rx mode, the clock is programmed to stopped by writing a 1 to the CLKDIS bit within the SCICR0 register. The is SCICLKSTPINTR checked to have occured after the SCISTOPTIME value has expired. Then the clock is restarted by writing a 0 to the CLKDIS bit within SCICR0. The SCICLKACTINTR is checked to occur after the SCISTARTTIME value has expired.

The SCI is then programmed into Tx mode and 4 words are written to the SCI TX FIFO. During the transmission the clock is programmed to be stopped, but should only stop after the transmission has completed and the CLKSTOPTIME duration has expired. The four words are then read from the trickbox RX FIFO.

Four more words are the written to the SCI TX FIFO, but no transmission should occur as the clock is stopped. The clock is then programmed to start and the four words will start to be tranmitted after the SCISTARTTIME has expired. The four words are read from the trickbox Rx FIFO.The clock is then reprogrammed to stop.

Whilst the clock is stopped, the Deactivation sequence is initiated and the clock remains stopped.

The above procedure is performed with the CLKVAL bit programmed to 0 and 1 to check that the clock can be programmed to stop in a low or high state respectively.

### 7.4.25 Receive Over Run tests

This test checks that if the SCI Receive FIFO is full and more data is attempted to be written to it, then the Receive OverRun interrupt is generated to inform the system that data has not be captured.

After an initialisation sequence, 8 values are written to the Trickbox Tx FIFO and are then transmitted to the SCI. The SCIFIFOSTATUS is checked for RXFF being set and the SCIRORINT is checked to be still de-asserted. One more value is written to the Trickbox and transmitted to SCI.

Sufficient time is allowed for reception to complete and then the SCIRORINTR is check to be asserted. One value is read from the SCI Rx FIFO and the SCIRORINTR is checked to have remained asserted. The remaining 7 values are read from the SCI Rx FIFO, the SCIRORINTR should remain asserted.

The SCIRORINTR is then cleared by writing a "1" to the RORIC bit within the SCIICR register. Its deasserted state is then checked. The Trickbox is disabled.

### 7.4.26 Additional code coverage state machine tests

These tests provide better coverage of the state machine within the SciCntl block. Not all paths are covered, but it is deemed that the code coverage is satisfactory.

# 8 SCI INTEGRATION TEST DETAILS

The SCI integration test vector suite allows the integrator to verify that the SCI has been connected into the target system correctly. These vectors test 3 classes of signals:

a) AMBA signals (SCI signals connected directly to the APB)

b) Intra-Chip signals (SCI signals connected to the DMA and Interrupt Controllers)

c) Primary I/O signals (SCI signals connected to the chip pads)

For more details on Integration testing, please refer to the PrimeCell Integration Vector Strategy document [Ref. 9].

The SCI Integration testbench is based on an AHB TICTalk testbench. The VHDL and Verilog TICTalk testbenches are used to verify that the SCI has been connected into the target system correctly. The VHDL and Verilog TICTalk testbench files reside in the ***integration/vhdl*** and ***integration/verilog*** directories respectively. The test vectors that are applied to the TICTalk testbenches are in the ***integration/bustest*** directory. The test vectors for integration testing are written in BusTalk and then converted into **.tif** format to be applied to the TICTalk integration testbench.

## 8.1 SCI VHDL Integration testbench

The AMBA TICTalk VHDL testbench used for integration testing of the SCI is located in the ***integration/vhdl/tbench*** directory. The Integration test vectors are located in the ***integration/bustest*** directory.

The test vectors used with this testbench can be applied to the SCI when it is part of an AMBA system design.

**Figure 3 Testbench for simulating TICTalk test vectors** illustrates the testbench to apply TICTalk test vectors.

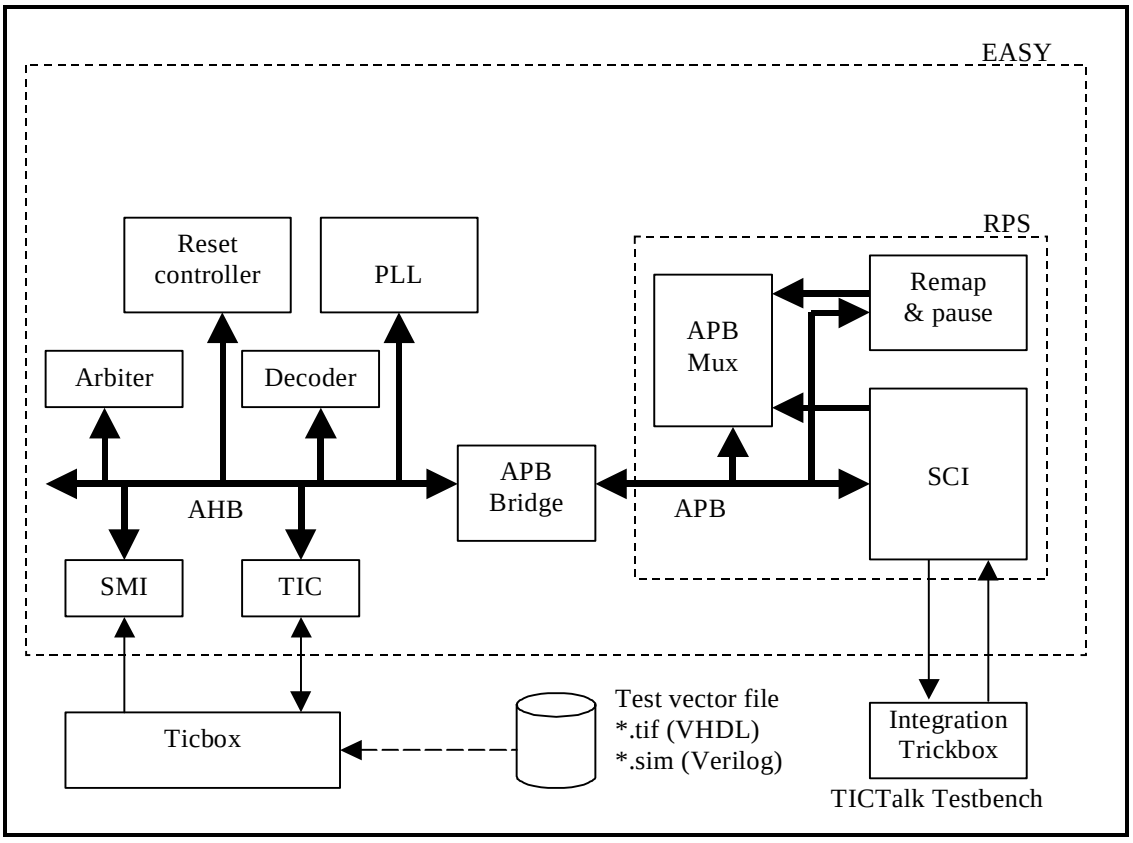


**Figure 3 Testbench for simulating TICTalk test vectors**

The SCI VHDL Integration testbench is a stripped down version of the standard EASY system testbench. For further information refer to the Example AMBA System user guide which is part of ARM Micropack documentation.

**Figure 4 Testbench hierarchy for simulating TICTalk test vectors** illustrates the hierarchy for the TICTalk testbench.

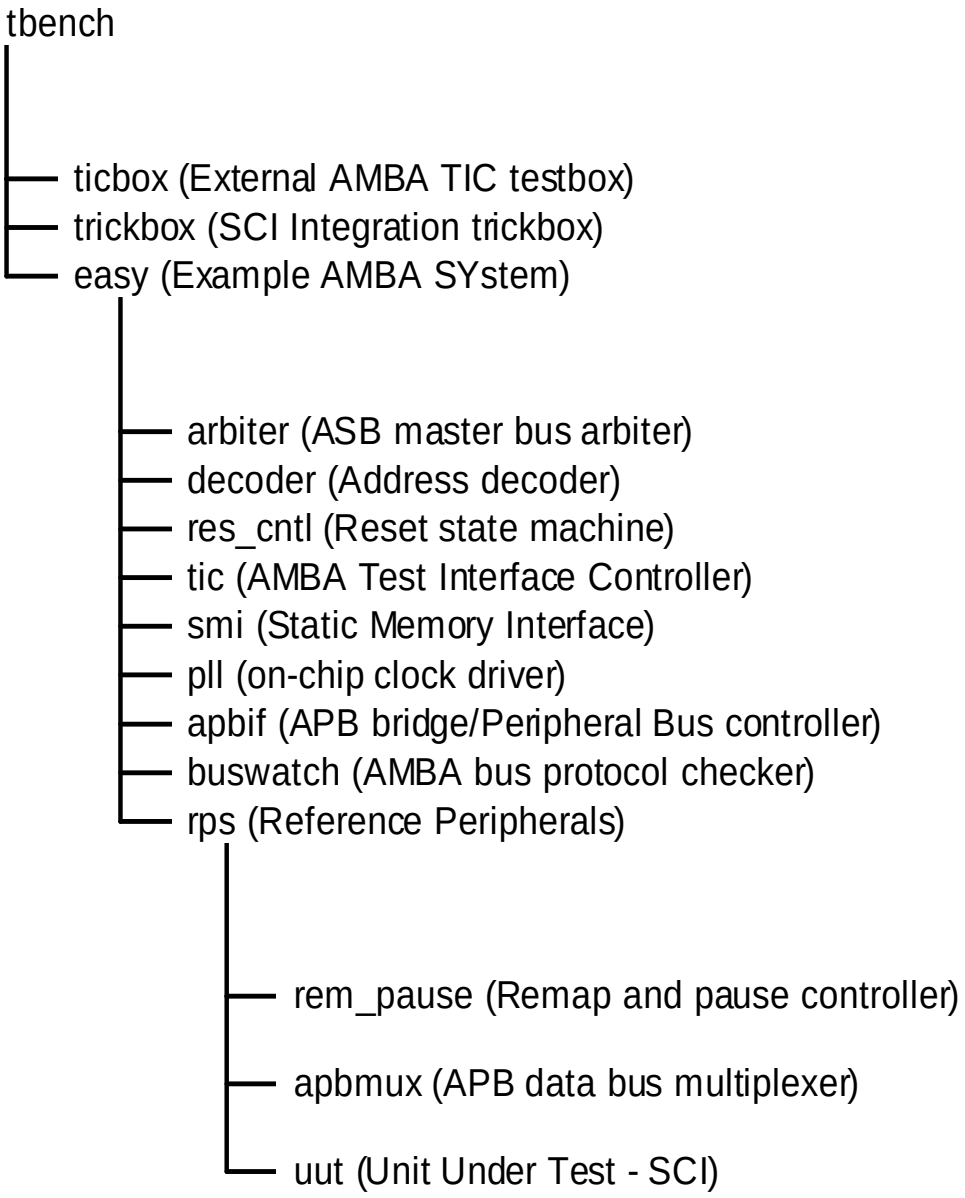


**Figure 4 Testbench hierarchy for simulating TICTalk test vectors**

The test vectors are applied to **tbench.vhd**, the top-level of the testbench that is used for running the integration tests of the SCI. This module instantiates the model of the EASY microcontroller, the integration trickbox and the ticbox. The SCI is instantiated in the rps.vhd, which is a part of the EASY system. The SCICLK is generated in the rps.vhd. All the Scan input pins are tied low to keep them from interfering with the test process.

The integration trickbox is used to connect the primary outputs to the primary inputs either directly or through an exclusive OR logic network.

### 8.1.1 Unit Under Test

The Unit Under Test may be one of the following

- SCI VHDL RTL
- SCI VHDL netlist from VHDL RTL Source

One out of these two versions may be referenced via the corresponding link to uut directory in ***integration/vhdl*** .

### 8.1.2 Test Vectors

The test vectors for the integration testing of the SCI are coded into separate C files for testing the reset values of all the SCI registers and for Integration testing. Make options are provided to run the tests together or individually.

### 8.1.3 Running Simulations

The **README** file in the ***integration/vhdl/tbench*** directory gives step by step instructions for running simulations using the SCI Integration test vectors on the VHDL source code.

Run time logs are available in the following files:

***integration/vhdl/Sci_rtl/Sci_rtl_modelsim.log***

***integration/vhdl/Sci_scanready_vhdl_net_max/Sci_scanready_vhdl_net_max_modelsim.log***

## 8.2 SCI Verilog Integration testbench

The AMBA TICTalk Verilog testbench used for integration testing of the SCI is located in the ***integration/verilog/tbench*** directory. The Integration test vectors are located in the ***integration/bustest/invec*** directory.

The test vectors used with this testbench can be applied to the SCI when it is part of an AMBA system design.

**Figure 5 Testbench for simulating TICTalk test vectors** illustrates the testbench to apply TICTalk test vectors.

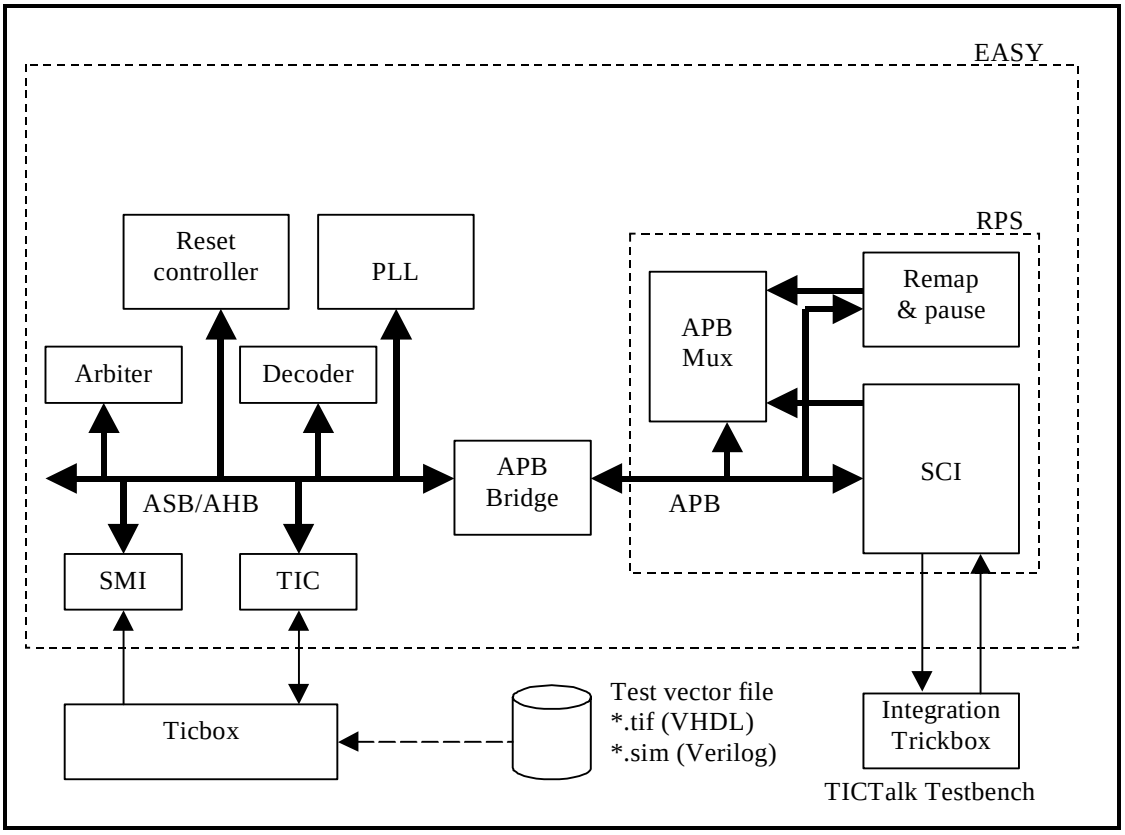


**Figure 5 Testbench for simulating TICTalk test vectors**

The SCI Verilog Integration testbench is a stripped down version of the standard EASY system testbench. For further information refer to the Example AMBA System user guide which is part of ARM Micropack documentation.

**Figure 6 Testbench hierarchy for simulating TICTalk test vectors** illustrates the hierarchy for the TICTalk testbench.

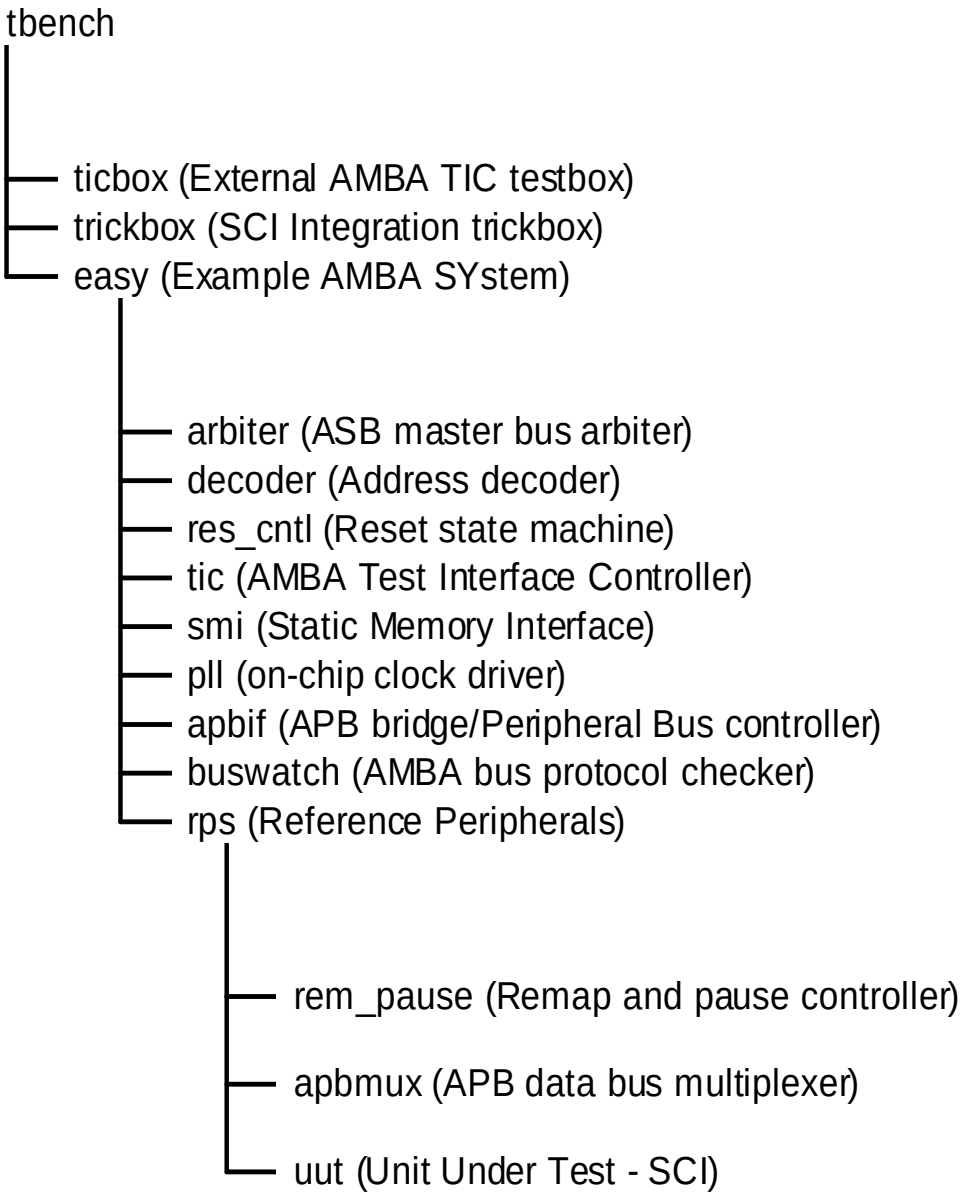


**Figure 6 Testbench hierarchy for simulating TICTalk test vectors**

The test vectors are applied to **tbench.v**, the top-level of the testbench that is used for running the integration tests of the SCI. This module instantiates the model of the EASY microcontroller, the integration trickbox and the ticbox. The SCI is instantiated in the rps.v, which is a part of the EASY system. The SCICLK is generated in the rps.v. All the Scan input pins are tied low to keep them from interfering with the test process.

The integration trickbox is used to connect the primary outputs to the primary inputs either directly or through an exclusive OR logic network.

### 8.2.1 Unit Under Test

The Unit Under Test may be one of the following

- SCI VERILOG RTL
- SCI VERILOG netlist from VHDL RTL Source
- SCI VERILOG netlist from VERILOG RTL Source

One out of these two versions may be referenced via the corresponding link to the uut directory in ***integration/verilog*** .

### 8.2.2 Test Vectors

The test vectors for the integration testing of the SCI are coded into a separate C files for testing the reset values of all the SCI registers and for Integration testing. Make options are provided to run the tests together or individually.

### 8.2.3 Running Simulations

The **README** file in the ***integration/verilog/tbench*** directory gives step by step instructions for running simulations using the SCI Integration test vectors on the VERILOG source code.

Run time logs are available in the following files:

*integration/verilog/Sci_rtl/Sci_rtl_LDV.log*

*integration/verilog/Sci_rtl/Sci_rtl_modelsim.log*

*integration/verilog/Sci_noscan_vhdl_net_min/Sci_noscan_vhdl_net_min_modelsim.log*

integration/verilog/Sci_scanready_verilog_net_typ/Sci_scanready_verilog_net_typ_LDV.log

## 8.3 SCI Integration Trickbox

Please refer back to Section 5.18.2 on Page 58 for details of the SCI Trickbox logic and how it is used to test the Primary Outputs.

## 8.4 Integration Tests

### 8.4.1 Integration Testing of Intra-Chip and Primary Inputs

Please refer back to Section 8.4.1on Page 57 as it covers the test logic and test sequences.

### 8.4.2 Integration Testing of Intra-Chip and Primary Outputs

Please refer back to Section 8.4.2 on Page 58as it covers the test logic and test sequences.